

DevMind AI - Código Completo

Gerado em: 23/06/2026 21:01:41

Total de arquivos: 25

Arquivos incluídos:

1. src/App.tsx
2. src/main.tsx
3. src/analysis/code_analyzer.ts
4. src/classifier/intent_classifier.ts
5. src/conversation/conversation_pt.ts
6. src/database/schema.ts
7. src/database/persistence.ts
8. src/generation/agent_system.ts
9. src/generation/context_engine.ts
10. src/generation/embeddings_engine.ts
11. src/generation/feedback_engine.ts
12. src/generation/grammar_finalizer.ts
13. src/generation/language_model.ts
14. src/generation/phi3_engine.ts
15. src/generation/pos_engine.ts
16. src/generation/response_ranker.ts
17. src/generation/semantic_search.ts
18. src/generation/stop_controller.ts
19. src/knowledge/programming_corpus.ts
20. src/logs/logger.ts
21. src/memory/memory_engine.ts
22. src/memory/user_memory.ts
23. src/security/guardrails.ts
24. src/training/validator.ts
25. src/utils/cn.ts


```

64 %   const AppContext = createContext<{
65 %       state: AppState;
66 %       setState: React.Dispatch<React.SetStateAction<AppState>>;
67 %       orchestrator: AIOrchestrator;
68 %       memoryEngine: MemoryEngine;
69 %       phi3Engine: Phi3Engine;
70 %   } | null>(null);
71 %
72 %   const useApp = () => {
73 %       const ctx = useContext(AppContext);
74 %       if (!ctx) throw new Error('useApp must be used inside Provider');
75 %       return ctx;
76 %   };
77 %
78 %
79 %
80 %   // =====
81 %   // COMPONENTE: Token Stream (Mostra a IA calculando N-Grams, POS
82 %   // =====
83 %
84 %   const TokenStream: React.FC<{ tokens: LiveToken[]; compact?: boolean }> =
85 %       ({ tokens, compact }) => {
86 %           <div className={ `flex flex-wrap gap-1.5 ${compact ? 'p-2.5' :
font-mono text-[10px] md:text-xs max-h-40 overflow-y-auto shadow-inner`} >
87 %               <div className="w-full text-slate-400 mb-2 text-[9px] md:text-
border-slate-800 pb-1.5">
88 %                   <span className="font-bold tracking-wider text-indigo-300
LOOKAHEAD</span>
89 %                       <span className="text-emerald-400 font-semibold">& Poda de
90 %                   </div>
91 %                   {tokens.map((t, i) => {
92 %                       const confidence = t.prob * 100;
93 %                       const lookahead = t.lookaheadScore || 1.0;
94 %                       const attention = t.attentionWeight || 1.0;
95 %                       const posTag = t.postTag || 'Geral';
96 %
97 %                       let colorClass = 'bg-slate-800 text-slate-300';
98 %                       if (posTag === 'Saudacao') colorClass = 'bg-blue-950/60 t
99 %                       else if (posTag === 'SubstantivoTech') colorClass = 'bg-p
100 %                      else if (posTag === 'VerboTech') colorClass = 'bg-indigo-
101 %                      else if (posTag === 'ConectorExplicativo') colorClass = '
border-emerald-700/40';
102 %                      else if (posTag === 'SolucaoBug') colorClass = 'bg-amber-
103 %                      else if (posTag === 'TermoRoblox' || posTag === 'Estrutur
border-rose-700/40';
104 %                      else if (confidence > 60) colorClass = 'bg-slate-700 text
105 %
106 %                      return (
107 %                          <span
108 %                              key={i}
109 %                              className={ `px-2 py-1 rounded-lg ${colorClass} transi
110 %                              title={ `Probabilidade: ${confidence.toFixed(1)}% | Lo
${attention.toFixed(2)} | Classe (POS): ${posTag}` }
111 %                              >
112 %                                  <span className="font-bold">{t.token === '\n' ? '!µ' :
113 %                                  <span className="text-[8px] opacity-60 text-slate-400
border-slate-700/60">{posTag}</span>
114 %                                  {lookahead > 1.1 && <span className="text-[8px] opaci
font-bold">& {lookahead.toFixed(1)}</span>}
115 %                                  </span>
116 %                              );
117 %                      }) }
118 %                  </div>
119 %              );
120 %          };
121 %

```

```

122 % // =====
123 % // COMPONENTE: Badge de Categoria, Intenção e Especialista (MoE)
124 % // =====
125 %
126 % const CategoryBadge: React.FC<{ category: Category; intention?: I
isCorrection?: boolean }> = ({ category, intention, expertName, confidence, isCorrection }) => {
127 %     const colors: Record<string, string> = {
128 %         CONVERSA: 'bg-blue-500/20 text-blue-300 border-blue-500/30',
129 %         PROGRAMAÇÃO: 'bg-violet-500/20 text-violet-300 border-violet-
130 %         ROBLOX: 'bg-rose-500/20 text-rose-300 border-rose-500/30',
131 %         LUAU: 'bg-purple-500/20 text-purple-300 border-purple-500/30',
132 %         JAVASCRIPT: 'bg-yellow-500/20 text-yellow-300 border-yellow-5
133 %         MATEMÁTICA: 'bg-emerald-500/20 text-emerald-300 border-emeral
134 %         PORTUGUÊS: 'bg-cyan-500/20 text-cyan-300 border-cyan-500/30',
135 %         'ERROS COMUNS': 'bg-red-500/20 text-red-300 border-red-500/30',
136 %         DOCUMENTAÇÃO: 'bg-indigo-500/20 text-indigo-300 border-indigo
137 %         IA: 'bg-amber-500/20 text-amber-300 border-amber-500/30',
138 %     };
139 %     const color = colors[category] || 'bg-slate-500/20 text-slate-3
140 %     return (
141 %         <div className="flex flex-wrap items-center gap-1.5">
142 %             {expertName && (
143 %                 <span className="bg-gradient-to-r from-indigo-500/30 to-p
py-0.5 rounded-full text-[10px] font-bold tracking-wide flex items-center gap-1 shadow-sm">
144 %                     ⌘ {expertName}
145 %                 </span>
146 %             )}
147 %             <span className={`inline-flex items-center gap-1 px-2.5 py-
148 %                 {category}
149 %                 {confidence !== undefined && (
150 %                     <span className="opacity-75 font-bold">{(confidence * 1
151 %                 )}
152 %                 </span>
153 %                 {intention && (
154 %                     <span className="bg-slate-700/60 text-slate-300 border bo
tracking-wide capitalize">
155 %                         ⌘ {intention}
156 %                     </span>
157 %                 )}
158 %                 {isCorrection && (
159 %                     <span className="bg-amber-500/20 text-amber-300 border bo
tracking-wide flex items-center gap-1 animate-pulse">
160 %                         ⌘ Auto-Correção Ativa
161 %                     </span>
162 %                 )}
163 %             </div>
164 %         );
165 %     };
166 %
167 % // =====
168 % // COMPONENTE: Mensagem de Chat (Botoes Like, Dislike, Super Like
169 % // =====
170 %
171 % const MessageView: React.FC<{
172 %     message: Message;
173 %     feedbackInfo?: { state: 'liked' | 'disliked' | 'superliked'; te
174 %     correctedByPhi3?: string;
175 %     onLike?: () => void;
176 %     onDislike?: () => void;
177 %     onSuperLike?: () => void;
178 %     onPhi3Correct?: () => void;
179 %     phi3Loading?: boolean;
180 % }> = ({ message, feedbackInfo, correctedByPhi3, onLike, onDislike
181 %     const isUser = message.role === 'user';
182 %     const stateVal = feedbackInfo?.state;
183 %     const [showOriginal, setShowOriginal] = useState(false);

```

```

184 %
185 %         return (
186 %             <div className={`flex ${isUser ? 'justify-end' : 'justify-start'}
187 %                 <div className={`max-w-[90%] md:max-w-[75%] rounded-2xl px-
188 %                     isUser ? 'bg-indigo-600 text-white shadow-lg shadow-indigo
text-slate-100 shadow-md'
189 %                 }`}>
190 %                     {message.category && !isUser && (
191 %                         <div className="mb-2">
192 %                             <CategoryBadge
193 %                                 category={message.category as Category}
194 %                                 intention={message.intention as IntentionType}
195 %                                 expertName={message.expertName as ExpertType}
196 %                                 isCorrection={message.isCorrection}
197 %                             />
198 %                         </div>
199 %                     )}
200 %                     { /* Resposta corrigida pelo Phi-3 (se existir) */ }
201 %                     {correctedByPhi3 && (
202 %                         <div className="mt-2 p-3 bg-violet-950/50 border border
203 %                             <div className="flex items-center justify-between">
204 %                                 <span className="font-bold text-violet-300 flex ite
205 %                                     Ø=Ý, Phi-3 Mini corrigiu:
206 %                                 </span>
207 %                                 <button
208 %                                     onClick={() => setShowOriginal(v => !v)}
209 %                                     className="text-slate-500 hover:text-slate-300 te
210 %                                 >
211 %                                     {showOriginal ? 'ocultar original' : 'ver original'
212 %                                 </button>
213 %                             </div>
214 %                             <p className="text-sm whitespace-pre-wrap leading-rel
215 %                                 {showOriginal && (
216 %                                     <p className="text-[11px] text-slate-400 font-mono
217 %                                         N-Gram original: {message.content}
218 %                                     </p>
219 %                                 )}
220 %                             </div>
221 %                         )}
222 %                     {!correctedByPhi3 && (
223 %                         <p className="text-sm whitespace-pre-wrap leading-relax
224 %                     )}
225 %
226 %                     { /* Status Real da Fila de Aprendizizado, Revisor B e Score
227 %                     <div className="text-[10px] opacity-75 mt-2 flex flex-wra
border-slate-700/40">
228 %                         <span>{new Date(message.timestamp).toLocaleTimeString([
229 %                         {!isUser && (
230 %                             <div className="flex flex-wrap gap-2 items-center">
231 %                                 { /* Score Calculado */ }
232 %                                 {feedbackInfo?.textInfo ? (
233 %                                     <span className="bg-slate-900 text-emerald-400 px
234 %                                         {feedbackInfo.textInfo}
235 %                                     </span>
236 %                                 ) : (
237 %                                     <span className="bg-slate-900 text-slate-400 px-2
238 %                                         Score Atual: 85% (Likes: 0, Dislikes: 0)
239 %                                     </span>
240 %                                 )}
241 %
242 %                                 { /* Fila de Aprovação (Threshold de 10 likes) */ }
243 %                                 <span className={`px-2 py-0.5 rounded font-mono font
244 %                                     stateVal === 'superliked' ? 'bg-amber-500/20 text
245 %                                     stateVal === 'liked' ? 'bg-emerald-500/20 text-em
246 %                                     stateVal === 'disliked' ? 'bg-rose-500/20 text-ro
247 %                                     'bg-slate-900 text-slate-400 border border-slate-

```

```
2 4 8 %           } ` } >
2 4 9 %           { stateVal === 'superliked' ? '+P Fila: Aprovada GL
2 5 0 %           stateVal === 'liked' ? '0=ÜM Fila: Approved (Camin
2 5 1 %           stateVal === 'disliked' ? '0=ÜN Fila: Rejected (Nã
2 5 2 %           '#ó Fila: Neutral (Aguarda 10 Likes) ' }
2 5 3 %         < / span >
2 5 4 %
2 5 5 %           { / * Métrica de Revisão Interna (Agente B) * / }
2 5 6 %           { message.score !== undefined && (
2 5 7 %             < span className="bg-slate-900 text-indigo-300 px-
items-center gap-1">
2 5 8 %                 0=báp Agente B: Aprovado (Qualidade {(message.sco
2 5 9 %                 < / span >
2 6 0 %             ) }
2 6 1 %         < / div >
2 6 2 %       ) }
2 6 3 %     < / div >
2 6 4 %
2 6 5 %     { !isUser && (
2 6 6 %       < div className="flex items-center gap-1.5 mt-2.5 pt-2 b
2 6 7 %       { / * BOTÃO PHI-3 MINI - Correção local sem API * / }
2 6 8 %       < button
2 6 9 %         onClick={onPhi3Correct}
2 7 0 %         disabled={phi3Loading}
2 7 1 %         className={ `px-2.5 py-1 rounded-lg text-xs font-sem
2 7 2 %           phi3Loading
2 7 3 %             ? 'bg-violet-900/40 text-violet-300 border bord
2 7 4 %             : correctedByPhi3
2 7 5 %             ? 'bg-violet-600/30 text-violet-200 border bo
2 7 6 %             : 'bg-violet-800/20 hover:bg-violet-700/40 te
2 7 7 %           } ` }
2 7 8 %           title="Ativar Phi-3 Mini local (sem API) para corri
2 7 9 %         >
2 8 0 %           0=Y, { phi3Loading ? 'Phi-3 Processando...' : correcte
2 8 1 %         < / button >
2 8 2 %         < button
2 8 3 %           onClick={onLike}
2 8 4 %           disabled={stateVal === 'liked' || stateVal === 'sup
2 8 5 %           className={ `px-2.5 py-1 rounded-lg text-xs font-med
2 8 6 %           stateVal === 'liked' ? 'bg-emerald-600/30 text-em
hover:bg-emerald-600/40 text-slate-300'
2 8 7 %           } ` }
2 8 8 %           title="Aprovar resposta na Fila de Aprendizado (+1
2 8 9 %         >
2 9 0 %           0=ÜM { stateVal === 'liked' ? 'Approved (+1)' : 'Like'
2 9 1 %         < / button >
2 9 2 %         < button
2 9 3 %           onClick={onSuperLike}
2 9 4 %           disabled={stateVal === 'superliked' }
2 9 5 %           className={ `px-2.5 py-1 rounded-lg text-xs font-sem
2 9 6 %           stateVal === 'superliked' ? 'bg-amber-500/30 text
animate-pulse' : 'bg-amber-600/20 hover:bg-amber-500/40 text-amber-300 border border-amber-500/40'
2 9 7 %           } ` }
2 9 8 %           title="Super Like! Reforça o peso na Fila em +10 Li
2 9 9 %         >
3 0 0 %           +P { stateVal === 'superliked' ? 'Promovido Global!'
3 0 1 %         < / button >
3 0 2 %         < button
3 0 3 %           onClick={onDislike}
3 0 4 %           disabled={stateVal === 'disliked' }
3 0 5 %           className={ `px-2.5 py-1 rounded-lg text-xs font-med
3 0 6 %           stateVal === 'disliked' ? 'bg-rose-600/30 text-ro
hover:bg-rose-600/40 text-slate-300'
3 0 7 %           } ` }
3 0 8 %           title="Rejeitar resposta na Fila de Aprendizado (-1
3 0 9 %         >
```

```

310 %             Ø=ÜN { stateVal === 'disliked' ? 'Rejected (-1)' : 'Disliked' }
311 %         </button>
312 %     </div>
313 %     )}
314 % </div>
315 % </div>
316 % );
317 % };
318 %
319 % // =====
320 % // COMPONENTE: Chat Principal
321 % // =====
322 %
323 % const ChatInterface: React.FC = () => {
324 %     const { state, setState, orchestrator, memoryEngine, phi3Engine } = useChatEngine();
325 %     const [input, setInput] = useState('');
326 %     const chatEndRef = useRef<HTMLDivElement>(null);
327 %
328 %     useEffect(() => {
329 %         chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
330 %     }, [state.messages, state.liveTokens, state.agentBStatus, state.cotLiveStatus]);
331 %
332 %     const handleSend = async () => {
333 %         if (!input.trim() || state.isGenerating || !state.currentUser) return;
334 %
335 %         const userText = input.trim();
336 %
337 %         // Guardrail de Segurança
338 %         const safetyCheck = orchestrator.safety.verifyPrompt(userText);
339 %         const processedText = safetyCheck.redactedText;
340 %
341 %         const userMsg: Message = {
342 %             userId: state.currentUser.uuid,
343 %             role: 'user',
344 %             content: processedText,
345 %             timestamp: new Date(),
346 %         };
347 %
348 %         userMsg.id = await memoryEngine.saveUserMessage(userMsg);
349 %
350 %         if (safetyCheck.isSafe) {
351 %             await memoryEngine.extractAndSaveFacts(state.currentUser.uuid, userMsg.id);
352 %             await orchestrator.persistenceGuard.createMirrorBackup();
353 %         }
354 %
355 %         setState(s => ({
356 %             ...s,
357 %             messages: [...s.messages, userMsg],
358 %             isGenerating: true,
359 %             liveTokens: [],
360 %             lastStopReason: null,
361 %             agentBStatus: null,
362 %             cotLiveStatus: 'Ø>Yà [Etapa 1] Classificando intenção e extração de fatos',
363 %             hallucinationWarning: null,
364 %             selfEvalResult: null,
365 %         }));
366 %         setInput('');
367 %
368 %         try {
369 %             if (!safetyCheck.isSafe) {
370 %                 const warning = `[ALERTA DE SEGURANÇA]: A mensagem enviada
A operação foi cancelada por segurança.`;
371 %                 const aiMsg: Message = {
372 %                     userId: state.currentUser.uuid,
373 %                     role: 'assistant',
374 %                     content: warning,

```

```

375 %         category: 'CONVERSA',
376 %         intention: 'informacao',
377 %         expertName: 'Especialista Conversa',
378 %         timestamp: new Date(),
379 %     };
380 %     aiMsg.id = await memoryEngine.saveUserMessage(aiMsg);
381 %     setState(s => ({
382 %         ...s,
383 %         messages: [...s.messages, aiMsg],
384 %         isGenerating: false,
385 %         lastStopReason: 'seguranca',
386 %         cotLiveStatus: null
387 %     }));
388 %     return;
389 % }
390 %
391 % // Perfil de memória individual isolada (ETAPAS 3 e 4)
392 % const memoryProfile = await memoryEngine.loadUserProfile(st
393 %
394 % // PIPELINE ORQUESTADOR DEFINITIVO (Com MoE e Entidades)
395 % const { result, promptForGenerator, primaryCategory } = awa
396 %     processedText,
397 %     memoryProfile,
398 %     state.currentUser.uuid
399 % );
400 %
401 % setState(s => ({
402 %     ...s,
403 %     lastClassification: {
404 %         category: result.category,
405 %         intention: result.intention,
406 %         expertName: result.expertName,
407 %         confidence: result.confidence,
408 %         signals: result.signals,
409 %         subject: result.subject,
410 %         entities: result.entities,
411 %         contextVector: result.contextVector,
412 %         isCorrectionLoop: result.isCorrectionLoop,
413 %         persistenceState: result.persistenceState
414 %     }
415 % }));
416 %
417 % // Temperatura adaptativa
418 % const adaptiveTemperature = (() => {
419 %     const creativeCats: Category[] = ['CONVERSA', 'PORTUGUÊS'
420 %     const preciseCats: Category[] = ['PROGRAMAÇÃO', 'ROBLOX',
421 %     if (creativeCats.includes(primaryCategory)) return Math.m
422 %     if (preciseCats.includes(primaryCategory)) return Math.mi
423 %     return state.temperature;
424 % })();
425 %
426 % // GERAÇÃO ASSÍNCRONA COM REESCRITOR E FINALIZADOR DE FRASE
427 % let generated = '';
428 % let activeTokens: LiveToken[] = [];
429 % const stopController = new GenerationStopController(primary
430 % let stopReason: string | null = null;
431 % let finalReviewQuality = 0.85;
432 % let hallucinationWarningFound: string | null = null;
433 % let selfEvalFound: any = null;
434 %
435 % for await (const step of orchestrator.agentSystem.generateW
436 %     promptForGenerator,
437 %     processedText,
438 %     result.searchResults,
439 %     primaryCategory,
440 %     result.intention,

```



```

441 %         result.expertName,
442 %         55,
443 %         adaptiveTemperature,
444 %         memoryProfile.interestsBiases,
445 %         result.entities,
446 %         memoryProfile.profile.gostaDe.length,
447 %         result.subject // Palavra Forte TF-IDF
448 %     )) {
449 %         if (step.type === 'cot' && step.cotMessage) {
450 %             setState(s => ({ ...s, cotLiveStatus: step.cotMessage |
451 %
452 %
453 %         if (step.type === 'token' && step.token) {
454 %             const tk = step.token;
455 %             const beforeDecision = stopController.evaluate(tk, gene
456 %             if (!beforeDecision.accept || beforeDecision.stop) {
457 %                 stopReason = beforeDecision.reason || 'controle_de_pa
458 %                 break;
459 %             }
460 %
461 %             const isPunct = /^[.,?!:;]$/ .test(tk.token);
462 %             if (generated.length > 0 && !isPunct && tk.token !== '\
463 %                 generated += ' ';
464 %             }
465 %             generated += tk.token === '\n' ? ' ' : tk.token;
466 %             activeTokens.push(tk);
467 %             setState(s => ({ ...s, liveTokens: [...activeTokens] }));
468 %
469 %             const afterDecision = stopController.recordAccepted(tk.
470 %             if (afterDecision.stop) {
471 %                 stopReason = afterDecision.reason || 'resposta_comple
472 %                 break;
473 %             }
474 %         }
475 %
476 %         if (step.type === 'review' && step.review) {
477 %             finalReviewQuality = step.review.qualityScore;
478 %             hallucinationWarningFound = step.review.hallucinationWa
479 %             selfEvalFound = step.review.selfEvaluation || null;
480 %
481 %             if (!step.review.approved) {
482 %                 setState(s => ({
483 %                     ...s,
484 %                     agentBStatus: `&p Agente B recusou primeira tentati
profunda...`,
485 %                     cotLiveStatus: `0=Ý [Etapa 8] Agente B ordenou refor
486 %                 }));
487 %                 generated = '';
488 %                 activeTokens = [];
489 %             } else {
490 %                 setState(s => ({
491 %                     ...s,
492 %                     agentBStatus: `0=þáþ ETAPA 8 & 9 (Revisor + Finalizad
${(step.review?.qualityScore || 0.85) * 100}%`,
493 %                     cotLiveStatus: `'( [Etapa 9 & 10] Frase verificada c
494 %                 }));
495 %             }
496 %         }
497 %
498 %         if (step.type === 'retry') {
499 %             setState(s => ({
500 %                 ...s,
501 %                 agentBStatus: `0=Ý Agente B insatisfeito com a respost
${result.expertName}...`,
502 %                 cotLiveStatus: `&mp [Etapa 8] Recalculando transições
0.4).toFixed(2)}))...`

```

```

5 0 3 %         } ) ) ;
5 0 4 %     }
5 0 5 %
5 0 6 %         if ( step.type === 'final' && step.generatedText ) {
5 0 7 %             generated = step.generatedText;
5 0 8 %         }
5 0 9 %     }
5 1 0 %
5 1 1 %     generated = generated.trim();
5 1 2 %     generated = orchestrator.safety.moderateOutput(generated);
5 1 3 %
5 1 4 %     if (!generated) {
5 1 5 %         generated = 'Não consegui gerar uma resposta com a base a
conhecimento.';
5 1 6 %     }
5 1 7 %
5 1 8 %     // Rankeia com Context Vector
5 1 9 %     const ranking = rankResponse(generated, activeTokens, process
5 2 0 %     const finalScore = Math.max(ranking.totalScore, finalReview
5 2 1 %
5 2 2 %     const aiMsg: Message = {
5 2 3 %         userId: state.currentUser.uuid,
5 2 4 %         role: 'assistant',
5 2 5 %         content: generated,
5 2 6 %         category: primaryCategory,
5 2 7 %         intention: result.intention,
5 2 8 %         expertName: result.expertName,
5 2 9 %         contextVector: JSON.stringify(result.contextVector),
5 3 0 %         score: finalScore,
5 3 1 %         isCorrection: result.isCorrectionLoop,
5 3 2 %         timestamp: new Date(),
5 3 3 %     };
5 3 4 %
5 3 5 %     aiMsg.id = await memoryEngine.saveUserMessage(aiMsg);
5 3 6 %
5 3 7 %     const responseKey = String(aiMsg.id ?? `idx-${state.message
5 3 8 %
5 3 9 %     // Coloca automaticamente a resposta gerada na Fila de Apre
Global)
5 4 0 %     await orchestrator.feedbackEngine.registerNeutral(
5 4 1 %         responseKey,
5 4 2 %         state.currentUser.uuid,
5 4 3 %         state.currentUser.name,
5 4 4 %         processedText,
5 4 5 %         generated,
5 4 6 %         primaryCategory,
5 4 7 %         result.intention,
5 4 8 %         result.contextVector
5 4 9 %     );
5 5 0 %
5 5 1 %     const stats = orchestrator.model.getStats();
5 5 2 %
5 5 3 %     setState(s => ({
5 5 4 %         ...s,
5 5 5 %         messages: [...s.messages, aiMsg],
5 5 6 %         isGenerating: false,
5 5 7 %         lastStopReason: stopReason,
5 5 8 %         agentBStatus: null,
5 5 9 %         cotLiveStatus: null,
5 6 0 %         hallucinationWarning: hallucinationWarningFound,
5 6 1 %         selfEvalResult: selfEvalFound,
5 6 2 %         modelStats: {
5 6 3 %             vocab: stats.vocabSize,
5 6 4 %             words: stats.totalWords,
5 6 5 %             categories: stats.trainedCategories,
5 6 6 %         }

```

```

567 %         }));
568 %
569 %         auditLog('AGENT_B_FINAL_SUCCESS', {
570 %             userId: state.currentUser.uuid,
571 %             stopReason,
572 %             intention: result.intention,
573 %             expert: result.expertName,
574 %             agentBQuality: finalReviewQuality.toFixed(2),
575 %             totalScore: finalScore.toFixed(2),
576 %         });
577 %     } catch (e) {
578 %         console.error(e);
579 %         setState(s => ({ ...s, isGenerating: false, agentBStatus: n
selfEvalResult: null }));
580 %     }
581 % };
582 %
583 %     const findPreviousUserMessage = (assistantIndex: number) => {
584 %         for (let i = assistantIndex - 1; i >= 0; i--) {
585 %             if (state.messages[i]?.role === 'user') return state.messages[i];
586 %         }
587 %         return null;
588 %     };
589 %
590 %     const getFeedbackKey = (message: Message, index: number) => String(index);
591 %
592 %     const [phi3LoadingKey, setPhi3LoadingKey] = useState<string | null>('');
593 %
594 %     /**
595 %      * Aciona o Phi-3 Mini local para corrigir/pesquisar a resposta
596 %      */
597 %     const handlePhi3Correct = async (message: Message, index: number) => {
598 %         if (message.role !== 'assistant') return;
599 %         const question = findPreviousUserMessage(index);
600 %         const key = getFeedbackKey(message, index);
601 %         const phi3 = phi3Engine;
602 %
603 %         // 1. Se o motor ainda não está pronto, baixa o modelo
604 %         if (!phi3.isReady()) {
605 %             try {
606 %                 await phi3.load();
607 %                 if (!phi3.isReady()) return;
608 %             } catch (err: any) {
609 %                 setState(s => ({ ...s, cotLiveStatus: 'L Phi-3 Mini falhou ao carregar' }));
610 %                 return;
611 %             }
612 %         }
613 %
614 %         setPhi3LoadingKey(key);
615 %         setState(s => ({ ...s, cotLiveStatus: 'O=Ÿ, Phi-3 Mini analisando' }));
616 %
617 %         try {
618 %             const category = (message.category || 'CONVERSA') as string;
619 %             const subject = state.lastClassification?.subject || 'geral';
620 %             const userMsg = question?.content || message.content;
621 %
622 %             // Se há pergunta do usuário, usa research. Senão, usa correctResponse
623 %             const result = question
624 %                 ? await phi3.correctResponse(userMsg, message.content, category)
625 %                 : await phi3.research(message.content, category, subject);
626 %
627 %             setState(s => ({
628 %                 ...s,
629 %                 phi3CorrectedMessages: { ...s.phi3CorrectedMessages, [key]: result },
630 %                 cotLiveStatus: `Phi-3 Mini (${result.usedBackend}) corrigido com
${(result.confidence * 100).toFixed(0)}%`,

```

```

631 %         }));
632 %     } catch (err: any) {
633 %         setState(s => ({
634 %             ...s,
635 %             cotLiveStatus: `!L Phi-3 Mini: ${err?.message} || 'erro des
636 %         }));
637 %     } finally {
638 %         setPhi3LoadingKey(null);
639 %     }
640 % };
641 %
642 % / * *
643 %  * Processador Global de Feedback (Fila de Aprendizado com Thre
644 %  * /
645 % const handleFeedback = async (kind: 'like' | 'dislike' | 'super
646 %     if (!state.currentUser || message.role !== 'assistant') retur
647 %     const question = findPreviousUserMessage(index);
648 %     if (!question) return;
649 %
650 %     const category = (message.category || 'CONVERSA') as Category
651 %     const intention = (message.intention || 'informacao') as Inte
652 %     const key = getFeedbackKey(message, index);
653 %
654 %     const contextVector = message.contextVector
655 %         ? orchestrator.contextEngine.deserializeVector(message.cont
656 %           : orchestrator.contextEngine.buildContextVector(question.co
Conversa', confidence: 1, signals: [], subject: 'geral', entities: { nomes: [], jogos: [], linguagens: [], objetos: [], funcoes: [],
erros: [], numeros: [] } }, []));
657 %
658 %     if (!contextVector) return;
659 %
660 %     if (kind === 'superlike') {
661 %         const passwordEntered = prompt('Digite a senha de autorizaç
662 %         if (passwordEntered === null) return;
663 %
664 %         const result = await orchestrator.feedbackEngine.registerLi
665 %             key,
666 %             state.currentUser.uuid,
667 %             state.currentUser.name,
668 %             question.content,
669 %             message.content,
670 %             category,
671 %             intention,
672 %             contextVector,
673 %             true, // isSuperLike = true
674 %             passwordEntered
675 %         );
676 %
677 %         if (!result.success) {
678 %             alert(`!L Falha: ${result.message}`);
679 %             return;
680 %         }
681 %
682 %         alert(`+P ${result.message}\n${result.scoreInfo}`);
683 %         setState(s => ({ ...s, feedbackStatus: { ...s.feedbackStatu
}));
684 %         return;
685 %     }
686 %
687 %     if (kind === 'like') {
688 %         const result = await orchestrator.feedbackEngine.registerLi
689 %             key,
690 %             state.currentUser.uuid,
691 %             state.currentUser.name,
692 %             question.content,
693 %             message.content,

```

```

694 %         category,
695 %         intention,
696 %         contextVector,
697 %         false // isSuperLike = false
698 %     );
699 %     alert(`ðŰM  ${result.message}\n${result.scoreInfo}`);
700 %     setState(s => ({ ...s, feedbackStatus: { ...s.feedbackStatus
701 %         return;
702 %     }
703 %
704 %     if (kind === 'dislike') {
705 %         const result = await orchestrator.feedbackEngine.registerDi
706 %             key,
707 %             state.currentUser.uuid,
708 %             state.currentUser.name,
709 %             question.content,
710 %             message.content,
711 %             category,
712 %             intention,
713 %             contextVector
714 %         );
715 %         alert(`ðŰN  ${result.message}\n${result.scoreInfo}`);
716 %         setState(s => ({ ...s, feedbackStatus: { ...s.feedbackStatus
717 %             return;
718 %         }
719 %     });
720 %
721 %     return (
722 %         <div className="flex flex-col h-full bg-slate-950 text-white
723 %             { /* HEADER PRINCIPAL COM STATUS DA PERSISTÊNCIA BLINDADA */
724 %             <div className="px-3 py-2.5 md:px-4 md:py-3 border-b border
shadow-sm">
725 %                 <div className="flex items-center gap-2.5 min-w-0 flex-1"
726 %                     <button
727 %                         onClick={() => setState(s => ({ ...s, showSidebar: !s
728 %                         className="md:hidden p-1.5 rounded-lg hover:bg-slate-
729 %                     >
730 %                         <svg width="18" height="18" viewBox="0 0 24 24" fill=
731 %                             <line x1="3" y1="12" x2="21" y2="12" />
732 %                             <line x1="3" y1="6" x2="21" y2="6" />
733 %                             <line x1="3" y1="18" x2="21" y2="18" />
734 %                         </svg>
735 %                     </button>
736 %                     <div className="min-w-0">
737 %                         <div className="flex items-center gap-2">
738 %                             <span className="text-lg md:text-xl">ðŰà</span>
739 %                             <h2 className="font-bold text-sm md:text-base trunc
740 %                             {state.lastClassification && (
741 %                                 <CategoryBadge
742 %                                     category={state.lastClassification.category}
743 %                                     intention={state.lastClassification.intention}
744 %                                     expertName={state.lastClassification.expertName
745 %                                     isCorrection={state.lastClassification.isCorrec
746 %                                 </>
747 %                             )}
748 %                         </div>
749 %                         <div className="flex flex-wrap items-center gap-2 mt-
750 %                             <p className="text-[10px] md:text-xs text-slate-400
751 %                                 Usu rio Ativo: <strong className="text-slate-200"
className="font-mono text-indigo-400">{state.currentUser?.uuid.split('-')[0]}</span>
752 %                             </p>
753 %                             {state.lastClassification?.persistenceState && (
754 %                                 <span className="bg-emerald-950/80 text-emerald-3
font-bold font-mono shadow-sm">
755 %                                     {state.lastClassification.persistenceState}

```

```

7 5 6 %                 < / s p a n >
7 5 7 %                 ) }
7 5 8 %                 < / d i v >
7 5 9 %                 < / d i v >
7 6 0 %                 < / d i v >
7 6 1 %                 < d i v   c l a s s N a m e = " f l e x   i t e m s - c e n t e r   g a p - 2 " >
7 6 2 %                 < b u t t o n
7 6 3 %                     o n C l i c k = { ( )   =>   s e t S t a t e ( s   =>   ( {   . . . s ,   s h o w D e b u g :   ! s . s
7 6 4 %                     c l a s s N a m e = { ` p x - 3   p y - 1 . 5   r o u n d e d - l g   t e x t - x s   f o n t - s e m i b
text-white border-indigo-500 shadow-sm' : 'bg-slate-800 hover:bg-slate-700 text-slate-300 border-slate-700'}}}
7 6 5 %                     t i t l e = " E x i b i r   i n s p e t o r   d e   p r e d i ç ã o ,   P O S ,   A g e n t e   B ,   F i
7 6 6 %                     >
7 6 7 %                     Ø=Ý   I n s p e t o r
7 6 8 %                     < / b u t t o n >
7 6 9 %                 < / d i v >
7 7 0 %                 < / d i v >
7 7 1 %
7 7 2 %                 { / *   I N S P E T O R   D E   D E B U G   E   A G E N T E   B   * / }
7 7 3 %                 { s t a t e . s h o w D e b u g   &&   (
7 7 4 %                 < d i v   c l a s s N a m e = " b g - s l a t e - 9 0 0 / 9 0   b o r d e r - b   b o r d e r - s l a t e - 8 0 0
backdrop-blur-sm shadow-inner">
7 7 5 %                 < d i v   c l a s s N a m e = " f l e x   f l e x - w r a p   g a p - 4   b o r d e r - b   b o r d e r - s l
7 7 6 %                 < s p a n >Ø=ÚÚ   V o c a b u l á r i o   N - G r a m :   < b   c l a s s N a m e = " t e x t - e m e r a
7 7 7 %                 < s p a n >Ø=ÚÝ   T o t a l   d e   T o k e n s :   < b   c l a s s N a m e = " t e x t - e m e r a l d -
7 7 8 %                 < s p a n >Ø<ß÷þ   C a t e g o r i a s   T r e i n a d a s :   < b   c l a s s N a m e = " t e x t - e m
7 7 9 %                 < / d i v >
7 8 0 %                 { s t a t e . l a s t C l a s s i f i c a t i o n   &&   (
7 8 1 %                 < d i v   c l a s s N a m e = " s p a c e - y - 1 . 5   b g - s l a t e - 9 5 0   p - 3   r o u n d e d -
7 8 2 %                 < d i v   c l a s s N a m e = " t e x t - i n d i g o - 4 0 0   f o n t - b o l d   u p p e r c a s e
TF-IDF (10 Etapas)</div>
7 8 3 %                 < d i v >Ø=ÚQ   E s p e c i a l i s t a   M o E :   < s p a n   c l a s s N a m e = " t e x t - e m e
font-bold">{state.lastClassification.expertName}</span></div>
7 8 4 %                 < d i v >Ø<ß˘   I n t e n ç ã o   D e t e c t a d a :   < s p a n   c l a s s N a m e = " t e x t - a
capitalize">{state.lastClassification.intention}</span></div>
7 8 5 %                 < d i v >Ø=ÚĬ   P a l a v r a   F o r t e   ( T F - I D F ) :   < s p a n   c l a s s N a m e = " t e
capitalize">{state.lastClassification.subject}</span></div>
7 8 6 %
7 8 7 %                 { / *   E T A P A   2 :   E x i b i ç ã o   d e   E n t i d a d e s   E x t r a í d a s   * / }
7 8 8 %                 < d i v   c l a s s N a m e = " p t - 1   b o r d e r - t   b o r d e r - s l a t e - 8 0 0 / 8 0   m
7 8 9 %                 < d i v   c l a s s N a m e = " t e x t - [ 1 0 p x ]   f o n t - b o l d   t e x t - i n d i g o
Extraídas:</div>
7 9 0 %                 < d i v   c l a s s N a m e = " f l e x   f l e x - w r a p   g a p - 2   t e x t - [ 1 0 p x ] "
7 9 1 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . e r r o s . l e n g t h
className="text-rose-400">{state.lastClassification.entities.erros.join(', ')}</b></span>}
7 9 2 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . l i n g u a g e n s . l
className="text-cyan-400">{state.lastClassification.entities.linguagens.join(', ')}</b></span>}
7 9 3 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . j o g o s . l e n g t h
className="text-amber-400">{state.lastClassification.entities.jogos.join(', ')}</b></span>}
7 9 4 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . o b j e t o s . l e n g
className="text-indigo-400">{state.lastClassification.entities.objetos.join(', ')}</b></span>}
7 9 5 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . f u n c o e s . l e n g
className="text-emerald-400">{state.lastClassification.entities.funcoes.join(', ')}</b></span>}
7 9 6 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e s . n u m e r o s . l e n g
className="text-purple-400">{state.lastClassification.entities.numeros.join(', ')}</b></span>}
7 9 7 %                 { O b j e c t . v a l u e s ( s t a t e . l a s t C l a s s i f i c a t i o n . e n t i t i e
className="text-slate-500 italic">Nenhuma entidade estrita isolada na frase.</span>}
7 9 8 %                 < / d i v >
7 9 9 %                 < / d i v >
8 0 0 %
8 0 1 %                 { s t a t e . l a s t C l a s s i f i c a t i o n . c o n t e x t V e c t o r   &&   (
8 0 2 %                 < d i v   c l a s s N a m e = " t e x t - [ 1 0 p x ]   t e x t - s l a t e - 4 0 0   p t - 1   b
8 0 3 %                 < s p a n   c l a s s N a m e = " t e x t - i n d i g o - 3 0 0 " > V e t o r   d e   C o n t
{JSON.stringify(state.lastClassification.contextVector)}
8 0 4 %                 < / d i v >
8 0 5 %                 ) }
8 0 6 %                 < / d i v >

```

```
8 0 7 %           ) }
8 0 8 %           { state . agent B Status && (
8 0 9 %             < div   className = " flex   items - center   gap - 1 . 5   bg - indigo - 9
font-sans font-medium">
8 1 0 %                 < span > { state . agent B Status } < / span >
8 1 1 %                 < / div >
8 1 2 %           ) }
8 1 3 %           { state . last Stop Reason && (
8 1 4 %             < div   className = " flex   items - center   gap - 1 . 5   bg - rose - 9 5 0
8 1 5 %             < span > 0 = ðŸ”” Disparo de Parada de Geraçã o : < / span > < stro
text-white">{state.lastStopReason}</strong>
8 1 6 %             < / div >
8 1 7 %           ) }
8 1 8 %           < div   className = " flex   items - center   gap - 3   pt - 1 " >
8 1 9 %             < label   className = " text - slate - 3 0 0   font - sans   font - mediu
8 2 0 %             < input
8 2 1 %               type = " range "   min = " 0 . 1 "   max = " 1 . 5 "   step = " 0 . 1 "
8 2 2 %               value = { state . temperature }
8 2 3 %               onChange = { e =>   setState ( s =>   ( { . . . s ,   temperature : p
8 2 4 %               className = " flex - 1   accent - indigo - 5 0 0   max - w - [ 2 0 0 px ] "
8 2 5 %             / >
8 2 6 %             < span   className = " text - indigo - 4 0 0   font - bold   text - xs " > {
8 2 7 %             < / div >
8 2 8 %           < / div >
8 2 9 %         ) }
8 3 0 %
8 3 1 %         { / *   MESSAGES   * / }
8 3 2 %         < div   className = " flex - 1   overflow - y - auto   px - 4   py - 4   md : px - 6   md
8 3 3 %           { state . messages . length   ===   0   && (
8 3 4 %             < div   className = " text - center   text - slate - 4 0 0   mt - 1 2   max - w -
border-slate-800/80 shadow-2xl">
8 3 5 %                 < div   className = " text - 5 xl   mb - 4   animate - bounce " > 0 = Ÿi < / div
8 3 6 %                 < h 3   className = " text - lg   font - bold   text - white   mb - 1 " > Olá
8 3 7 %                 < p   className = " text - xs   text - slate - 3 0 0   leading - relaxed
8 3 8 %                 0 motor de inteligência com <strong>Especialistas (
está ativado. O sistema analisa intenções, extrai entidades, convoca especialistas (Roblox, Lua, Matemática) e verifica frases
incompletas antes de te responder.
8 3 9 %                 < / p >
8 4 0 %                 < div   className = " bg - slate - 9 5 0   p - 4   rounded - 2 xl   border
8 4 1 %                 < p   className = " text - xs   font - semibold   text - indigo - 4 0 0
os Especialistas:</p>
8 4 2 %                 < p   className = " text - xs   text - slate - 3 0 0   hover : text - whi
rounded - lg   border   border - slate - 8 0 0   transition "   onClick = { ( )   =>   setInput ( '
Automático)</p>
8 4 3 %                 < p   className = " text - xs   text - slate - 3 0 0   hover : text - whi
rounded - lg   border   border - slate - 8 0 0   transition "   onClick = { ( )   =>   setInput ( '
(Especialista Lua)</p>
8 4 4 %                 < p   className = " text - xs   text - slate - 3 0 0   hover : text - whi
rounded - lg   border   border - slate - 8 0 0   transition "   onClick = { ( )   =>   setInput ( '
erro attempt to index nil" (Especialista Roblox)</p>
8 4 5 %                 < p   className = " text - xs   text - slate - 3 0 0   hover : text - whi
rounded - lg   border   border - slate - 8 0 0   transition "   onClick = { ( )   =>   setInput ( '
um triângulo" (Especialista Matemática)</p>
8 4 6 %                 < p   className = " text - xs   text - amber - 3 0 0   hover : text - whi
rounded - lg   border   border - slate - 8 0 0   transition "   onClick = { ( )   =>   setInput ( '
8 4 7 %                 < / div >
8 4 8 %             < / div >
8 4 9 %         ) }
8 5 0 %
8 5 1 %         { / *   ETAPA 11 - DETECTOR DE ALUCINAÇÃO WARN   * / }
8 5 2 %         { state . hallucinationWarning && (
8 5 3 %           < div   className = " bg - amber - 9 5 0 / 8 0   border   border - amber - 5 0 0
shadow-lg animate-fade-in flex items-start gap-3">
8 5 4 %             < span   className = " text - 2 xl " > & p < / span >
8 5 5 %             < div >
8 5 6 %               < strong   className = " font - bold   text - white   block " > ETAP
```

```

857 %             <p className="leading-relaxed">{state.hallucination
858 %             </div>
859 %         </div>
860 %     ) }
861 %
862 %     { /* ETAPA 19 - AUTOAVALIAÇÃO E FRASE COMPLETA BOX */ }
863 %     { state.selfEvalResult && (
864 %         <div className="bg-slate-900/90 border border-slate-700
865 %             <div className="text-indigo-400 font-bold uppercase t
flex justify-between">
866 %             <span>ETAPA 19 - AUTOAVALIAÇÃO DO AGENTE B</span>
867 %             <span className="text-emerald-400">Estágio 9: Frase
868 %             </div>
869 %             <div className="grid grid-cols-3 gap-2 text-center pt
870 %                 <div className={ `p-1.5 rounded ${state.selfEvalResu
border-emerald-800/60' : 'bg-rose-950/60 text-rose-300'}`}>
871 %                 Ø=Üi Faz Sentido? <b>{state.selfEvalResult.fazSenti
872 %                 </div>
873 %                 <div className={ `p-1.5 rounded ${state.selfEvalResu
border-emerald-800/60' : 'bg-rose-950/60 text-rose-300'}`}>
874 %                 'p Frase Completa? <b>{state.selfEvalResult.estab
875 %                 </div>
876 %                 <div className={ `p-1.5 rounded ${state.selfEvalResu
border border-emerald-800/60' : 'bg-rose-950/60 text-rose-300'}`}>
877 %                 Ø<ß Respondeu Correto? <b>{state.selfEvalResult.re
878 %                 </div>
879 %             </div>
880 %         </div>
881 %     ) }
882 %
883 %     { /* PAINEL STATUS PHI-3 MINI */ }
884 %     { state.phi3Status.status !== 'idle' && state.phi3Status.s
885 %         <div className={ `mx-1 p-3 rounded-xl border text-xs fon
886 %             state.phi3Status.status === 'error' ? 'bg-rose-950/50
887 %             state.phi3Status.status === 'generating' ? 'bg-violet
888 %             'bg-violet-950/40 border-violet-700/50 text-violet-30
889 %         } `}>
890 %             <div className="flex items-center gap-2">
891 %                 <span className="text-base">{state.phi3Status.statu
892 %                 <span className="font-bold">{state.phi3Status.progr
893 %             </div>
894 %             {state.phi3Status.progress > 0 && state.phi3Status.pr
895 %                 <div className="w-full bg-slate-800 rounded-full h-
896 %                     <div
897 %                         className="bg-gradient-to-r from-violet-500 to-
898 %                         style={ { width: `${state.phi3Status.progress}%`
899 %                     />
900 %                 </div>
901 %             ) }
902 %         </div>
903 %     ) }
904 %     { state.phi3Status.status === 'ready' && (
905 %         <div className="mx-1 p-2.5 rounded-xl border border-vio
flex items-center gap-2">
906 %             <span>Ø=Ý, </span>
907 %             <span>{state.phi3Status.progressLabel}</span>
908 %             <span className="ml-auto text-violet-500 text-[10px]"
909 %         </div>
910 %     ) }
911 %
912 %     { state.messages.map( (msg, i) => {
913 %         const feedbackKey = getFeedbackKey(msg, i);
914 %         return (
915 %             <MessageView
916 %                 key={ i }
917 %                 message={ msg }

```



```

918 %         feedbackInfo={state.feedbackStatus[feedbackKey]}
919 %         correctedByPhi3={state.phi3CorrectedMessages[feedbackKey]}
920 %         phi3Loading={phi3LoadingKey === feedbackKey}
921 %         onLike={() => handleFeedback('like', msg, i)}
922 %         onDislike={() => handleFeedback('dislike', msg, i)}
923 %         onSuperLike={() => handleFeedback('superlike', msg, i)}
924 %         onPhi3Correct={() => handlePhi3Correct(msg, i)}
925 %     />
926 %     );
927 % })}
928 %
929 %     {state.isGenerating && (
930 %         <div className="flex justify-start">
931 %             <div className="max-w-[90%] md:max-w-[75%] w-full">
932 %                 {state.cotLiveStatus && (
933 %                     <div className="mb-2.5 px-3 py-2 bg-indigo-950/70
font-mono flex items-center gap-2 shadow-sm animate-pulse">
934 %                         <span className="w-2 h-2 rounded-full bg-emerald-500"></span>
935 %                         <span>{state.cotLiveStatus}</span>
936 %                     </div>
937 %                 )}
938 %             <div className="bg-slate-800 border border-slate-700" style={
939 %                 {state.lastClassification && (
940 %                     <div className="mb-2">
941 %                         <CategoryBadge
942 %                             category={state.lastClassification.category}
943 %                             intention={state.lastClassification.intention}
944 %                             expertName={state.lastClassification.expertName}
945 %                             confidence={state.lastClassification.confidence}
946 %                             isCorrection={state.lastClassification.isCorrection}
947 %                         />
948 %                     </div>
949 %                 )}
950 %             <p className="text-sm whitespace-pre-wrap leading-relaxed">
951 %                 {state.liveTokens.map((t, i) => {
952 %                     const isPunct = /^[.,?!:;]$/ .test(t.token);
953 %                     const prev = i > 0 ? state.liveTokens[i-1].token : '';
954 %                     const space = i > 0 && !isPunct && t.token !== '\n' ? ' ' : (space + t.token);
955 %                     return (t.token === '\n' ? '\n' : space + t.token);
956 %                 }).join('')}
957 %                 <span className="inline-block w-2 h-4 bg-indigo-950"></span>
958 %             </p>
959 %             {state.agentBStatus && (
960 %                 <div className="mt-3 pt-2 border-t border-slate-700
gap-1.5 animate-pulse">
961 %                     <span>{state.agentBStatus}</span>
962 %                 </div>
963 %             )}
964 %         </div>
965 %         {state.showDebug && state.liveTokens.length > 0 && (
966 %             <TokenStream tokens={state.liveTokens} compact />
967 %         )}
968 %     </div>
969 % </div>
970 % })}
971 %     <div ref={chatEndRef} />
972 % </div>
973 %
974 % { /* INPUT */ }
975 % <div className="p-3 md:p-4 bg-slate-900 border-t border-slate-800">
976 %     <div className="flex gap-2">
977 %         <input
978 %             type="text"
979 %             value={input}
980 %             onChange={e => setInput(e.target.value)}
981 %             onKeyDown={e => e.key === 'Enter' && handleSend()}

```

```

982 %             placeholder="Pergunte algo ou cole seu código..."
983 %             disabled={state.isGenerating}
984 %             className="flex-1 bg-slate-950 border border-slate-700
focus:border-indigo-500 disabled:opacity-50 min-w-0 text-white shadow-inner placeholder:text-slate-500"
985 %             />
986 %         <button
987 %             onClick={handleSend}
988 %             disabled={!input.trim() || state.isGenerating}
989 %             className="bg-indigo-600 hover:bg-indigo-500 disabled:
rounded-xl font-semibold transition-colors shrink-0 flex items-center gap-1.5 shadow-md shadow-indigo-600/20"
990 %             >
991 %                 <span>Enviar</span>
992 %                 <svg width="16" height="16" viewBox="0 0 24 24" fill="
993 %                     <line x1="22" y1="2" x2="11" y2="13" />
994 %                     <polygon points="22 2 15 22 11 13 2 9 22 2" />
995 %                 </svg>
996 %             </button>
997 %         </div>
998 %     </div>
999 % </div>
1000 % );
1001 % };
1002 %
1003 % // =====
1004 % // COMPONENTE: Sidebar (Conhecimento + Memória + Painel Dev com C
1005 % // =====
1006 %
1007 % const Sidebar: React.FC = () => {
1008 %     const { state, setState, orchestrator } = useApp();
1009 %     const [tab, setTab] = useState<'knowledge' | 'dev' | 'memory'>('
1010 %     const [newTitle, setNewTitle] = useState('');
1011 %     const [newContent, setNewContent] = useState('');
1012 %     const [newCategory, setNewCategory] = useState<Category>('CONVE
1013 %     const [knowledge, setKnowledge] = useState<any[]>([]);
1014 %     const [userFacts, setUserFacts] = useState<any[]>([]);
1015 %
1016 %     // Analítica do Painel Dev (Fila de Aprendizado, Ranking e Corr
1017 %     const [devAnalytics, setDevAnalytics] = useState<{
1018 %         totalApproved: number;
1019 %         totalNeutral: number;
1020 %         totalRejected: number;
1021 %         approvedList: LearningQueueItem[];
1022 %         neutralList: LearningQueueItem[];
1023 %         rejectedList: LearningQueueItem[];
1024 %         userRanking: User[];
1025 %         humanCorrectionsList: HumanCorrection[];
1026 %     }>({ totalApproved: 0, totalNeutral: 0, totalRejected: 0, appro
[], humanCorrectionsList: [] });
1027 %
1028 %     // Estado temporário para o Corretor Humano
1029 %     const [correctingItemId, setCorrectingItemId] = useState<number>
1030 %     const [customCorrectedText, setCustomCorrectedText] = useState('
1031 %
1032 %     useEffect(() => {
1033 %         refresh();
1034 %     }, [tab, state.currentUser]);
1035 %
1036 %     const refresh = async () => {
1037 %         if (tab === 'knowledge') {
1038 %             const docs = await db.knowledge.toArray();
1039 %             setKnowledge(docs);
1040 %         }
1041 %         if (tab === 'dev') {
1042 %             const analytics = await orchestrator.feedbackEngine.getDeve
1043 %             setDevAnalytics(analytics);
1044 %         }

```

```

1045 %         if (tab === 'memory' && state.currentUser) {
1046 %             const facts = await db.facts.where('userId').equals(state.c
1047 %                 setUserFacts(facts);
1048 %         }
1049 %     };
1050 %
1051 %     const handleAddDoc = async () => {
1052 %         if (!newTitle.trim() || !newContent.trim()) return;
1053 %         const validation = orchestrator.validator.validate(newContent
1054 %         if (!validation.valid) {
1055 %             alert(`Texto rejeitado: ${validation.reason}`);
1056 %             return;
1057 %         }
1058 %         await db.knowledge.add({
1059 %             title: newTitle,
1060 %             content: newContent,
1061 %             category: newCategory,
1062 %             createdAt: new Date(),
1063 %         });
1064 %         orchestrator.model.train(newContent, newCategory);
1065 %         await orchestrator.persistenceGuard.createMirrorBackup();
1066 %
1067 %         const stats = orchestrator.model.getStats();
1068 %         setState(s => ({
1069 %             ...s,
1070 %             modelStats: { vocab: stats.vocabSize, words: stats.totalWor
1071 %         }));
1072 %         setNewTitle('');
1073 %         setNewContent('');
1074 %         refresh();
1075 %     };
1076 %
1077 %     const handleSwitchUser = async (user: User) => {
1078 %         localStorage.setItem('codeai.activeUserId', user.uuid);
1079 %         const msgs = await db.messages.where('userId').equals(user.uu
1080 %         auditLog('SWITCH_USER', { userId: user.uuid, userName: user.n
1081 %         setState(s => ({
1082 %             ...s,
1083 %             currentUser: user,
1084 %             messages: msgs,
1085 %             showSidebar: false,
1086 %             lastClassification: null,
1087 %             feedbackStatus: {},
1088 %             lastStopReason: null,
1089 %             agentBStatus: null,
1090 %             cotLiveStatus: null,
1091 %             hallucinationWarning: null,
1092 %             selfEvalResult: null,
1093 %         }));
1094 %     };
1095 %
1096 %     const handleLogout = () => {
1097 %         localStorage.removeItem('codeai.activeUserId');
1098 %         auditLog('LOGOUT', { userId: state.currentUser?.uuid });
1099 %         setState(s => ({ ...s, currentUser: null, messages: [], showS
agentBStatus: null, cotLiveStatus: null, hallucinationWarning: null, selfEvalResult: null, devPanelUnlocked: false }));
1100 %     };
1101 %
1102 %     const handleDeleteDoc = async (id: number) => {
1103 %         await db.knowledge.delete(id);
1104 %         await orchestrator.persistenceGuard.createMirrorBackup();
1105 %         refresh();
1106 %     };
1107 %
1108 %     // Trava de Segurança para o Painel Dev
1109 %     const unlockDevPanel = () => {

```

```

1110 %         const pwd = prompt('Digite a senha de acesso administrativo a
1111 %         if (pwd === 'pão') {
1112 %             setState(s => ({ ...s, devPanelUnlocked: true }));
1113 %             refresh();
1114 %         } else {
1115 %             alert('Senha Incorreta!');
1116 %         }
1117 %     };
1118 %
1119 %     // Salva correção manual de desenvolvedor
1120 %     const submitHumanCorrection = async (item: LearningQueueItem) => {
1121 %         if (!customCorrectedText.trim() || !item.id) return;
1122 %         await orchestrator.feedbackEngine.saveHumanCorrection(
1123 %             item.id,
1124 %             item.question,
1125 %             item.answer,
1126 %             customCorrectedText.trim(),
1127 %             item.category as Category
1128 %         );
1129 %         alert('Correção Humana salva com sucesso! Treinamento inten
1130 %         setCorrectingItemId(null);
1131 %         setCustomCorrectedText('');
1132 %         refresh();
1133 %     };
1134 %
1135 %     return (
1136 %         <div className={`
1137 %             ${state.showSidebar ? 'fixed inset-0 z-50' : 'hidden'}
1138 %             md:relative md:flex md:flex-col md:w-80 md:z-auto md:bg-sla
1139 %             bg-black/60 md:bg-transparent
1140 %         `} onClick={(e) => {
1141 %             if (e.target === e.currentTarget) setState(s => ({ ...s, sh
1142 %         }} >
1143 %             <div className="bg-slate-900 border-l border-slate-800 h-fu
1144 %                 { /* Tabs */ }
1145 %                 <div className="flex border-b border-slate-800 bg-slate-9
1146 %                     <button
1147 %                         onClick={() => setTab('knowledge')}
1148 %                         className={`flex-1 px-1 py-3 text-xs font-semibold tr
1149 %                         >
1150 %                         Ø=ÜÜ Base
1151 %                     </button>
1152 %                     <button
1153 %                         onClick={() => setTab('memory')}
1154 %                         className={`flex-1 px-1 py-3 text-xs font-semibold tr
1155 %                         >
1156 %                         Ø>Ýà Memória
1157 %                     </button>
1158 %                     <button
1159 %                         onClick={() => setTab('dev')}
1160 %                         className={`flex-1 px-1 py-3 text-xs font-semibold tr
1161 %                         >
1162 %                         Ø=ÞàÞ Painel Dev
1163 %                     </button>
1164 %                 </div>
1165 %
1166 %                 { /* KNOWLEDGE TAB */ }
1167 %                 { tab === 'knowledge' && (
1168 %                     <div className="flex-1 flex flex-col overflow-hidden">
1169 %                         <div className="flex-1 overflow-y-auto p-4 space-y-3"
1170 %                             <p className="text-xs text-slate-400 leading-relaxe
1171 %                                 Alimento o modelo com textos ou documentação brut
1172 %                             </p>

```

```
1173 % { knowledge.length === 0 && (
1174 %   <p className="text-xs text-slate-600 text-center
1175 % ) }
1176 % { knowledge.map((doc, i) => (
1177 %   <div key={i} className="bg-slate-800/80 p-3 rounded
transition">
1178 %     <div className="flex items-start justify-between
1179 %       <div className="min-w-0 flex-1">
1180 %         <div className="mb-1.5">
1181 %           <CategoryBadge category={doc.category as
1182 %         </div>
1183 %         <p className="text-xs font-bold text-white
1184 %         <p className="text-[11px] text-slate-300 mt
1185 %       </div>
1186 %       <button
1187 %         onClick={() => handleDeleteDoc(doc.id)}
1188 %         className="text-rose-400 hover:text-rose-300
rounded-lg transition"
1189 %           title="Excluir documento"
1190 %         >
1191 %           '
1192 %         </button>
1193 %       </div>
1194 %     </div>
1195 %   ) ) }
1196 % </div>
1197 % <div className="p-4 border-t border-slate-800 bg-slate-800/80
1198 %   <input
1199 %     type="text" placeholder="Título do Conhecimento"
1200 %     value={newTitle} onChange={e => setNewTitle(e.target.value)}
1201 %     className="w-full bg-slate-900 border border-slate-800 focus:border-indigo-500 outline-none"
1202 %   />
1203 %   <select
1204 %     value={newCategory}
1205 %     onChange={e => setNewCategory(e.target.value as Category)}
1206 %     className="w-full bg-slate-900 border border-slate-800 focus:border-indigo-500 outline-none"
1207 %   >
1208 %     <option value="CONVERSA">CONVERSA</option>
1209 %     <option value="PROGRAMAÇÃO">PROGRAMAÇÃO</option>
1210 %     <option value="ROBLOX">ROBLOX</option>
1211 %     <option value="LUAU">LUAU</option>
1212 %     <option value="JAVASCRIPT">JAVASCRIPT</option>
1213 %     <option value="MATEMÁTICA">MATEMÁTICA</option>
1214 %     <option value="PORTUGUÊS">PORTUGUÊS</option>
1215 %     <option value="ERROS COMUNS">ERROS COMUNS</option>
1216 %     <option value="DOCUMENTAÇÃO">DOCUMENTAÇÃO</option>
1217 %   </select>
1218 %   <textarea
1219 %     placeholder="Cole texto bruto (parágrafos naturais)"
1220 %     rows={3}
1221 %     value={newContent} onChange={e => setNewContent(e.target.value)}
1222 %     className="w-full bg-slate-900 border border-slate-800 focus:border-indigo-500 outline-none resize-none"
1223 %   />
1224 %   <button
1225 %     onClick={handleAddDoc}
1226 %     disabled={!newTitle.trim() || !newContent.trim()}
1227 %     className="w-full bg-emerald-600 hover:bg-emerald-700 rounded-lg py-2.5 text-xs font-bold shadow-md shadow-emerald-600/20 transition-all"
1228 %   >
1229 %     '• Treinar Modelo
1230 %   </button>
1231 % </div>
1232 % </div>
```

```
1 2 3 3 %           ) }
1 2 3 4 %
1 2 3 5 %           { /* MEMORY TAB */ }
1 2 3 6 %           { tab === 'memory' && (
1 2 3 7 %               <div className="flex-1 overflow-y-auto p-4 space-y-3">
1 2 3 8 %                   <p className="text-xs text-slate-300 leading-relaxed
1 2 3 9 %                       Memória individual isolada de <b className="text-in
1 2 4 0 %                   </p>
1 2 4 1 %                   <div className="bg-slate-800/80 p-3 rounded-xl border
1 2 4 2 %                       <p className="text-[10px] text-slate-400 font-semib
1 2 4 3 %                       <p className="text-xs font-mono text-emerald-400 br
1 2 4 4 %                   </div>
1 2 4 5 %                   <div className="bg-slate-800/80 p-3 rounded-xl border
1 2 4 6 %                       <p className="text-[10px] text-slate-400 font-semib
1 2 4 7 %                       <p className="text-xs text-white font-bold mt-0.5">
1 2 4 8 %                   </div>
1 2 4 9 %                   <p className="text-xs font-bold text-slate-300 mt-4 m
1 2 5 0 %                       <span>0=0i Fatos & Interesses Biases:</span>
1 2 5 1 %                   </p>
1 2 5 2 %               {userFacts.length === 0 ? (
1 2 5 3 %                   <p className="text-xs text-slate-500 bg-slate-950 p
leading-relaxed">
1 2 5 4 %                       Nenhum fato extraído ainda. Diga algo como <strong
<strong className="text-slate-300">"eu gosto de..."</strong> no chat.
1 2 5 5 %                   </p>
1 2 5 6 %               ) : (
1 2 5 7 %                   userFacts.map((f, i) => (
1 2 5 8 %                       <div key={i} className="bg-slate-800/90 p-3 round
1 2 5 9 %                           <p className="text-[10px] text-amber-400 font-b
1 2 6 0 %                           <p className="text-xs text-white font-medium mt
1 2 6 1 %                       </div>
1 2 6 2 %                   ))
1 2 6 3 %               ) }
1 2 6 4 %           </div>
1 2 6 5 %       ) }
1 2 6 6 %
1 2 6 7 %           { /* DEV TAB / PAINEL DE DESENVOLVEDOR AVANÇADO (FILAS, RAN
1 2 6 8 %           { tab === 'dev' && (
1 2 6 9 %               <div className="flex-1 overflow-y-auto p-4 space-y-5 bg
1 2 7 0 %                   {!state.devPanelUnlocked ? (
1 2 7 1 %                       <div className="text-center py-12 space-y-4">
1 2 7 2 %                           <div className="text-4xl">0=Ý </div>
1 2 7 3 %                           <h3 className="text-sm font-bold text-white">Pain
1 2 7 4 %                           <p className="text-xs text-slate-400 leading-rela
1 2 7 5 %                               Acesso restrito a desenvolvedores para gerencia
1 2 7 6 %                           </p>
1 2 7 7 %                           <button
1 2 7 8 %                               onClick={unlockDevPanel}
1 2 7 9 %                               className="bg-indigo-600 hover:bg-indigo-500 te
shadow-indigo-600/20 transition"
1 2 8 0 %                               >
1 2 8 1 %                                   0=Ý Desbloquear Painel
1 2 8 2 %                               </button>
1 2 8 3 %                           </div>
1 2 8 4 %                       ) : (
1 2 8 5 %                           <>
1 2 8 6 %                               <SourcePdfExporter />
1 2 8 7 %
1 2 8 8 %                               { /* 0<BÆ Ranking Global de Usuários por Likes */ }
1 2 8 9 %                               <div className="space-y-2.5">
1 2 9 0 %                                   <p className="text-xs text-amber-400 font-bold
1 2 9 1 %                                       <span>0<BÆ Ranking Global de Usuários (Likes)</
1 2 9 2 %                                   </p>
1 2 9 3 %                                   <div className="space-y-1.5 max-h-36 overflow-y
1 2 9 4 %                                       {devAnalytics.userRanking.map((u, i) => (
1 2 9 5 %                                           <button
```

```
1 2 9 6 %         key = { i }
1 2 9 7 %         onClick = { ( ) => handleSwitchUser ( u ) }
1 2 9 8 %         className = { `w-full text-left p-2.5 rounded
1 2 9 9 %             u.uuid === state.currentUser?.uuid
1 3 0 0 %             ? 'bg-indigo-600/30 border-indigo-500
1 3 0 1 %             : 'bg-slate-800/80 border-slate-700/80
1 3 0 2 %         } ` }
1 3 0 3 %         >
1 3 0 4 %         <span className="text-xs font-bold truncate
1 3 0 5 %             { i === 0 ? '0>Y G ' : i === 1 ? '0>Y H ' : i =
1 3 0 6 %         </span>
1 3 0 7 %         <span className="text-[10px] font-mono te
py-0.5 rounded-lg truncate ml-2 font-bold">
1 3 0 8 %             Likes: { u.likesCount || 0 }
1 3 0 9 %         </span>
1 3 1 0 %         </button>
1 3 1 1 %     ) ) }
1 3 1 2 % </div>
1 3 1 3 % <button
1 3 1 4 %     onClick = { handleLogout }
1 3 1 5 %     className = "w-full bg-slate-800 hover:bg-indig
text-xs font-semibold border border-slate-700 shadow-sm transition"
1 3 1 6 %     >
1 3 1 7 %         '• Criar Novo Usuário (Novo ID)
1 3 1 8 %     </button>
1 3 1 9 % </div>
1 3 2 0 %
1 3 2 1 % <div className="border-t border-slate-800 pt-4 sp
1 3 2 2 %     <div className="flex items-center justify-betwe
1 3 2 3 %         <p className="text-xs text-indigo-400 font-bo
1 3 2 4 %             <span>&#p Nuvem & Fila de Aprovação</span>
1 3 2 5 %         </p>
1 3 2 6 %         <button
1 3 2 7 %             onClick = { async ( ) => {
1 3 2 8 %                 await orchestrator.feedbackEngine.syncClo
1 3 2 9 %                 const stats = orchestrator.model.getStats
1 3 3 0 %                 setState ( s => ( {
1 3 3 1 %                     ...s,
1 3 3 2 %                     modelStats: { vocab: stats.vocabSize, w
1 3 3 3 %                 } ) ) ;
1 3 3 4 %                 alert ( 'Sincronização da Fila de Aprendiz
1 3 3 5 %             } ) }
1 3 3 6 %             className = "bg-indigo-600 hover:bg-indigo-50
shadow-md transition active:scale-95"
1 3 3 7 %             title = "Recalcular e aplicar todos os Approv
1 3 3 8 %         >
1 3 3 9 %             0>Y Sync Fila
1 3 4 0 %         </button>
1 3 4 1 %     </div>
1 3 4 2 %
1 3 4 3 %     { /* Estatísticas de Feedback */ }
1 3 4 4 %     <div className="grid grid-cols-3 gap-2 text-cen
1 3 4 5 %         <div className="space-y-0.5">
1 3 4 6 %             <div className="text-xs font-mono font-bold
1 3 4 7 %                 <div className="text-[10px] text-slate-400
1 3 4 8 %             </div>
1 3 4 9 %             <div className="space-y-0.5 border-x border-s
1 3 5 0 %                 <div className="text-xs font-mono font-bold
1 3 5 1 %                     <div className="text-[10px] text-slate-400
1 3 5 2 %                 </div>
1 3 5 3 %             <div className="space-y-0.5">
1 3 5 4 %                 <div className="text-xs font-mono font-bold
1 3 5 5 %                     <div className="text-[10px] text-slate-400
1 3 5 6 %                 </div>
1 3 5 7 %             </div>
1 3 5 8 %         </div>
```

```
1 3 5 9 % { / * Corretor Humano - Lista de Aprovadas */ }
1 3 6 0 % <div className="space-y-2">
1 3 6 1 % <p className="text-xs text-emerald-400 font-b
1 3 6 2 % <span>0=0M Conhecimento Global (Score 10+) &
1 3 6 3 % </p>
1 3 6 4 % <div className="space-y-2 max-h-48 overflow-y
1 3 6 5 % {devAnalytics.approvedList.length === 0 &&
p-3 rounded-xl border border-slate-800 text-center">Sem respostas approved no banco.</p>
1 3 6 6 % {devAnalytics.approvedList.map((app, idx) =
1 3 6 7 % const scoreData = orchestrator.feedbackEn
1 3 6 8 % const totalScoreVal = app.likes + (app.su
1 3 6 9 % return (
1 3 7 0 % <div key={idx} className={ `bg-slate-950
${app.superLike ? 'border-amber-500/40 bg-amber-950/10' : 'border-slate-800'}`}>
1 3 7 1 % <div className="text-[10px] text-slate
border-slate-800/80 pb-1.5">
1 3 7 2 % <span className="font-bold text-slate
text-indigo-400">({app.userId.split('-')[0]})</span></span>
1 3 7 3 % <span className={ `font-bold px-2 py
'bg-amber-500/20 text-amber-300 border border-amber-500/40' : 'bg-emerald-500/20 text-emerald-300'}`}>
1 3 7 4 % {app.superLike || totalScoreVal >
1 3 7 5 % </span>
1 3 7 6 % </div>
1 3 7 7 % <div className="grid grid-cols-2 gap-
border-slate-800/60 font-mono">
1 3 7 8 % <div>0<ß+p Cat: <span className="text
1 3 7 9 % <div>0<ß Intenção: <span className="
1 3 8 0 % <div>&i Score: <span className="text
1 3 8 1 % <div>0<ß Pontos: <span className="te
1 3 8 2 % </div>
1 3 8 3 % <div className="text-slate-300 font-s
className="text-indigo-400">P:</strong> {app.question}</div>
1 3 8 4 % <div className="text-slate-200 font-s
1 3 8 5 % <strong className="text-indigo-400"
1 3 8 6 % {app.correctedAnswer && <span classs
Corrigido Humanamente pelo Dev</span>}
1 3 8 7 % </div>
1 3 8 8 % </div>
1 3 8 9 % );
1 3 9 0 % } ) }
1 3 9 1 % </div>
1 3 9 2 % </div>
1 3 9 3 %
1 3 9 4 % { / * Neutral / Corretor Humano */ }
1 3 9 5 % <div className="space-y-2">
1 3 9 6 % <p className="text-xs text-amber-400 font-bol
1 3 9 7 % <span>#6 Neutral (Fila de Aprovação Abaixo d
1 3 9 8 % </p>
1 3 9 9 % <div className="space-y-2 max-h-56 overflow-y
1 4 0 0 % {devAnalytics.neutralList.length === 0 && <
rounded-xl border border-slate-800 text-center">Sem respostas neutral no banco.</p>
1 4 0 1 % {devAnalytics.neutralList.map((neu, idx) =>
1 4 0 2 % const scoreData = orchestrator.feedbackEn
1 4 0 3 % const totalScoreVal = neu.likes + (neu.su
1 4 0 4 % return (
1 4 0 5 % <div key={idx} className="bg-slate-950
shadow-sm">
1 4 0 6 % <div className="text-[10px] text-slate
border-slate-800/80 pb-1.5">
1 4 0 7 % <span className="font-bold text-slate
text-indigo-400">({neu.userId.split('-')[0]})</span></span>
1 4 0 8 % <span className="bg-amber-500/20 te
border-amber-500/30">#6 NEUTRAL ({totalScoreVal}/10 pts)</span>
1 4 0 9 % </div>
1 4 1 0 % <div className="grid grid-cols-2 gap-
border-slate-800/60 font-mono">
```



```

1 4 1 1 %
1 4 1 2 %
1 4 1 3 %
1 4 1 4 %
1 4 1 5 %
1 4 1 6 %
className="text-indigo-400">P:</strong> {neu.question}</div>
1 4 1 7 %
className="text-indigo-400">R:</strong> {neu.answer}</div>
1 4 1 8 %
1 4 1 9 %
1 4 2 0 %
1 4 2 1 %
1 4 2 2 %
Resposta Corrigida Humana:</p>
1 4 2 3 %
1 4 2 4 %
1 4 2 5 %
1 4 2 6 %
1 4 2 7 %
1 4 2 8 %
focus:border-indigo-500 outline-none resize-none"
1 4 2 9 %
1 4 3 0 %
1 4 3 1 %
1 4 3 2 %
1 4 3 3 %
py-1.5 rounded-lg shadow transition"
1 4 3 4 %
1 4 3 5 %
1 4 3 6 %
1 4 3 7 %
1 4 3 8 %
1 4 3 9 %
py-1.5 rounded-lg transition"
1 4 4 0 %
1 4 4 1 %
1 4 4 2 %
1 4 4 3 %
1 4 4 4 %
1 4 4 5 %
1 4 4 6 %
1 4 4 7 %
1 4 4 8 %
1 4 4 9 %
font-medium text-[10px] px-2.5 py-1 rounded-lg transition flex items-center gap-1"
1 4 5 0 %
1 4 5 1 %
1 4 5 2 %
1 4 5 3 %
1 4 5 4 %
1 4 5 5 %
1 4 5 6 %
1 4 5 7 %
1 4 5 8 %
1 4 5 9 %
1 4 6 0 %
1 4 6 1 %
1 4 6 2 %
1 4 6 3 %
1 4 6 4 %
1 4 6 5 %
1 4 6 6 %
1 4 6 7 %
p-3 rounded-xl border border-slate-800 text-center">Sem respostas rejected no banco.</p>
1 4 6 8 %

```

```

<div><div><div><div></div><div className="text-slate-300 font-s
<div className="text-slate-200 font-s
{ /* Formulário de Correção Humana */ }
{ correctingItemId === neu.id ? (
  <div className="space-y-2 pt-2 border
    <p className="text-[10px] text-em
  <textarea
    value={customCorrectedText}
    onChange={e => setCustomCorrectedText(e)}
    placeholder="Escreva a resposta corrigida"
    rows={3}
    className="w-full bg-slate-900
  />
<div className="flex gap-2">
  <button
    onClick={() => submitHumanCorrection()}
    className="flex-1 bg-emerald-500 text-white
  >
    Salvar Correção (Peso 100x)
  </button>
  <button
    onClick={() => { setCorrectingItemId(neu.id) }}
    className="bg-slate-700 hover:bg-slate-600
  >
    Cancelar
  </button>
</div>
</div>
) : (
  <div className="pt-2 border-t border-slate-800
    <button
      onClick={() => { setCorrectingItemId(neu.id) }}
      className="bg-indigo-950/60 hover:bg-indigo-800/60
    >
      Corrigir Manualmente </button>
  </div>
) }
</div>
);
} ) }
</div>
</div>
{ /* Rejeitadas */ }
<div className="space-y-2">
  <p className="text-xs text-rose-400 font-bold">
    <span>ÚN Rejected ({devAnalytics.rejectedList.length})
  </p>
  <div className="space-y-2 max-h-48 overflow-y-auto">
    {devAnalytics.rejectedList.length === 0 &&
    devAnalytics.rejectedList.map((rej, idx) =>

```

```
1 4 6 9 %                                const scoredData = orchestrator.feedbackEn
1 4 7 0 %                                return (
1 4 7 1 %                                <div key={idx} className="bg-slate-950
shadow-sm">
1 4 7 2 %                                <div className="text-[10px] text-slate
border-slate-800/80 pb-1.5">
1 4 7 3 %                                <span className="font-bold text-slate
text-indigo-400">({rej.userId.split('-')[0]})</span></span>
1 4 7 4 %                                <span className="bg-rose-500/20 text
border-rose-500/30">0=UN REJECTED ( Penaliza ) </span>
1 4 7 5 %                                </div>
1 4 7 6 %                                <div className="grid grid-cols-2 gap-
border-slate-800/60 font-mono">
1 4 7 7 %                                <div>0<ß÷p Cat: <span className="text
1 4 7 8 %                                <div>0<ß¬ Intenção: <span className="
1 4 7 9 %                                <div>&i Score: <span className="text
1 4 8 0 %                                </div>
1 4 8 1 %                                <div className="text-slate-300 font-s
className="text-indigo-400">P:</strong> {rej.question}</div>
1 4 8 2 %                                <div className="text-slate-200 font-s
className="text-indigo-400">R:</strong> {rej.answer}</div>
1 4 8 3 %                                <div className="text-[10px] text-slate
1 4 8 4 %                                <span className="text-indigo-400/80
1 4 8 5 %                                </div>
1 4 8 6 %                                </div>
1 4 8 7 %                                );
1 4 8 8 %                                } ) }
1 4 8 9 %                                </div>
1 4 9 0 %                                </div>
1 4 9 1 %                                </div>
1 4 9 2 %                                </>
1 4 9 3 %                                ) }
1 4 9 4 %                                </div>
1 4 9 5 %                                ) }
1 4 9 6 %                                </div>
1 4 9 7 %                                </div>
1 4 9 8 %                                );
1 4 9 9 %                                };
1 5 0 0 %
1 5 0 1 %                                // =====
1 5 0 2 %                                // LOGIN / CRIAÇÃO DE USUÁRIO
1 5 0 3 %                                // =====
1 5 0 4 %
1 5 0 5 %                                const LoginScreen: React.FC<{ onCreate: (name: string) => void; e
onCreate, existingUsers, onSelect }) => {
1 5 0 6 %                                const [name, setName] = useState('');
1 5 0 7 %                                return (
1 5 0 8 %                                <div className="h-screen bg-slate-950 flex items-center justify
1 5 0 9 %                                <div className="bg-slate-900 border border-slate-800 p-6 md
1 5 1 0 %                                <div className="text-center space-y-2">
1 5 1 1 %                                <div className="w-16 h-16 bg-indigo-600/20 border borde
justify-center text-4xl mx-auto mb-4 shadow-inner">0>Yà</div>
1 5 1 2 %                                <h1 className="text-2xl font-bold text-white tracking-w
1 5 1 3 %                                <p className="text-xs text-slate-400 leading-relaxed">
1 5 1 4 %                                Assistente com Poda de Saudações (POS Engine) e Armaz
sabedoria persistente.
1 5 1 5 %                                </p>
1 5 1 6 %                                </div>
1 5 1 7 %
1 5 1 8 %                                {existingUsers.length > 0 && (
1 5 1 9 %                                <div className="space-y-2">
1 5 2 0 %                                <p className="text-xs font-bold text-indigo-400 upper
1 5 2 1 %                                <div className="space-y-2 max-h-36 overflow-y-auto pr
1 5 2 2 %                                {existingUsers.map((u, i) => (
1 5 2 3 %                                <button
1 5 2 4 %                                key={i}
```

```
1 5 2 5 %             onClick={ () => onSelect(u) }
1 5 2 6 %             className="w-full text-left p-3 bg-slate-800/80
hover:border-slate-600 transition flex justify-between items-center group shadow-sm"
1 5 2 7 %             >
1 5 2 8 %             <span className="text-xs font-bold text-white g
1 5 2 9 %             <span className="text-[10px] font-mono text-amb
border-slate-800 font-bold">Likes: {u.likesCount || 0}</span>
1 5 3 0 %             </button>
1 5 3 1 %             ) ) }
1 5 3 2 %             </div>
1 5 3 3 %             <div className="border-t border-slate-800/80 pt-2" />
1 5 3 4 %             </div>
1 5 3 5 %         ) }
1 5 3 6 %
1 5 3 7 %         <div className="space-y-3">
1 5 3 8 %             <p className="text-xs font-bold text-indigo-400 uppercase
1 5 3 9 %             <input
1 5 4 0 %                 type="text"
1 5 4 1 %                 placeholder="Qual o seu nome?"
1 5 4 2 %                 value={name}
1 5 4 3 %                 onChange={e => setName(e.target.value)}
1 5 4 4 %                 onKeyDown={e => e.key === 'Enter' && name.trim() && o
1 5 4 5 %                 className="w-full bg-slate-950 border border-slate-70
outline-none shadow-inner placeholder:text-slate-500"
1 5 4 6 %                 />
1 5 4 7 %             <button
1 5 4 8 %                 onClick={ () => name.trim() && onCreate(name) }
1 5 4 9 %                 disabled={!name.trim()}
1 5 5 0 %                 className="w-full bg-indigo-600 hover:bg-indigo-500 d
p-3.5 font-bold shadow-lg shadow-indigo-600/20 transition-all flex items-center justify-center gap-2"
1 5 5 1 %             >
1 5 5 2 %                 <span>0<Y" Gerar ID Persistente e Iniciar</span>
1 5 5 3 %             </button>
1 5 5 4 %         </div>
1 5 5 5 %     </div>
1 5 5 6 % </div>
1 5 5 7 % );
1 5 5 8 % };
1 5 5 9 %
1 5 6 0 % // =====
1 5 6 1 % // APP RAIZ E INICIALIZADOR DEFINITIVO
1 5 6 2 % // =====
1 5 6 3 %
1 5 6 4 % export default function App() {
1 5 6 5 %     const defaultPhi3State: Phi3State = {
1 5 6 6 %         status: 'idle',
1 5 6 7 %         progress: 0,
1 5 6 8 %         progressLabel: 'Phi-3 Mini: clique em "0=Y Corrigir com Phi-3 "
1 5 6 9 %         backend: null,
1 5 7 0 %         modelId: 'onnx-community/Phi-3.5-mini-instruct-onnx-web',
1 5 7 1 %     };
1 5 7 2 %     const [state, setState] = useState<AppState>({
1 5 7 3 %         currentUser: null,
1 5 7 4 %         messages: [],
1 5 7 5 %         isGenerating: false,
1 5 7 6 %         liveTokens: [],
1 5 7 7 %         temperature: 0.7,
1 5 7 8 %         lastClassification: null,
1 5 7 9 %         showSidebar: false,
1 5 8 0 %         modelStats: { vocab: 0, words: 0, categories: [] },
1 5 8 1 %         showDebug: false,
1 5 8 2 %         feedbackStatus: {},
1 5 8 3 %         lastStopReason: null,
1 5 8 4 %         agentBStatus: null,
1 5 8 5 %         cotLiveStatus: null,
1 5 8 6 %         hallucinationWarning: null,
```

```

1587 %         selfEvalResult: null,
1588 %         devPanelUnlocked: false,
1589 %         phi3Status: defaultPhi3State,
1590 %         phi3CorrectedMessages: {}},
1591 %     });
1592 %     const [isLoading, setIsLoading] = useState(true);
1593 %     const [existingUsers, setExistingUsers] = useState<User[]>([]);
1594 %     const orchestratorRef = useRef<AIOrchestrator | null>(null);
1595 %     const memoryEngineRef = useRef<MemoryEngine>(new MemoryEngine());
1596 %     const phi3EngineRef = useRef<Phi3Engine>(new Phi3Engine((phi3St
1597 %         setState(s => ({ ...s, phi3Status: phi3State })));
1598 %     }));
1599 %
1600 %     useEffect(() => {
1601 %         const init = async () => {
1602 %             const orch = new AIOrchestrator();
1603 %
1604 %             await orch.initializeCorpora([
1605 %                 { category: 'CONVERSA', texts: [CONVERSATION_PT_CORPUS, C
1606 %                 { category: 'PROGRAMAÇÃO', texts: [PROGRAMMING_CORPUS, JA
1607 %                 { category: 'ROBLOX', texts: [ROBLOX_CORPUS] },
1608 %                 { category: 'LUAU', texts: [LUAU_CORPUS] },
1609 %                 { category: 'JAVASCRIPT', texts: [JAVASCRIPT_CORPUS] },
1610 %                 { category: 'MATEMÁTICA', texts: [MATH_CORPUS] },
1611 %                 { category: 'PORTUGUÊS', texts: [CONVERSATION_PT_CORPUS]
1612 %                 { category: 'ERROS COMUNS', texts: [ERRORS_CORPUS] },
1613 %                 { category: 'DOCUMENTAÇÃO', texts: [DOCUMENTATION_CORPUS,
1614 %                 { category: 'IA', texts: [HELP_CORPUS, DOCUMENTATION_CORP
1615 %             ]});
1616 %
1617 %             orch.registerCorpus('CONVERSA', async () => [CONVERSATION_P
1618 %             orch.registerCorpus('PROGRAMAÇÃO', async () => [PROGRAMMING
1619 %             orch.registerCorpus('ROBLOX', async () => [ROBLOX_CORPUS]);
1620 %             orch.registerCorpus('LUAU', async () => [LUAU_CORPUS]);
1621 %             orch.registerCorpus('JAVASCRIPT', async () => [JAVASCRIPT_C
1622 %             orch.registerCorpus('MATEMÁTICA', async () => [MATH_CORPUS]
1623 %             orch.registerCorpus('PORTUGUÊS', async () => [CONVERSATION_
1624 %             orch.registerCorpus('ERROS COMUNS', async () => [ERRORS_COR
1625 %             orch.registerCorpus('DOCUMENTAÇÃO', async () => [DOCUMENTAT
1626 %             orch.registerCorpus('IA', async () => [HELP_CORPUS, DOCUMEN
1627 %
1628 %             const userDocs = await db.knowledge.toArray();
1629 %             for (const doc of userDocs) {
1630 %                 const cat = (doc.category || 'CONVERSA') as Category;
1631 %                 orch.model.train(doc.content, cat);
1632 %             }
1633 %
1634 %             await orch.feedbackEngine.syncCloudWeights();
1635 %
1636 %             orchestratorRef.current = orch;
1637 %
1638 %             const users = await db.users.toArray();
1639 %             setExistingUsers(users);
1640 %
1641 %             const activeUserId = localStorage.getItem('codeai.activeUser
1642 %             let restoredUser: User | null = null;
1643 %             let restoredMessages: Message[] = [];
1644 %
1645 %             if (activeUserId) {
1646 %                 restoredUser = users.find(u => u.uuid === activeUserId) |
1647 %                 if (restoredUser) {
1648 %                     restoredMessages = await db.messages.where('userId').eq
1649 %                     auditLog('RESTORE_SESSION', { userId: restoredUser.uuid
1650 %                 }
1651 %             }
1652 %

```

```

1653 %         const stats = orch.model.getStats();
1654 %         setState(s => ({
1655 %             ...s,
1656 %             currentUser: restoredUser,
1657 %             messages: restoredMessages,
1658 %             modelStats: { vocab: stats.vocabSize, words: stats.totalW
1659 %         }));
1660 %
1661 %         setIsLoading(false);
1662 %     });
1663 %     init();
1664 % }, []));
1665 %
1666 %     const handleCreateUser = async (name: string) => {
1667 %         const trimmed = name.trim();
1668 %         const existing = existingUsers.find(u => u.name.toLowerCase()
1669 %
1670 %         if (existing) {
1671 %             localStorage.setItem('codeai.activeUserId', existing.uuid);
1672 %             const msgs = await db.messages.where('userId').equals(exist
1673 %             setState(s => ({ ...s, currentUser: existing, messages: msg
null, cotLiveStatus: null, hallucinationWarning: null, selfEvalResult: null }));
1674 %             return;
1675 %         }
1676 %
1677 %         const uuid = (crypto as any).randomUUID
1678 %             ? crypto.randomUUID()
1679 %             : 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, c
1680 %                 const r = Math.random() * 16 | 0;
1681 %                 return (c === 'x' ? r : (r & 0x3 | 0x8)).toString(16);
1682 %             });
1683 %
1684 %         const user: User = { uuid, name: trimmed, likesCount: 0, crea
1685 %         const id = await db.users.add(user);
1686 %         user.id = id;
1687 %         localStorage.setItem('codeai.activeUserId', user.uuid);
1688 %
1689 %         // Garante backup super permanente após criação
1690 %         if (orchestratorRef.current) {
1691 %             await orchestratorRef.current.persistenceGuard.createMirror
1692 %         }
1693 %
1694 %         setExistingUsers(prev => [...prev, user]);
1695 %         setState(s => ({ ...s, currentUser: user, messages: [], feedb
cotLiveStatus: null, hallucinationWarning: null, selfEvalResult: null }));
1696 %     };
1697 %
1698 %     const handleSelectUser = async (user: User) => {
1699 %         localStorage.setItem('codeai.activeUserId', user.uuid);
1700 %         const msgs = await db.messages.where('userId').equals(user.uu
1701 %         setState(s => ({ ...s, currentUser: user, messages: msgs, fee
cotLiveStatus: null, hallucinationWarning: null, selfEvalResult: null }));
1702 %     };
1703 %
1704 %     if (isLoading) {
1705 %         return (
1706 %             <div className="h-screen bg-slate-950 flex flex-col items-c
1707 %                 <div className="w-12 h-12 border-4 border-indigo-500 bord
1708 %                     <p className="text-xs font-bold text-slate-400 tracking-w
Blindada...</p>
1709 %                 </div>
1710 %             );
1711 %         }
1712 %
1713 %         if (!state.currentUser || !orchestratorRef.current) {
1714 %             return <LoginScreen onCreate={handleCreateUser} existingUsers

```

```

1 7 1 5 %      }
1 7 1 6 %
1 7 1 7 %      r e t u r n   (
1 7 1 8 %          < A p p C o n t e x t . P r o v i d e r   v a l u e = { {
1 7 1 9 %              s t a t e ,
1 7 2 0 %              s e t S t a t e ,
1 7 2 1 %              o r c h e s t r a t o r :   o r c h e s t r a t o r R e f . c u r r e n t ,
1 7 2 2 %              m e m o r y E n g i n e :   m e m o r y E n g i n e R e f . c u r r e n t ,
1 7 2 3 %              p h i 3 E n g i n e :   p h i 3 E n g i n e R e f . c u r r e n t ,
1 7 2 4 %          } } >
1 7 2 5 %          < d i v   c l a s s N a m e = " h - s c r e e n   w - s c r e e n   f l e x   b g - s l a t e - 9 5 0   o v e r f l o
1 7 2 6 %              < d i v   c l a s s N a m e = " f l e x - 1   m i n - w - 0 " >
1 7 2 7 %                  < C h a t I n t e r f a c e   / >
1 7 2 8 %              < / d i v >
1 7 2 9 %              < S i d e b a r   / >
1 7 3 0 %          < / d i v >
1 7 3 1 %          < / A p p C o n t e x t . P r o v i d e r >
1 7 3 2 %      ) ;
1 7 3 3 %  }
1 7 3 4 %

```

```
1 %   import { StrictMode } from "react";
2 %   import { createRoot } from "react-dom/client";
3 %   import "./index.css";
4 %   import App from "./App";
5 %
6 %   createRoot(document.getElementById("root")!).render(
7 %       <StrictMode>
8 %         <App />
9 %       </StrictMode>
10 %   );
11 %
```

```

1 % // =====
2 % // ANALISADOR DE CÓDIGO
3 % // Detecta linguagem e erros, gera HIPÓTESES (não respostas pront
4 % // Cada hipótese é apenas um CONTEXTO que é injetado na geração p
5 % // =====
6 %
7 % export type CodeLanguage = 'lua' | 'roblox' | 'javascript' | 'typ
8 %
9 % export interface DetectedError {
10 %     type: string;
11 %     line?: number;
12 %     excerpt: string;
13 % }
14 %
15 % export interface CodeAnalysis {
16 %     language: CodeLanguage;
17 %     hasCode: boolean;
18 %     errors: DetectedError[];
19 %     hypotheses: string[]; // Contextos a injetar na geração - NÃO s
20 %     attemptNumber: number;
21 % }
22 %
23 % // Padrões de linguagem
24 % const LANGUAGE_PATTERNS: Record<CodeLanguage, RegExp[]> = {
25 %     lua: [
26 %         /\blocal \w+\s*=/,
27 %         /\bfunction \w*\s*\(/,
28 %         /\bend\b/,
29 %         /\bthen\b/,
30 %         /\belseif\b/,
31 %         /\bpairs\s*\(/,
32 %         /\bipairs\s*\(/,
33 %         /\: \s*Connect\s*\(/,
34 %         /\bprint\s*\(/,
35 %     ],
36 %     roblox: [
37 %         /\bgame\s*[\.\:]/i,
38 %         /\bworkspace\s*[\.\:]/i,
39 %         /\bPlayers\s*[\.\:]/,
40 %         /\bWorkspace\s*[\.\:]/,
41 %         /\bReplicatedStorage\b/,
42 %         /\bServerScriptService\b/,
43 %         /\bRemoteEvent\b/,
44 %         /\bRemoteFunction\b/,
45 %         /\bHumanoid\b/,
46 %         /\:WaitForChild\s*\(/,
47 %         /\:FindFirstChild\s*\(/,
48 %         /\:GetService\s*\(/,
49 %         /\bInstance\.new\s*\(/,
50 %     ],
51 %     javascript: [
52 %         /\bconst \w+\s*=/,
53 %         /\blet \w+\s*=/,
54 %         /\bvar \w+\s*=/,
55 %         /\b=>\s*\(\(/,
56 %         /\bconsole\.log\s*\(/,
57 %         /\bdocument\./,
58 %         /\.\.then\s*\(/,
59 %         /\.\.catch\s*\(/,
60 %         /\basync function/,
61 %         /\bawait \w+/,
62 %     ],
63 %     typescript: [

```



```

64 %         /:\s*string\b/,
65 %         /:\s*number\b/,
66 %         /:\s*boolean\b/,
67 %         /\binterface \w+/,
68 %         /\btype \w+\s*=/,
69 %         /\benum \w+/,
70 %         /\bimplements /,
71 %         /<\w+(\s*,\s*\w+)*>/,    // generics
72 %     ],
73 %     python: [
74 %         /\bdef \w+\s*\(/,
75 %         /\bimport \w+/,
76 %         /\bfrom \w+ import/,
77 %         /\bprint\s*\(/,
78 %         /:\s*$/m,
79 %         /\belif\b/,
80 %         /__init__/,
81 %         /\bself\b/,
82 %         /\blambda\b/,
83 %     ],
84 %     unknown: [],
85 % };
86 %
87 % // Erros conhecidos: cada erro tem palavras-chave de DOCUMENTAÇÃO
88 % // (NÃO são respostas - são fragmentos de texto que vão ALIMENTAR
89 % const ERROR_KNOWLEDGE: Record<string, string[]> = {
90 %     'attempt to index nil': [
91 %         'O erro attempt to index nil ocorre quando você tenta acessar
variável foi inicializada antes de usar. Use if variavel then antes de acessar campos. Em Roblox use WaitForChild ou FindFirstChild
para garantir que o objeto existe.',
92 %         'Solução comum: adicionar verificação if variavel and variavel',
93 %     ],
94 %     'syntax error': [
95 %         'O erro syntax error indica que há um problema na estrutura d
Verifique se todos os blocos têm seu end correspondente em Lua. Verifique vírgulas e ponto e vírgula faltando.',
96 %         'Conte os end abertos e fechados. Verifique se esqueceu de fe
precisa de um end.',
97 %     ],
98 %     'infinite yield': [
99 %         'O erro infinite yield possible on WaitForChild ocorre quando o
correto. Verifique se o objeto está no caminho esperado. Adicione timeout: WaitForChild nome 5.',
100 %         'Confira o Explorer do Roblox. O objeto pode estar em outro l
101 %     ],
102 %     'undefined is not a function': [
103 %         'O erro undefined is not a function ocorre quando você tenta
corretamente. Verifique se o método existe no objeto. Use console.log para inspecionar o valor antes de chamar.',
104 %         'Pode ser typo no nome do método. Verifique a versão da bibli
105 %     ],
106 %     'cannot read property': [
107 %         'O erro cannot read property ocorre quando você acessa propri
interrogação ponto. Verifique se a variável foi inicializada. Use console.log para inspecionar.',
108 %         'Adicione verificação if obj and obj.prop. Use destructuring
assíncronas.',
109 %     ],
110 %     'indentation error': [
111 %         'O erro IndentationError em Python ocorre quando a indentaçã
tabs e espaços. Configure seu editor para mostrar caracteres invisíveis.',
112 %         'Todos os blocos do mesmo nível devem ter mesma indentação. A
113 %     ],
114 %     'nameerror': [
115 %         'O erro NameError em Python ocorre quando você usa uma variáv
Verifique se a variável está no escopo certo. Importe módulos antes de usar.',
116 %         'Pode ser typo no nome da variável. Pode ser problema de esco
117 %     ],
118 %     'typeerror': [

```

```

119 %      'O erro TypeError ocorre quando operação é feita com tipos in
de operar. Use instanceof para verificar tipos.',
120 %      'Pode ser que você está somando string com número. Pode ser q
o que a função retorna.',
121 %      ],
122 %      'missing end': [
123 %      'O erro missing end indica que falta a palavra end para fecha
Cada um precisa de seu próprio end. Identifique o código corretamente para visualizar.',
124 %      'Use editor com syntax highlighting. Verifique blocos aninhad
125 %      ],
126 %      'nil value': [
127 %      'O erro attempt to call a nil value ocorre quando você tenta
certo. Verifique se a função está no escopo. Verifique se foi definida antes do uso.',
128 %      'Em Roblox verifique se o serviço foi obtido com GetService.
129 %      ],
130 %  };
131 %
132 %  export class CodeAnalyzer {
133 %      private attemptHistory: Map<string, number> = new Map();
134 %
135 %      /**
136 %       * Detecta a linguagem do código com base em padrões
137 %       */
138 %      detectLanguage(code: string): CodeLanguage {
139 %          const scores: Record<CodeLanguage, number> = {
140 %              lua: 0, roblox: 0, javascript: 0, typescript: 0, python: 0,
141 %          };
142 %
143 %          for (const [lang, patterns] of Object.entries(LANGUAGE_PATTER
144 %              for (const pattern of patterns) {
145 %                  if (pattern.test(code)) {
146 %                      scores[lang as CodeLanguage]++;
147 %                  }
148 %              }
149 %          }
150 %
151 %          // Roblox é superset de Lua - se ambos forem detectados, esco
152 %          if (scores.roblox > 0) return 'roblox';
153 %
154 %          let best: CodeLanguage = 'unknown';
155 %          let bestScore = 0;
156 %          for (const [lang, score] of Object.entries(scores)) {
157 %              if (score > bestScore) {
158 %                  bestScore = score;
159 %                  best = lang as CodeLanguage;
160 %              }
161 %          }
162 %          return bestScore > 0 ? best : 'unknown';
163 %      }
164 %
165 %      /**
166 %       * Verifica se o texto contém código
167 %       */
168 %      hasCode(text: string): boolean {
169 %          const indicators = [
170 %              /\bfunction \w+/, /\blocal \w+/, /\bdef \w+/,
171 %              /\bconst \w+/, /\blet \w+/, /\bclass \w+/,
172 %              />= \s* \{ \{ \} /, / print \s* \{ \} /, / console \. log /,
173 %              /\[ \[ \s \S \] * \} /, /\[ \[ \[ \s \S \] * \} /,
174 %          ];
175 %          return indicators.filter(p => p.test(text)).length >= 1;
176 %      }
177 %
178 %      /**
179 %       * Detecta erros mencionados ou prováveis no código
180 %       */

```

```

181 %         detectErrors(text: string): DetectedError[] {
182 %             const errors: DetectedError[] = [];
183 %             const lower = text.toLowerCase();
184 %
185 %             for (const errorType of Object.keys(ERROR_KNOWLEDGE)) {
186 %                 if (lower.includes(errorType)) {
187 %                     // Encontrar excerpt
188 %                     const idx = lower.indexOf(errorType);
189 %                     const start = Math.max(0, idx - 30);
190 %                     const end = Math.min(text.length, idx + errorType.length);
191 %                     errors.push({
192 %                         type: errorType,
193 %                         excerpt: text.substring(start, end),
194 %                     });
195 %                 }
196 %             }
197 %
198 %             return errors;
199 %         }
200 %
201 %     /**
202 %      * Gera hipóteses (contextos de documentação) para alimentar o
203 %      * Sistema de TENTATIVAS A, B, C - se chamado várias vezes com
204 %      */
205 %     generateHypotheses(text: string, sessionId: string = 'default') {
206 %         const errors = this.detectErrors(text);
207 %         const language = this.detectLanguage(text);
208 %         const hypotheses: string[] = [];
209 %
210 %         // Para cada erro detectado, busca documentação
211 %         for (const err of errors) {
212 %             const docs = ERROR_KNOWLEDGE[err.type] || [];
213 %             if (docs.length === 0) continue;
214 %
215 %             // Sistema de tentativas: cicla entre as opções (A, B, C...)
216 %             const attemptKey = `${sessionId}:${err.type}`;
217 %             const currentAttempt = this.attemptHistory.get(attemptKey);
218 %             const nextAttempt = (currentAttempt + 1) % docs.length;
219 %             this.attemptHistory.set(attemptKey, nextAttempt);
220 %
221 %             hypotheses.push(docs[currentAttempt]);
222 %         }
223 %
224 %         // Se há código mas nenhum erro específico, adiciona contexto
225 %         if (errors.length === 0 && this.hasCode(text) && language !== '') {
226 %             hypotheses.push(`Você enviou um código em ${language}. Vou`);
227 %         }
228 %
229 %         return hypotheses;
230 %     }
231 %
232 %     /**
233 %      * Análise completa
234 %      */
235 %     analyze(text: string, sessionId: string = 'default'): CodeAnalysis {
236 %         const hasCode = this.hasCode(text);
237 %         const language = hasCode ? this.detectLanguage(text) : 'unknown';
238 %         const errors = this.detectErrors(text);
239 %         const hypotheses = this.generateHypotheses(text, sessionId);
240 %
241 %         const attemptKey = `${sessionId}:overall`;
242 %         const attemptNumber = (this.attemptHistory.get(attemptKey) || 0) + 1;
243 %         this.attemptHistory.set(attemptKey, attemptNumber);
244 %
245 %         return {
246 %             language,

```

```
2 4 7 %           h a s C o d e ,
2 4 8 %           e r r o r s ,
2 4 9 %           h y p o t h e s e s ,
2 5 0 %           a t t e m p t N u m b e r ,
2 5 1 %           } ;
2 5 2 %           }
2 5 3 %           }
2 5 4 %
```

```

1 % // =====
2 % // CLASSIFICADOR DE INTENÇÃO E TF-IDF (MoE - Mixture of Experts)
3 % // Implementa Detectores das 7 Intenções exatas, Extração de Assu
4 % // NENHUMA resposta pronta é gerada aqui.
5 % // =====
6 %
7 % export type Category =
8 %     | 'CONVERSA'
9 %     | 'PROGRAMAÇÃO'
10 %     | 'ROBLOX'
11 %     | 'LUAU'
12 %     | 'JAVASCRIPT'
13 %     | 'MATEMÁTICA'
14 %     | 'PORTUGUÊS'
15 %     | 'ERROS COMUNS'
16 %     | 'DOCUMENTAÇÃO'
17 %     | 'IA';
18 %
19 % // As 7 Intenções Exatas da Atualização Definitiva
20 % export type IntentionType =
21 %     | 'saudacao'
22 %     | 'pergunta'
23 %     | 'pedido'
24 %     | 'emocao'
25 %     | 'agradecimento'
26 %     | 'despedida'
27 %     | 'informacao'
28 %     | 'correcao'; // Mantido para o gatilho de auto-correção
29 %
30 % export type ExpertType =
31 %     | 'Especialista Roblox'
32 %     | 'Especialista Lua'
33 %     | 'Especialista Programação'
34 %     | 'Especialista IA'
35 %     | 'Especialista Conversa'
36 %     | 'Especialista Matemática';
37 %
38 % export interface ExtractedEntities {
39 %     nomes: string[];
40 %     jogos: string[];
41 %     linguagens: string[];
42 %     objetos: string[];
43 %     funcoes: string[];
44 %     erros: string[];
45 %     numeros: string[];
46 % }
47 %
48 % export interface ClassificationResult {
49 %     category: Category;
50 %     intention: IntentionType;
51 %     expertName: ExpertType;
52 %     confidence: number;
53 %     signals: string[];
54 %     subject: string; // Palavra forte calculada via TF-IDF
55 %     entities: ExtractedEntities;
56 % }
57 %
58 % const SIGNALS: Record<Category, string[]> = {
59 %     CONVERSA: [
60 %         'oi', 'olá', 'ola', 'hey', 'eai', 'eae', 'fala', 'bom dia', '
61 %         'tudo bem', 'tudo certo', 'como vai', 'beleza', 'obrigado', '
62 %         'quem é você', 'o que você faz', 'seu nome', 'amigo', 'conver
63 %     ],

```

```

64 %     PROGRAMAÇÃO: [
65 %         'código', 'codigo', 'script', 'função', 'function', 'class',
66 %         'api', 'backend', 'frontend', 'programar', 'desenvolvimento',
67 %         'javascript', 'typescript', 'react', 'html', 'css', 'node',
68 %     ],
69 %     ROBLOX: [
70 %         'roblox', 'workspace', 'players', 'replicatedstorage', 'serve
71 %         'startergui', 'humanoid', 'remoteevent', 'remotefunction', 'd
72 %     ],
73 %     LUAU: [
74 %         'luau', 'lua', 'local ', 'pairs', 'ipairs', 'metatable', 'set
75 %         'table.', 'math.', 'string.', 'rawget', 'rawset', 'repeat',
76 %     ],
77 %     JAVASCRIPT: [
78 %         'javascript', 'js', 'const ', 'let ', 'var ', 'console.log',
79 %         'fetch', 'array.map', '.filter', '.reduce', 'document.get',
80 %     ],
81 %     MATEMÁTICA: [
82 %         'matemática', 'matematica', 'cálculo', 'calculco', 'soma', 'su
83 %         'divisão', 'equação', 'porcentagem', 'álgebra', 'geometria',
84 %     ],
85 %     PORTUGUÊS: [
86 %         'português', 'portugues', 'gramática', 'verbo', 'adjetivo',
87 %         'concordância', 'ortografia', 'acentuação', 'crase', 'pontuaç
88 %     ],
89 %     'ERROS COMUNS': [
90 %         'erro', 'error', 'bug', 'falha', 'não funciona', 'nao funcion
91 %         'nil value', 'syntax error', 'cannot read property', 'infiniti
92 %         'crash', 'travou', 'pau', 'quebrou', 'linha',
93 %     ],
94 %     DOCUMENTAÇÃO: [
95 %         'documentação', 'docs', 'manual', 'guia', 'tutorial', 'referê
96 %         'como funciona a biblioteca', 'documentar', 'readme', 'api do
97 %     ],
98 %     IA: [
99 %         'ia', 'inteligência artificial', 'llm', 'modelo', 'redes neur
100 %         'tokens', 'markov', 'n-gram', 'inteligente', 'raciocínio', 'a
101 %     ]
102 % };
103 %
104 % function hasCode(text: string): boolean {
105 %     return [
106 %         /function \w+/, /local \w+\s*=/, /const \w+\s*=/, /let \w+\s*
107 %         /class \w+/, /\bif\s*\(/, /\bfor\s*\(/, />= \s*[ ( ) /, /print\s
108 %     ].some(p => p.test(text));
109 % }
110 %
111 % function hasMath(text: string): boolean {
112 %     return [
113 %         /\d+\s*[\+ \- \* \/ \^] \s* \d+/, /\b(sen|cos|tan|log|sqrt)\b/, /\b
114 %     ].some(p => p.test(text));
115 % }
116 %
117 % export class IntentClassifier {
118 %     /**
119 %      * ETAPA 1 - CLASSIFICADOR DE INTENÇÃO (7 Intenções da Atualiza
120 %      */
121 %     private detectIntention(text: string, _category: Category): Int
122 %     const normalized = text.toLowerCase().trim();
123 %
124 %     // 1. Correção explícita
125 %     if (/^(está errado|esta errado|errado|não é assim|incorreto|f
assim|piorou|corrige|corrigir)\b/i.test(normalized)) {
126 %         return 'correcao';
127 %     }
128 %

```

```

129 % // 2. Agradecimento
130 % if (/^(obrigado|obrigada|valeu|grato|agradeço|thanks|muito o b
131 %     return 'agradecimento';
132 % }
133 %
134 % // 3. Despedida
135 % if (/^(tchau|falou|flw|adeus|até logo|até mais|bye|fui)\b/.te
136 %     return 'despedida';
137 % }
138 %
139 % // 4. Emoção
140 % if (/estou (triste|feliz|animado|cansado|chateado|bravo|empol
141 %     return 'emocao';
142 % }
143 %
144 % // 5. Saudação
145 % if (/^(oi|olá|ola|eai|eae|fala|bom dia|boa tarde|boa noite)\b
146 %     // Se tiver mais palavras expressivas (ex: "Oi meu Minecraf
147 %     if (normalized.split(/ /).length <= 3) {
148 %         return 'saudacao';
149 %     }
150 % }
151 %
152 % // 6. Pedido
153 % if (/^(crie|faça|gera|me mostre|me explica|escreva|ajuda|reso
154 %     return 'pedido';
155 % }
156 %
157 % // 7. Pergunta
158 % if (/\/?$/.test(normalized) || /^(qual|como|por que|porque|que
159 %     return 'pergunta';
160 % }
161 %
162 % // 8. Informação (Relatos, explicações do usuário, ex: "meu m
163 %     return 'informacao';
164 % }
165 %
166 % /**
167 %  * Extração de Entidades
168 %  */
169 % private extractEntities(text: string): ExtractedEntities {
170 %     const entities: ExtractedEntities = {
171 %         nomes: [], jogos: [], linguagens: [], objetos: [], funcoes:
172 %     };
173 %
174 %     const lower = text.toLowerCase();
175 %
176 %     if (lower.includes('roblox')) entities.jogos.push('roblox');
177 %     if (lower.includes('minecraft')) entities.jogos.push('minecra
178 %     if (lower.includes('free fire')) entities.jogos.push('free fi
179 %     if (lower.includes('fortnite')) entities.jogos.push('fortnite
180 %
181 %     if (lower.includes('lua')) entities.linguagens.push('lua');
182 %     if (lower.includes('javascript') || lower.includes('js')) ent
183 %     if (lower.includes('typescript') || lower.includes('ts')) ent
184 %     if (lower.includes('python')) entities.linguagens.push('pytho
185 %     if (lower.includes('html')) entities.linguagens.push('html');
186 %     if (lower.includes('css')) entities.linguagens.push('css');
187 %
188 %     const objectKeywords = ['workspace', 'players', 'replicatedst
'part', 'document', 'window', 'math', 'table'];
189 %     for (const obj of objectKeywords) {
190 %         if (lower.includes(obj)) entities.objetos.push(obj);
191 %     }
192 %
193 %     const fnMatches = text.match(/\/b([a-zA-Z0-9_]+)\s*(\/g);

```

```

194 %         if (fnMatches) entities.funcoes = fnMatches.map(f => f.replace
195 %         if (lower.includes('function ')) {
196 %             const match = text.match(/function ([a-zA-Z0-9_]+)/);
197 %             if (match) entities.funcoes.push(match[1]);
198 %         }
199 %
200 %         const linhaMatch = text.match(/linha \d+/i);
201 %         if (linhaMatch) entities.erros.push(linhaMatch[0].toLowerCase
202 %
203 %         const errorMatches = ['attempt to index nil', 'nil value', 's
'missing end', 'travou'];
204 %         for (const err of errorMatches) {
205 %             if (lower.includes(err)) entities.erros.push(err);
206 %         }
207 %
208 %         const numMatches = text.match(/\b\d+(\.\d+)?\b/g);
209 %         if (numMatches) entities.numeros = numMatches;
210 %
211 %         const nameMatches = text.match(/\b[A-Z][a-z]+\b/g);
212 %         if (nameMatches) {
213 %             const knownNouns = new Set(['Oi', 'Ola', 'Olá', 'Bom', 'Boa', 'Q
Onde', 'Lua', 'Roblox', 'JavaScript', 'TypeScript', 'Python', 'Sim', 'Não', 'Nao']);
214 %             for (const n of nameMatches) {
215 %                 if (!knownNouns.has(n) && n.length > 2) {
216 %                     entities.nomes.push(n);
217 %                 }
218 %             }
219 %         }
220 %
221 %         return entities;
222 %     }
223 %
224 %     /**
225 %     * MoE - Mixture of Experts
226 %     */
227 %     private selectExpert(category: Category, intention: IntentionTy
228 %         if (category === 'ROBLOX') return 'Especialista Roblox';
229 %         if (category === 'LUAU') return 'Especialista Lua';
230 %         if (category === 'IA') return 'Especialista IA';
231 %         if (category === 'MATEMÁTICA') return 'Especialista Matemática
232 %         if (['PROGRAMAÇÃO', 'JAVASCRIPT', 'ERROS COMUNS', 'DOCUMENTAÇ
233 %             return 'Especialista Programação';
234 %         }
235 %         return 'Especialista Conversa';
236 %     }
237 %
238 %     /**
239 %     * ETAPA 2 - EXTRAÇÃO DE ASSUNTO PRINCIPAL (TF-IDF Local)
240 %     * Encontra a Palavra Forte (ex: "Oi meu Minecraft travou hoje"
241 %     */
242 %     private detectSubjectTFIDF(text: string, category: Category, en
243 %         const words = text.toLowerCase().match(/[wÀ-ú]+/g) || [];
244 %         const stopWords = new Set(['oi', 'ola', 'olá', 'bom', 'dia', 'tard
', 'de', 'do', 'da', 'em', 'um', 'uma', 'para', 'com', 'que', 'é', 'qual', 'como', 'por', 'tudo', 'bem']);
245 %
246 %         // Prioridade Máxima: Entidades Extraídas (O assunto de maior
247 %         if (entities.jogos.length > 0) return entities.jogos[0];
248 %         if (entities.linguagens.length > 0) return entities.linguagen
249 %         if (entities.erros.length > 0) return entities.erros[0];
250 %         if (entities.objetos.length > 0) return entities.objetos[0];
251 %
252 %         // Calculo de frequência/peso para as palavras restantes
253 %         const meaningful = words.filter(w => w.length > 2 && !stopWor
254 %
255 %         if (meaningful.length > 0) {
256 %             // Retorna a palavra mais expressiva/longa como o assunto c

```



```

257 %         return meaningful.sort((a, b) => b.length - a.length)[0];
258 %     }
259 %     return category.toLowerCase();
260 % }
261 %
262 % classify(message: string): ClassificationResult {
263 %     const normalized = message.toLowerCase().trim();
264 %     const scores: Record<string, { score: number; signals: string[] }> = {};
265 %
266 %     for (const cat of Object.keys(SIGNALS) as Category[]) {
267 %         scores[cat] = { score: 0, signals: [] };
268 %     }
269 %
270 %     for (const [category, signals] of Object.entries(SIGNALS)) {
271 %         for (const signal of signals) {
272 %             if (normalized.includes(signal)) {
273 %                 const weight = signal.length > 5 ? 2 : 1;
274 %                 scores[category].score += weight;
275 %                 scores[category].signals.push(signal);
276 %             }
277 %         }
278 %     }
279 %
280 %     if (hasCode(message)) {
281 %         scores.PROGRAMAÇÃO.score += 2;
282 %         scores.PROGRAMAÇÃO.signals.push('[código detectado]');
283 %     }
284 %
285 %     if (hasMath(message)) {
286 %         scores.MATEMÁTICA.score += 3;
287 %         scores.MATEMÁTICA.signals.push('[fórmula/cálculo detectado]');
288 %     }
289 %
290 %     const wordCount = normalized.split(/ /).length;
291 %     if (wordCount <= 3 && scores.CONVERSA.score === 0 && !hasCode(message)) {
292 %         scores.CONVERSA.score = 1;
293 %         scores.CONVERSA.signals.push('[mensagem curta]');
294 %     }
295 %
296 %     let bestCategory: Category = 'CONVERSA';
297 %     let bestScore = 0;
298 %     for (const [cat, data] of Object.entries(scores)) {
299 %         if (data.score > bestScore) {
300 %             bestScore = data.score;
301 %             bestCategory = cat as Category;
302 %         }
303 %     }
304 %
305 %     if (bestScore === 0) {
306 %         bestCategory = 'CONVERSA';
307 %         bestScore = 0.1;
308 %     }
309 %
310 %     const totalSignals = Object.values(scores).reduce((a, b) => a.concat(b.signals), []);
311 %     const confidence = totalSignals.length > 0 ? bestScore / totalSignals.length : 0;
312 %
313 %     const intention = this.detectIntention(message, bestCategory);
314 %     const expertName = this.selectExpert(bestCategory, intention);
315 %     const entities = this.extractEntities(message);
316 %     const subject = this.detectSubjectTFIDF(message, bestCategory);
317 %
318 %     return {
319 %         category: bestCategory,
320 %         intention,
321 %         expertName,
322 %         confidence,

```

```
3 2 3 %           s i g n a l s :   s c o r e s [ b e s t C a t e g o r y ] . s i g n a l s ,
3 2 4 %           s u b j e c t ,
3 2 5 %           e n t i t i e s ,
3 2 6 %           } ;
3 2 7 %       }
3 2 8 %   }
3 2 9 %
```

```
1 % // =====
2 % // CORPUS DE CONVERSACÃO E PORTUGUÊS BRASILEIRO DEFINITIVO
3 % // Milhares de exemplos reais de diálogos, perguntas, respostas e
4 % // NENHUMA resposta pronta ou par fixo. O modelo extrai padrões s
5 % // =====
6 %
7 % export const CONVERSATION_PT_CORPUS = `
8 % Bom dia amigo. O dia começou animado hoje. O sol brilha forte lá
9 % Boa tarde colega. Espero que esteja tendo um ótimo dia de trabalh
10 % Boa noite guerreiro. Chegou o momento de descansar e recarregar a
11 % Olá. Como vai você. Já fazia um tempo desde a nossa última troca
12 % Oi. Tudo joia com você. Como andam as novidades e o andamento do
13 % Tudo bem. Eu estou espetacular, muito obrigado por perguntar. E c
fluidez.
14 % MUITÍSSIMO obrigado pela sua atenção e carinho. De nada, é sempre
15 % Eu sou o seu assistente inteligente. Um sistema generativo basead
16 % O que você gostaria de investigar ou desenvolver hoje. Tenho conh
de falhas.
17 % Sinta-se inteiramente livre para perguntar o que quiser. Minha me
sua dúvida.
18 % A vida é uma longa estrada de aprendizado contínuo. Todos os dias
pensar.
19 % O universo da tecnologia evolui a passos largos. O desenvolviment
lógico bem afiado.
20 % Amo resolver problemas de arquitetura e lógica. Escrever código l
técnica.
21 % Aprender conceitos profundos exige paciência e muita prática. Dep
22 % Jamais desanime diante de um erro no console. Cada exceção ou fal
funcionamento interno do sistema.
23 % A internet interliga mentes criativas ao redor de todo o planeta.
programadores.
24 % A leitura aprofundada de documentações técnicas encurta o caminho
eficientes.
25 % Como posso prestar auxílio no seu projeto hoje. Em qual funcional
atualmente.
26 % Que excelente notícia. Acho o seu ponto de vista de extrema intel
27 % Vamos raciocinar juntos sobre a estrutura do sistema. Posso suger
rotina.
28 % Você poderia experimentar refatorar o escopo dessa forma. Tenho p
software.
29 % Caso necessite de suporte adicional em qualquer módulo, basta me
você.
30 % A dedicação contínua sempre gera frutos notáveis. O esforço intel
qualidade final.
31 % O desenvolvimento de um jogo no Roblox exige planejamento, imagin
32 % A base de conhecimento foi estruturada para separar claramente ca
contexto.
33 % Entendi perfeitamente o seu raciocínio. Acompanhei cada passo da
a questão.
34 % Poxa, que espetáculo de pergunta. Gosto muito quando exploramos c
35 % Hmm, deixe-me analisar essa premissa com bastante atenção. Existe
as tabelas.
36 % Olha só que sacada brilhante. Não tinha contemplado essa otimizaç
projeto.
37 % Uau, faz total sentido. Essa modularização elimina dependências d
simples.
38 % Compreendo a sua frustração com esse obstáculo técnico. Vamos res
raiz do bug.
39 % Exatamente, você capturou a essência do problema. O escopo da var
assíncrono.
40 % Com certeza, podemos implementar essa melhoria agora mesmo. Vamos
confiança.
41 % `;
```

```
4 2 %  
4 3 %   e x p o r t   c o n s t   C O N V E R S A T I O N _ E N _ C O R P U S   =   `  
4 4 %   H e l l o   f r i e n d .   W e l c o m e   b a c k   t o   t h e   m a i n   c o n s o l e .   I   a m   a n   i n t e l l i g e  
logic.  
4 5 %   G o o d   m o r n i n g .   L e t   u s   m a k e   t o d a y   i n c r e d i b l y   p r o d u c t i v e .   T e l l   m e   a b  
now.  
4 6 %   G o o d   a f t e r n o o n .   I   h o p e   y o u r   d e v e l o p m e n t   s e s s i o n   i s   p r o c e e d i n g   s m o  
4 7 %   G o o d   e v e n i n g .   T a k e   s o m e   t i m e   t o   r e s t   a n d   c l e a r   y o u r   m i n d .   A   f r e s h  
bugs.  
4 8 %   T h a n k   y o u   v e r y   m u c h   f o r   t h e   f e e d b a c k .   Y o u   a r e   e n t i r e l y   w e l c o m e .   I  
task.  
4 9 %   ` ;  
5 0 %
```

```

1 % // =====
2 % // DATABASE SCHEMA - PROJETO DEFINITIVO MINI-LLM
3 % // Armazenamento estruturado, isolado por usuário, preparado para
4 % // =====
5 %
6 % import Dexie from 'dexie';
7 %
8 % export interface User {
9 %     id?: number;
10 %     uuid: string;
11 %     name: string;
12 %     likesCount?: number; // Pontuação para o Ranking de Usuários (e
13 %     createdAt: Date;
14 % }
15 %
16 % export interface Message {
17 %     id?: number;
18 %     userId: string;
19 %     role: 'user' | 'assistant';
20 %     content: string;
21 %     category?: string;
22 %     intention?: string;
23 %     contextVector?: string;
24 %     score?: number;
25 %     isCorrection?: boolean;
26 %     expertName?: string; // Nome do Especialista selecionado (MoE)
27 %     timestamp: Date;
28 % }
29 %
30 % export interface KnowledgeDoc {
31 %     id?: number;
32 %     title: string;
33 %     content: string;
34 %     category: string; // CONVERSA, PROGRAMAÇÃO, ROBLOX, LUAU, JAVAS
35 %     createdAt: Date;
36 % }
37 %
38 % export interface UserMemoryFact {
39 %     id?: number;
40 %     userId: string; // ID único do usuário (Isolamento absoluto)
41 %     key: string; // ex: 'projeto_atual', 'preferencia_linguagem'
42 %     value: string;
43 %     createdAt: Date;
44 % }
45 %
46 % // Fila de Aprendizado Definitiva (Approved / Neutral / Rejected)
47 % export interface LearningQueueItem {
48 %     id?: number;
49 %     responseId: string;
50 %     userId: string;
51 %     userName: string;
52 %     question: string;
53 %     answer: string;
54 %     correctedAnswer?: string; // Se um dev corrigiu manualmente no
55 %     category: string;
56 %     intention: string;
57 %     contextVector: string;
58 %     status: 'approved' | 'neutral' | 'rejected';
59 %     likes: number; // +1
60 %     dislikes: number; // -1
61 %     superLikes: number; // +10
62 %     superLike: boolean;
63 %     timestamp: Date;

```

```

64 % }
65 %
66 % // Corretor Humano (Armazena as intervenções diretas de desenvol
67 % export interface HumanCorrection {
68 %     id?: number;
69 %     entrada: string;
70 %     resposta_ruim: string;
71 %     resposta_corrigida: string;
72 %     score: number;
73 %     timestamp: Date;
74 % }
75 %
76 % export class GenerativeDB extends Dexie {
77 %     users!: Dexie.Table<User, number>;
78 %     messages!: Dexie.Table<Message, number>;
79 %     knowledge!: Dexie.Table<KnowledgeDoc, number>;
80 %     facts!: Dexie.Table<UserMemoryFact, number>;
81 %     learning_queue!: Dexie.Table<LearningQueueItem, number>;
82 %     human_corrections!: Dexie.Table<HumanCorrection, number>;
83 %
84 %     constructor() {
85 %         super('DevMind_Definitive_AI_v5');
86 %         this.version(1).stores({
87 %             users: '++id, uuid, name, likesCount',
88 %             messages: '++id, userId, timestamp, category',
89 %             knowledge: '++id, title, category',
90 %             facts: '++id, userId, key',
91 %             learning_queue: '++id, responseId, userId, category, status',
92 %             human_corrections: '++id, entrada, timestamp',
93 %         });
94 %     }
95 % }
96 %
97 % export const db = new GenerativeDB();
98 %

```

```

1 % // =====
2 % // PERSISTÊNCIA BLINDADA E BACKUP REDUNDANTE
3 % // Protege o IndexedDB contra eliminação do navegador via navegador
4 % // e mantém um espelho redundante duplo no localStorage para evitar
5 % // =====
6 %
7 % import { db } from './schema';
8 % import { auditLog } from '../logs/logger';
9 %
10 % export class PersistenceGuard {
11 %     private BACKUP_KEY = 'DEV_MIND_SUPER_PERMANENT_BACKUP_V5';
12 %
13 %     constructor() {}
14 %
15 %     /**
16 %      * Solicita persistência de alto nível ao navegador para blindar
17 %      */
18 %     public async requestPersistentStorage(): Promise<boolean> {
19 %         try {
20 %             if (navigator.storage && navigator.storage.persist) {
21 %                 const persistent = await navigator.storage.persist();
22 %                 auditLog('STORAGE_PERSISTENCE', { persistent });
23 %                 return persistent;
24 %             }
25 %         } catch (e) {
26 %             console.warn('Navegador não suporta request de persistência');
27 %         }
28 %         return false;
29 %     }
30 %
31 %     /**
32 %      * Realiza backup redundante no localStorage para todas as tabelas
33 %      */
34 %     public async createMirrorBackup(): Promise<void> {
35 %         try {
36 %             const users = await db.users.toArray();
37 %             const knowledge = await db.knowledge.toArray();
38 %             const facts = await db.facts.toArray();
39 %             const learning_queue = await db.learning_queue.toArray();
40 %             const human_corrections = await db.human_corrections.toArray();
41 %
42 %             const snapshot = {
43 %                 users,
44 %                 knowledge,
45 %                 facts,
46 %                 learning_queue,
47 %                 human_corrections,
48 %                 timestamp: Date.now()
49 %             };
50 %
51 %             const serialized = JSON.stringify(snapshot);
52 %             localStorage.setItem(this.BACKUP_KEY, serialized);
53 %             auditLog('SUPER_PERMANENT_BACKUP_SAVED', { sizeBytes: serialized.length });
54 %         } catch (e) {
55 %             console.warn('Erro ao criar backup redundante no localStorage');
56 %         }
57 %     }
58 %
59 %     /**
60 %      * Verifica se o IndexedDB foi apagado pelo navegador e restaura
61 %      */
62 %     public async restoreMirrorIfLost(): Promise<{ restored: boolean }> {
63 %         try {

```

```

64 %         const userCount = await db.users.count();
65 %         const queueCount = await db.learning_queue.count();
66 %
67 %         // Se o banco perdeu os usuários ou fila de aprendizado, a
68 %         if (userCount === 0 || queueCount === 0) {
69 %             const backupData = localStorage.getItem(this.BACKUP_KEY);
70 %             if (backupData) {
71 %                 const snapshot = JSON.parse(backupData);
72 %
73 %                 if (snapshot.users && snapshot.users.length > 0) {
74 %                     await db.users.bulkAdd(snapshot.users.map((u: any) =>
75 %                 })
76 %                 if (snapshot.knowledge && snapshot.knowledge.length > 0) {
77 %                     await db.knowledge.bulkAdd(snapshot.knowledge.map((k:
78 %                 })
79 %                 if (snapshot.facts && snapshot.facts.length > 0) {
80 %                     await db.facts.bulkAdd(snapshot.facts.map((f: any) =>
81 %                 })
82 %                 if (snapshot.learning_queue && snapshot.learning_queue.
83 %                     await db.learning_queue.bulkAdd(snapshot.learning_que
})));
84 %             }
85 %             if (snapshot.human_corrections && snapshot.human_correc
86 %                 await db.human_corrections.bulkAdd(snapshot.human_cor
Date(h.timestamp) }));
87 %             }
88 %
89 %             auditLog('SUPER_PERMANENT_BACKUP_RESTORED', { snapshotT
90 %                 return { restored: true, details: `Restauração Completa
Date(snapshot.timestamp).toLocaleDateString()}` };
91 %             }
92 %         }
93 %     } catch (e) {
94 %         console.error('Falha ao tentar restaurar backup super perma
95 %     }
96 %     return { restored: false, details: 'Banco de dados Dexie inta
97 % }
98 % }
99 %

```



```

1 % // =====
2 % // SISTEMA DE AGENTES A e B COM RACIOCÍNIO MULTIETAPA (10 Etapas
3 % // Implementa o fluxo obrigatório de 10 passos visíveis no Chain
4 % // verificação de Frase Completa (SIM/NÃO) e trava absoluta contr
5 % // NENHUMA resposta pronta é usada. 100% focado em raciocínio pur
6 % // =====
7 %
8 % import { StatisticalLanguageModel } from './language_model';
9 % import { SearchResult } from './semantic_search';
10 % import { GrammarFinalizer } from './grammar_finalizer';
11 % import type { Category, IntentionType, ExpertType, ExtractedEntities
12 %
13 % export interface AgentBReview {
14 %     approved: boolean;
15 %     reason: string;
16 %     suggestedFix?: string;
17 %     qualityScore: number;
18 %     confidence: number;
19 %     chainOfThought: string[]; // 0 processo de pensamento explícito
20 %     finalizerNotes: string[]; // Registro de intervenções do Reescr
21 %     isSentenceComplete: boolean; // ETAPA 9 - FRASE COMPLETA? SIM O
22 %     hallucinationWarning?: string;
23 %     selfEvaluation?: {
24 %         fazSentido: boolean;
25 %         estavaCompleta: boolean;
26 %         respondeuCorretamente: boolean;
27 %     };
28 % }
29 %
30 % export class AgentSystem {
31 %     private finalizer = new GrammarFinalizer();
32 %
33 %     constructor(private model: StatisticalLanguageModel) {}
34 %
35 %     /**
36 %      * Agente B: Revisor da resposta gerada com análise estrutural,
37 %      */
38 %     reviewResponse(
39 %         generatedText: string,
40 %         userMessage: string,
41 %         searchResults: SearchResult[],
42 %         category: Category,
43 %         intention: IntentionType,
44 %         expert: ExpertType,
45 %         entities: ExtractedEntities,
46 %         gostaDeCount: number,
47 %         subjectTFIDF: string
48 %     ): { review: AgentBReview; polishedText: string } {
49 %         const userNormalized = userMessage.toLowerCase().trim();
50 %         const chainOfThought: string[] = [];
51 %         const finalizerNotes: string[] = [];
52 %
53 %         // FLUXO OBRIGATÓRIO DE 10 ETAPAS (Atualização Definitiva)
54 %         chainOfThought.push(`[Etapa 1: Classificar Intenção] Mensagem
"${expert}".`);
55 %         const entitySummary = Object.entries(entities)
56 %             .filter(([key, values]) => values.length > 0)
57 %             .map(([key, values]) => `${key}: ${values.join(', ')}`)
58 %             .join(' | ') || 'nenhuma entidade forte';
59 %         chainOfThought.push(`[Etapa 2: Extrair Assunto Principal] Pal
${entitySummary}.`);
60 %         chainOfThought.push(`[Etapa 3: Buscar Memória] Perfil carrega
61 %         chainOfThought.push(`[Etapa 4: Buscar Respostas Aprovadas] Ch

```

```

Likes).`);
62 %         chainOfThought.push(`[Etapa 5: Buscar Histórico Recente] Inje
63 %         chainOfThought.push(`[Etapa 6: Calcular Score] Calculando rel
64 %         chainOfThought.push(`[Etapa 7: Gerar Resposta] Agente A emit
65 %
66 %         // =====
67 %         // ETAPA 8 - CORRIGIR GRAMÁTICA (Reescritor Automático)
68 %         // =====
69 %         let workingText = generatedText.trim();
70 %
71 %         const rewriteResult = this.finalizer.rewriteIfDry(workingText
72 %         if (rewriteResult.rewrote) {
73 %             workingText = rewriteResult.text;
74 %             finalizerNotes.push(rewriteResult.notice || 'Reescritor Aut
75 %             chainOfThought.push(`[Etapa 8: Corrigir Gramática] Reescri
inicial.`);
76 %         } else {
77 %             chainOfThought.push(`[Etapa 8: Corrigir Gramática] A formul
78 %         }
79 %
80 %         // =====
81 %         // ETAPA 9 - VERIFICAR SE A FRASE TERMINOU (Frase Completa? S
82 %         // =====
83 %         const finalizeResult = this.finalizer.checkAndRepairSentence(
84 %         workingText = finalizeResult.text;
85 %         const isSentenceComplete = finalizeResult.isSentenceComplete;
86 %
87 %         if (finalizeResult.repaired) {
88 %             finalizerNotes.push(finalizeResult.actionExplanation || 'Fi
89 %             chainOfThought.push(`[Etapa 9: Verificar se a Frase Termin
injetou coesão (Resultado: SIM).`);
90 %         } else {
91 %             chainOfThought.push(`[Etapa 9: Verificar se a Frase Termin
cortes.`);
92 %         }
93 %
94 %         const normalized = workingText.toLowerCase().trim();
95 %         const words = (normalized.match(/[\\wÀ-ú]+/g) || []).length;
96 %         const userWords = (userNormalized.match(/[\\wÀ-ú]+/g) || []).l
97 %
98 %         // =====
99 %         // REVISOR B (Garantia de Qualidade e Trava contra lixo)
100 %         // =====
101 %         if (words < 2) {
102 %             chainOfThought.push(`[Falha Crítica] A resposta finalizada
103 %             return {
104 %                 review: { approved: false, reason: 'Resposta muito curta
chainOfThought, finalizerNotes, isSentenceComplete: false },
105 %                 polishedText: workingText
106 %             };
107 %         }
108 %
109 %         if (userWords > 2 && words <= 2 && ['CONVERSA', 'AJUDA', 'ROB
110 %             if (/^(oi|outra|o que|o que isso|sim|nao|não|ola|olá|beleza
111 %                 chainOfThought.push(`[Falha Semântica] O usuário enviou u
regeneração rica.`);
112 %                 return {
113 %                     review: { approved: false, reason: 'O usuário enviou um
genérico.', qualityScore: 0.15, confidence: 0.25, chainOfThought, finalizerNotes, isSentenceComplete },
114 %                     polishedText: workingText
115 %                 };
116 %             }
117 %         }
118 %
119 %         if (normalized === userNormalized) {
120 %             chainOfThought.push(`[Falha de Espelho] A resposta espelhou

```

```

1 2 1 %         return {
1 2 2 %             review: { approved: false, reason: 'A resposta espelhou e
confidence: 0.1, chainOfThought, finalizerNotes, isSentenceComplete },
1 2 3 %             polishedText: workingText
1 2 4 %         };
1 2 5 %     }
1 2 6 %
1 2 7 %     const allContextTerms = new Set<string>();
1 2 8 %     for (const result of searchResults) {
1 2 9 %         const terms = result.text.toLowerCase().match(/[ \wÀ-ú]+/g);
1 3 0 %         for (const t of terms) {
1 3 1 %             if (t.length > 2) allContextTerms.add(t);
1 3 2 %         }
1 3 3 %     }
1 3 4 %
1 3 5 %     const responseTerms = normalized.match(/[ \wÀ-ú]+/g) || [];
1 3 6 %     const relevantTerms = responseTerms.filter(t => allContextTer
1 3 7 %     const relevanceRatio = relevantTerms / Math.max(1, responseTe
1 3 8 %
1 3 9 %     const uniqueTermsCount = new Set(responseTerms).size;
1 4 0 %     const uniqueRatio = uniqueTermsCount / Math.max(1, responseTe
1 4 1 %
1 4 2 %     if (uniqueRatio < 0.45 && words > 6) {
1 4 3 %         chainOfThought.push(`[Falha de Repetição] A resposta possui
1 4 4 %         return {
1 4 5 %             review: { approved: false, reason: 'A resposta possui mui
0.25, confidence: 0.3, chainOfThought, finalizerNotes, isSentenceComplete },
1 4 6 %             polishedText: workingText
1 4 7 %         };
1 4 8 %     }
1 4 9 %
1 5 0 %     let hallucinationWarning: string | undefined = undefined;
1 5 1 %     let confidence = 0.85;
1 5 2 %
1 5 3 %     const isTechnicalCategory = ['PROGRAMAÇÃO', 'ROBLOX', 'LUAU',
'IA'].includes(category);
1 5 4 %
1 5 5 %     if (isTechnicalCategory) {
1 5 6 %         const hasGoodDoc = searchResults.length > 0 && searchResult
1 5 7 %         const hasGoodMemory = gostaDeCount > 0;
1 5 8 %         const hasContextOverlap = relevanceRatio > 0.15;
1 5 9 %
1 6 0 %         if (!hasGoodDoc && !hasGoodMemory && !hasContextOverlap) {
1 6 1 %             confidence = 0.40;
1 6 2 %             hallucinationWarning = `ETAPA 11 [Detector de Alucinação]
pois não encontrei documentação oficial exata na base RAG. A resposta reflete uma estimativa conceitual.`;
1 6 3 %             chainOfThought.push(`[Detector de Alucinação] Ausência de
resposta.`);
1 6 4 %             if (!workingText.includes('Não tenho certeza absoluta'))
1 6 5 %                 workingText += ` (Nota de transparência: Não tenho cert
documentos exatos na base de conhecimento).`;
1 6 6 %         }
1 6 7 %     }
1 6 8 % }
1 6 9 %
1 7 0 % let qualityScore = 0.40;
1 7 1 % qualityScore += relevanceRatio * 0.35;
1 7 2 % if (words >= 3 && words <= 50) qualityScore += 0.25;
1 7 3 % if (uniqueRatio > 0.6) qualityScore += 0.20;
1 7 4 %
1 7 5 % if (isTechnicalCategory && relevanceRatio < 0.15 && words < 1
1 7 6 %     chainOfThought.push(`[Falha Técnica] Resposta técnica vaga
1 7 7 %     return {
1 7 8 %         review: { approved: false, reason: 'Resposta técnica vaga
qualityScore: Math.max(0, qualityScore - 0.4), confidence: 0.3, chainOfThought, finalizerNotes, isSentenceComplete },
1 7 9 %         polishedText: workingText

```

```

180 %         };
181 %     }
182 %
183 %     qualityScore = Math.min(1, Math.max(0, qualityScore));
184 %     const approved = qualityScore >= 0.60;
185 %
186 %     const fazSentido = qualityScore >= 0.60;
187 %     const estavaCompleta = isSentenceComplete && words >= 3;
188 %     const respondeuCorretamente = isTechnicalCategory ? (relevanc
189 %
190 %     chainOfThought.push(`[Autoavaliação] Minha resposta faz senti
${estavaCompleta ? 'SIM' : 'NÃO'}. Respondeu corretamente? ${respondeuCorretamente ? 'SIM' : 'NÃO'}.`);
191 %
192 %     // ETAPA 10 - SOMENTE ENTÃO ENVIAR
193 %     chainOfThought.push(`[Etapa 10: Somente Então Enviar] Verific
194 %
195 %     return {
196 %         review: {
197 %             approved,
198 %             reason: approved ? `Aprovada pelo Agente B com índice de
pelo Agente B: Índice ${((qualityScore * 100).toFixed(0))}% (Mínimo 60%). Exigindo segunda tentativa.`
199 %             qualityScore,
200 %             confidence,
201 %             chainOfThought,
202 %             finalizerNotes,
203 %             isSentenceComplete,
204 %             hallucinationWarning,
205 %             selfEvaluation: { fazSentido, estavaCompleta, respondeuCo
206 %         },
207 %         polishedText: workingText
208 %     };
209 % }
210 %
211 % /**
212 %  * Pipeline completa: 10 Etapas da Atualização Definitiva + Age
213 %  */
214 % async *generateWithReview(
215 %     promptForGenerator: string,
216 %     userMessage: string,
217 %     searchResults: SearchResult[],
218 %     category: Category,
219 %     intention: IntentionType,
220 %     expert: ExpertType,
221 %     maxTokens: number,
222 %     temperature: number,
223 %     biases: string[],
224 %     entities: ExtractedEntities,
225 %     gostadeCount: number,
226 %     subjectTFIDF: string
227 % ): AsyncGenerator<{
228 %     type: 'token' | 'review' | 'retry' | 'final' | 'cot';
229 %     token?: { token: string; prob: number; lookaheadScore?: number;
230 %     review?: AgentBReview;
231 %     generatedText?: string;
232 %     cotMessage?: string;
233 % } > {
234 %     // Transmite o Chain of Thought narrando as etapas do raciocí
235 %     yield { type: 'cot', cotMessage: `ð>ÿà [Etapa 1] Classificando
236 %     await new Promise(r => setTimeout(r, 200));
237 %     yield { type: 'cot', cotMessage: `ð=ÿ [Etapas 2 e 3] Extraíndo
memória...` };
238 %     await new Promise(r => setTimeout(r, 200));
239 %     yield { type: 'cot', cotMessage: `ð=Ûi [Etapas 4, 5 e 6] Buscan
240 %     await new Promise(r => setTimeout(r, 200));
241 %     yield { type: 'cot', cotMessage: `ð=þ€ [Etapa 7] Gerando respos
242 %

```

```

243 %         let generatedText = '';
244 %         const tokens: { token: string; prob: number; lookaheadScore?:
245 %
246 %         for await (const tk of this.model.generateStream(
247 %             promptForGenerator,
248 %             maxTokens,
249 %             temperature,
250 %             category,
251 %             biases,
252 %             userMessage
253 %         )) {
254 %             const isPunct = /^[.,?!:;]$/ .test(tk.token);
255 %             if (generatedText.length > 0 && !isPunct && tk.token !== '\
256 %                 generatedText += ' ';
257 %             }
258 %             generatedText += tk.token === '\n' ? ' ' : tk.token;
259 %             tokens.push(tk);
260 %             yield { type: 'token', token: tk };
261 %         }
262 %
263 %         generatedText = generatedText.trim();
264 %
265 %         // === AGENTE B REVISAR, CORRIGIR E VERIFICAR FRASE FINAL (ETAPA
266 %         yield { type: 'cot', cotMessage: `ð=þáþ [Etapas 8 e 9] Agente B
Completa? SIM)...` };
267 %         const { review, polishedText } = this.reviewResponse(generate
entities, gostaDeCount, subjectTFIDF);
268 %         yield { type: 'review', review, generatedText: polishedText };
269 %
270 %         if (review.approved) {
271 %             yield { type: 'final', generatedText: polishedText };
272 %             return;
273 %         }
274 %
275 %         // === SEGUNDA TENTATIVA (AGENTE B REPROVOU E COORDENA NOVA G
276 %         yield { type: 'retry', generatedText: polishedText };
277 %         yield { type: 'cot', cotMessage: `ð=Ÿ [CoT] Agente B insatisfe
sob supervisão do ${expert}...` };
278 %
279 %         let retryText = '';
280 %         const retryTokens: { token: string; prob: number; lookaheadSc
281 %
282 %         const lowerTemp = Math.max(0.15, temperature * 0.40);
283 %         const refinedPrompt = `${promptForGenerator} [ESTRUTURA COMPL
284 %
285 %         for await (const tk of this.model.generateStream(
286 %             refinedPrompt,
287 %             Math.floor(maxTokens * 0.85),
288 %             lowerTemp,
289 %             category,
290 %             biases,
291 %             userMessage
292 %         )) {
293 %             const isPunct = /^[.,?!:;]$/ .test(tk.token);
294 %             if (retryText.length > 0 && !isPunct && tk.token !== '\n')
295 %                 retryText += ' ';
296 %             }
297 %             retryText += tk.token === '\n' ? ' ' : tk.token;
298 %             retryTokens.push(tk);
299 %             yield { type: 'token', token: tk };
300 %         }
301 %
302 %         retryText = retryText.trim();
303 %
304 %         const retryPolished = this.reviewResponse(retryText, userMess
gostaDeCount, subjectTFIDF);

```

```
305 %         yield { type: 'review', review: retryPolished.review, generat
306 %         yield { type: 'final', generatedText: retryPolished.polishedT
307 %     }
308 % }
309 %
```



```

62 %         keywords: allTerms.slice(0, 8),
63 %     };
64 % }
65 %
66 %     public serializeVector(vector: ContextVector): string {
67 %         return JSON.stringify(vector);
68 %     }
69 %
70 %     public deserializeVector(json: string): ContextVector | null {
71 %         try {
72 %             return JSON.parse(json) as ContextVector;
73 %         } catch {
74 %             return null;
75 %         }
76 %     }
77 %
78 %     public validateCoherence(generatedSentence: string, vector: ContextVector): boolean {
79 %         const sentenceTerms = this.extractMeaningfulTerms(generatedSentence);
80 %         if (sentenceTerms.length === 0) return { valid: false, score: 0 };
81 %
82 %         if (vector.expectedStructure === 'curta' && sentenceTerms.length > 0) {
83 %             return { valid: false, score: 0.15 };
84 %         }
85 %
86 %         let matchCount = 0;
87 %         for (const term of sentenceTerms) {
88 %             if (vector.keywords.includes(term) || term.includes(vector.keywords[0])) {
89 %                 matchCount += 1;
90 %             }
91 %         }
92 %
93 %         const baseScore = Math.min(1, (matchCount / Math.max(1, sentenceTerms.length)));
94 %
95 %         return {
96 %             valid: baseScore >= 0.25,
97 %             score: baseScore,
98 %         };
99 %     }
100 % }
101 %

```


Ø=ÜÄ src/generation/embeddings_engine.ts

```
1 % // =====
2 % // EMBEDDINGS ENGINE - Mapeamento Semântico e Vetorial Avançado
3 % // Simula embeddings de alta densidade para compreensão cirúrgica
4 % // =====
5 %
6 % export interface LexicalVector {
7 %     word: string;
8 %     vector: number[]; // [0: Tech, 1: Social, 2: Bug/Erro, 3: Dúvida]
Lógica/Math
9 % }
10 %
11 % export class EmbeddingsEngine {
12 %     private dictionary: Map<string, number[]> = new Map();
13 %     private cache: Map<string, number[]> = new Map();
14 %
15 %     constructor() {
16 %         this.initializeConceptVectors();
17 %     }
18 %
19 %     /**
20 %      * Inicializa vetores conceituais densos para mapeamento semântico
21 %      */
22 %     private initializeConceptVectors() {
23 %         // === CONVERSA & SOCIAL ===
24 %         this.registerConcept(['oi', 'olá', 'ola', 'eai', 'eae', 'fala', 'he
0.0, 0.0]);
25 %         this.registerConcept(['tudo', 'bem', 'certo', 'joia', 'beleza', 'ó
0.0, 0.0]);
26 %         this.registerConcept(['obrigado', 'obrigada', 'valeu', 'agradeço
27 %         this.registerConcept(['tchau', 'falou', 'flw', 'adeus', 'bye', 'fu
28 %         this.registerConcept(['sim', 'exato', 'isso', 'concordo', 'claro'
0.1]);
29 %         this.registerConcept(['não', 'nao', 'errado', 'discordo', 'piorou
0.0, 0.1]);
30 %
31 %         // === GÍRIAS E COMPREENSÃO DE PROBLEMAS ===
32 %         this.registerConcept(['erro', 'error', 'bug', 'falha', 'problema'
0.2, 0.4]);
33 %         this.registerConcept(['pau', 'quebrou', 'ferrou', 'travou', 'cong
0.2]);
34 %         this.registerConcept(['funciona', 'rodou', 'executou', 'sucesso'
0.3]);
35 %         this.registerConcept(['ajuda', 'socorro', 'help', 'salva', 'expli
0.0, 0.2]);
36 %
37 %         // === CONCEITOS ROBLOX E LUA ===
38 %         this.registerConcept(['roblox', 'jogo', 'game', 'mapa', 'place', '
39 %         this.registerConcept(['lua', 'luau', 'script', 'local', 'function
0.0, 0.9, 0.3]);
40 %         this.registerConcept(['workspace', 'players', 'humanoid', 'part'
0.0, 0.1, 0.2, 0.3, 0.0, 1.0, 0.1]);
41 %         this.registerConcept(['nil', 'nulo', 'inexistente', 'attempt', 'i
0.0, 0.6, 0.9, 0.2]);
42 %
43 %         // === CONCEITOS WEB, PROGRAMAÇÃO E MATEMÁTICA ===
44 %         this.registerConcept(['javascript', 'js', 'typescript', 'ts', 're
0.5]);
45 %         this.registerConcept(['const', 'let', 'var', 'promise', 'async', '
0.6]);
46 %         this.registerConcept(['código', 'codigo', 'programação', 'progra
0.0, 0.4, 0.7]);
47 %         this.registerConcept(['matemática', 'matematica', 'cálculo', 'ca
porcentagem'], [0.8, 0.1, 0.0, 0.3, 0.5, 0.0, 0.1, 1.0]);
```

```

48 %     }
49 %
50 %     private registerConcept(words: string[], vector: number[]) {
51 %         for (const w of words) {
52 %             this.dictionary.set(w.toLowerCase(), vector);
53 %         }
54 %     }
55 %
56 %     /**
57 %      * Obtém ou aproxima o vetor léxico de uma palavra com normaliz
58 %      */
59 %     public getVector(word: string): number[] {
60 %         const clean = word.toLowerCase().normalize('NFD').replace(/[\
61 %         if (this.cache.has(clean)) return this.cache.get(clean)!;
62 %         if (this.dictionary.has(clean)) {
63 %             const v = this.dictionary.get(clean)!;
64 %             this.cache.set(clean, v);
65 %             return v;
66 %         }
67 %
68 %         // Busca por radical ou prefixo semântico
69 %         for (const [key, vec] of this.dictionary.entries()) {
70 %             if (clean.length > 3 && (clean.startsWith(key.substring(0,
71 %                 this.cache.set(clean, vec);
72 %             return vec;
73 %         }
74 %     }
75 %
76 %     const neutral = [0.2, 0.5, 0.1, 0.2, 0.5, 0.1, 0.1, 0.2];
77 %     this.cache.set(clean, neutral);
78 %     return neutral;
79 % }
80 %
81 % /**
82 %  * Calcula a similaridade de cosseno entre duas palavras ou fra
83 %  */
84 % public cosineSimilarity(textA: string, textB: string): number {
85 %     const vecA = this.getPhraseVector(textA);
86 %     const vecB = this.getPhraseVector(textB);
87 %
88 %     let dotProduct = 0;
89 %     let normA = 0;
90 %     let normB = 0;
91 %
92 %     for (let i = 0; i < vecA.length; i++) {
93 %         dotProduct += vecA[i] * vecB[i];
94 %         normA += Math.pow(vecA[i], 2);
95 %         normB += Math.pow(vecB[i], 2);
96 %     }
97 %
98 %     if (normA === 0 || normB === 0) return 0;
99 %     return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
100 % }
101 %
102 % /**
103 %  * Obtém o vetor médio resultante de uma frase completa
104 %  */
105 % public getPhraseVector(phrase: string): number[] {
106 %     const words = phrase.toLowerCase().match(/[wÀ-ú]+/g) || [];
107 %     if (words.length === 0) return [0, 0, 0, 0, 0, 0, 0, 0];
108 %
109 %     const totalVector = [0, 0, 0, 0, 0, 0, 0, 0];
110 %     let count = 0;
111 %
112 %     for (const w of words) {
113 %         if (w.length > 2) {

```

```
114 %           const v = this.getVector(w);
115 %           for (let i = 0; i < v.length; i++) totalVector[i] += v[i]
116 %           count++;
117 %       }
118 %   }
119 %
120 %   if (count === 0) return [0.2, 0.5, 0.1, 0.2, 0.5, 0.1, 0.1, 0
121 %   return totalVector.map(val => val / count);
122 %   }
123 %   }
124 %
```

```

1 % // =====
2 % // FEEDBACK ENGINE & RANKING GLOBAL (MoE + Fila de Aprovação + Ba
3 % // Gerencia Likes (+1), Dislikes (-1), Super Likes (+10), Correto
4 % // e realiza espelhamento duplo via PersistenceGuard.
5 % // =====
6 %
7 % import { db, LearningQueueItem, HumanCorrection, User } from '../
8 % import { StatisticalLanguageModel } from '../language_model';
9 % import { Category, IntentionType } from '../classifier/intent_cla
10 % import { ContextVector } from '../context_engine';
11 % import { PersistenceGuard } from '../database/persistence';
12 %
13 % export class FeedbackEngine {
14 %     private SUPER_LIKE_PASSWORD = 'pão';
15 %     private GLOBAL_KNOWLEDGE_THRESHOLD = 10; // 10 likes para virar
16 %     private persistenceGuard = new PersistenceGuard();
17 %
18 %     constructor(private model: StatisticalLanguageModel) {}
19 %
20 %     /**
21 %      * Adiciona um item gerado à fila de aprendizado no estado NEUT
22 %      */
23 %     async registerNeutral(
24 %         responseId: string,
25 %         userId: string,
26 %         userName: string,
27 %         question: string,
28 %         answer: string,
29 %         category: Category,
30 %         intention: IntentionType,
31 %         contextVector: ContextVector
32 %     ): Promise<void> {
33 %         const existing = await db.learning_queue.where('responseId').
34 %         if (!existing) {
35 %             await db.learning_queue.add({
36 %                 responseId,
37 %                 userId,
38 %                 userName,
39 %                 question,
40 %                 answer,
41 %                 category,
42 %                 intention,
43 %                 contextVector: JSON.stringify(contextVector),
44 %                 status: 'neutral',
45 %                 likes: 0,
46 %                 dislikes: 0,
47 %                 superLikes: 0,
48 %                 superLike: false,
49 %                 timestamp: new Date(),
50 %             });
51 %             await this.persistenceGuard.createMirrorBackup();
52 %         }
53 %     }
54 %
55 %     /**
56 %      * Atualiza pontuação do usuário (Sistema de Ranking: Gabriel L
57 %      */
58 %     private async updateUserLikesRanking(userId: string, addedLikes
59 %         const user = await db.users.where('uuid').equals(userId).firs
60 %         if (user && user.id) {
61 %             const currentLikes = user.likesCount || 0;
62 %             await db.users.update(user.id, { likesCount: currentLikes +
63 %         }

```

```

64 %     }
65 %
66 %     / **
67 %     * Promove resposta para Conhecimento Global caso alcance o thr
68 %     * /
69 %     private async checkAndPromoteToGlobalKnowledge(item: LearningQue
70 %         if (totalPoints >= this.GLOBAL_KNOWLEDGE_THRESHOLD && item.st
71 %             await db.learning_queue.update(item.id!, { status: 'approve
72 %
73 %             const existingKnowledge = await db.knowledge.where('title')
74 %             const duplicate = existingKnowledge.some(k => k.content ===
75 %
76 %             if (!duplicate) {
77 %                 await db.knowledge.add({
78 %                     title: `[Aprovado Global] ${item.category} (Score: ${to
79 %                     content: item.correctedAnswer || item.answer,
80 %                     category: item.category,
81 %                     createdAt: new Date(),
82 %                 });
83 %                 await this.persistenceGuard.createMirrorBackup();
84 %             }
85 %         }
86 %     }
87 %
88 %     / **
89 %     * Processa Like (+1) ou Super Like (+10) (Atualiza Fila, Ranki
90 %     * /
91 %     async registerLike(
92 %         responseId: string,
93 %         userId: string,
94 %         userName: string,
95 %         question: string,
96 %         answer: string,
97 %         category: Category,
98 %         intention: IntentionType,
99 %         contextVector: ContextVector,
100 %         isSuperLike: boolean,
101 %         passwordEntered?: string
102 %     ): Promise<{ success: boolean; message: string; scoreInfo: stri
103 %         if (isSuperLike && passwordEntered !== this.SUPER_LIKE_PASSWO
104 %             return { success: false, message: 'Senha incorreta para o S
105 %         }
106 %
107 %         const existing = await db.learning_queue.where('responseId').
108 %         const likeInc = isSuperLike ? 0 : 1;
109 %         const superLikeInc = isSuperLike ? 1 : 0;
110 %         const addedPoints = isSuperLike ? 10 : 1;
111 %
112 %         let targetItem: LearningQueueItem;
113 %
114 %         if (existing) {
115 %             const newLikes = existing.likes + likeInc;
116 %             const newSuperLikes = existing.superLikes + superLikeInc;
117 %             const totalPoints = newLikes + (newSuperLikes * 10) - existi
118 %
119 %             await db.learning_queue.update(existing.id!, {
120 %                 likes: newLikes,
121 %                 superLikes: newSuperLikes,
122 %                 superLike: isSuperLike || existing.superLike,
123 %                 status: totalPoints >= this.GLOBAL_KNOWLEDGE_THRESHOLD ?
124 %                 timestamp: new Date()
125 %             });
126 %             targetItem = { ...existing, likes: newLikes, superLikes: ne
127 %         } else {
128 %             const newItem: LearningQueueItem = {
129 %                 responseId, userId, userName, question, answer, category,

```

```

130 %         contextVector: JSON.stringify(contextVector),
131 %         status: addedPoints >= this.GLOBAL_KNOWLEDGE_THRESHOLD ?
132 %         likes: likeInc, dislikes: 0, superLikes: superLikeInc,
133 %         superLike: isSuperLike, timestamp: new Date(),
134 %     };
135 %     const id = await db.learning_queue.add(newItem);
136 %     targetItem = { ...newItem, id };
137 % }
138 %
139 %     await this.updateUserLikesRanking(userId, addedPoints);
140 %
141 %     const currentScore = targetItem.likes + (targetItem.superLike
142 %     await this.checkAndPromoteToGlobalKnowledge(targetItem, curre
143 %
144 %     const weightMultiplier = isSuperLike ? 50 : 15;
145 %     this.model.train(targetItem.correctedAnswer || targetItem.ans
146 %
147 %     // Garante backup super permanente após a transação
148 %     await this.persistenceGuard.createMirrorBackup();
149 %
150 %     const totalCalculatedScore = this.calculateItemScore(targetIt
151 %
152 %     return {
153 %         success: true,
154 %         message: isSuperLike ? '+P Super Like (+10) salvo com sucess
Fila de Aprovação.',
155 %         scoreInfo: `Score Atual: ${totalCalculatedScore.percentage}
Dislikes: ${targetItem.dislikes}`,
156 %     };
157 % }
158 %
159 % /**
160 %  * Processa Dislike (-1) (Muda status para REJECTED, não treina
161 %  */
162 % async registerDislike(
163 %     responseId: string,
164 %     userId: string,
165 %     userName: string,
166 %     question: string,
167 %     answer: string,
168 %     category: Category,
169 %     intention: IntentionType,
170 %     contextVector: ContextVector
171 % ): Promise<{ success: boolean; message: string; scoreInfo: stri
172 %     const existing = await db.learning_queue.where('responseId').
173 %     let targetItem: LearningQueueItem;
174 %
175 %     if (existing) {
176 %         const newDislikes = existing.dislikes + 1;
177 %         await db.learning_queue.update(existing.id!, {
178 %             dislikes: newDislikes,
179 %             status: 'rejected',
180 %             timestamp: new Date()
181 %         });
182 %         targetItem = { ...existing, dislikes: newDislikes, status:
183 %     } else {
184 %         const newItem: LearningQueueItem = {
185 %             responseId, userId, userName, question, answer, category,
186 %             contextVector: JSON.stringify(contextVector), status: 're
187 %             likes: 0, dislikes: 1, superLikes: 0, superLike: false, t
188 %         };
189 %         const id = await db.learning_queue.add(newItem);
190 %         targetItem = { ...newItem, id };
191 %     }
192 %
193 %     await this.updateUserLikesRanking(userId, -1);

```

```

194 %         this.model.penalizeSequence(answer);
195 %
196 %         await this.persistenceGuard.createMirrorBackup();
197 %
198 %         const totalCalculatedScore = this.calculateItemScore(targetItem);
199 %
200 %         return {
201 %             success: true,
202 %             message: 'ðŸŒ™ Dislike (-1) registrado. Probabilidade da dire
203 %             scoreInfo: `Score Atual: ${totalCalculatedScore.percentage}
Dislikes: ${targetItem.dislikes}``,
204 %         };
205 %     }
206 %
207 %     /**
208 %      * Corretor Humano - Salva correção de resposta ruim no painel
209 %      */
210 %     async saveHumanCorrection(
211 %         queueId: number,
212 %         entrada: string,
213 %         respostaRuim: string,
214 %         respostaCorrigida: string,
215 %         category: Category
216 %     ): Promise<void> {
217 %         await db.human_corrections.add({
218 %             entrada,
219 %             resposta_ruim: respostaRuim,
220 %             resposta_corrigida: respostaCorrigida,
221 %             score: 1,
222 %             timestamp: new Date()
223 %         });
224 %
225 %         await db.learning_queue.update(queueId, {
226 %             correctedAnswer: respostaCorrigida,
227 %             status: 'approved',
228 %             likes: 10,
229 %             timestamp: new Date()
230 %         });
231 %
232 %         this.model.train(respostaCorrigida, category, 100);
233 %
234 %         // Backup permanente imediato da correção do desenvolvedor
235 %         await this.persistenceGuard.createMirrorBackup();
236 %     }
237 %
238 %     public calculateItemScore(item: LearningQueueItem): { percentage
239 %         const totalPositive = item.likes + (item.superLikes * 10);
240 %         const totalVotes = totalPositive + item.dislikes;
241 %
242 %         if (totalVotes === 0) return { percentage: 85, totalPositive:
243 %
244 %         const pct = (totalPositive / totalVotes) * 100;
245 %         return {
246 %             percentage: Math.round(pct),
247 %             totalPositive,
248 %             dislikes: item.dislikes,
249 %         };
250 %     }
251 %
252 %     async syncCloudWeights(): Promise<void> {
253 %         const allQueue = await db.learning_queue.toArray();
254 %         const humanCorrections = await db.human_corrections.toArray()
255 %
256 %         for (const corr of humanCorrections) {
257 %             this.model.train(corr.resposta_corrigida, 'PROGRAMAÇÃO', 10
258 %         }

```

```

259 %
260 %     const approved = allQueue.filter((item: LearningQueueItem) =>
261 %     for (const item of approved) {
262 %         const weight = item.superLike ? 50 : Math.max(15, item.like
263 %         this.model.train(item.correctedAnswer || item.answer, item.
264 %     }
265 %
266 %     const rejected = allQueue.filter((item: LearningQueueItem) =>
267 %     for (const item of rejected) {
268 %         this.model.penalizeSequence(item.answer);
269 %     }
270 %
271 %     await this.persistenceGuard.createMirrorBackup();
272 % }
273 %
274 % async getDeveloperAnalytics(): Promise<{
275 %     totalApproved: number;
276 %     totalNeutral: number;
277 %     totalRejected: number;
278 %     approvedList: LearningQueueItem[];
279 %     neutralList: LearningQueueItem[];
280 %     rejectedList: LearningQueueItem[];
281 %     userRanking: User[];
282 %     humanCorrectionsList: HumanCorrection[];
283 % }> {
284 %     const allQueue = await db.learning_queue.reverse().toArray();
285 %     const humanCorrectionsList = await db.human_corrections.rever
286 %
287 %     const userRanking = (await db.users.toArray()).sort((a, b) =>
288 %
289 %     const approvedList = allQueue.filter((item: LearningQueueItem
290 %     const neutralList = allQueue.filter((item: LearningQueueItem)
291 %     const rejectedList = allQueue.filter((item: LearningQueueItem
292 %
293 %     return {
294 %         totalApproved: approvedList.length,
295 %         totalNeutral: neutralList.length,
296 %         totalRejected: rejectedList.length,
297 %         approvedList,
298 %         neutralList,
299 %         rejectedList,
300 %         userRanking,
301 %         humanCorrectionsList,
302 %     };
303 % }
304 % }
305 %

```


Ø=ÜÄ src/generation/grammar_finalizer.ts

```
1 % // =====
2 % // REESCRITOR AUTOMÁTICO E FINALIZADOR DE FRASES (Grammar Finaliz
3 % // Realiza a verificação explícita de Frase Completa (SIM/NÃO) e
4 % // NENHUMA resposta pronta é imposta; ele executa a polidez sintá
5 % // =====
6 %
7 % import { Category, IntentionType, ExpertType } from '../classifie
8 %
9 % export interface SentenceCheckResult {
10 %     text: string;
11 %     isSentenceComplete: boolean;
12 %     repaired: boolean;
13 %     actionExplanation?: string;
14 % }
15 %
16 % export class GrammarFinalizer {
17 %     constructor() {}
18 %
19 %     /**
20 %      * Reescritor Automático: Transforma respostas cruas ou secas e
21 %      */
22 %     public rewriteIfDry(text: string, intention: IntentionType, exp
string) {
23 %         const clean = text.trim();
24 %         const lower = clean.toLowerCase();
25 %
26 %         // Reescrita inteligente para saudações curtas ou cruas (ex:
27 %         if (intention === 'saudacao' && clean.split(/ /).length <= 4)
28 %             if (/^(oi tudo bem|oi tudo bem\?|ola tudo bem|olá tudo bem|
29 %                 return {
30 %                     text: 'Oi! Tudo bem com você hoje? Como posso te ajudar
31 %                     rewrote: true,
32 %                     notice: 'Reescritor Automático: Expansão de cordialidad
33 %                 };
34 %             }
35 %             if (/^(oi|olá|ola|eai|eae|bom dia|boa tarde|boa noite)$/.te
36 %                 const punctuation = clean.endsWith('!') || clean.endsWith
37 %                 return {
38 %                     text: `${clean}${punctuation} Tudo ótimo por aqui. Em q
39 %                     rewrote: true,
40 %                     notice: 'Reescritor Automático: Injeção de abertura con
41 %                 };
42 %             }
43 %         }
44 %
45 %         // Se for uma resposta técnica extremamente seca sob responsa
46 %         if (intention === 'pergunta' && clean.split(/ /).length <= 5
47 %             return {
48 %                 text: `Com base nas diretrizes do ${expert}, a verificação
completo?`,
49 %                 rewrote: true,
50 %                 notice: `Reescritor Automático: Formulação expandida pelo
51 %             };
52 %         }
53 %
54 %         return { text: clean, rewrote: false };
55 %     }
56 %
57 %     /**
58 %      * ETAPA 9 - VERIFICAÇÃO DE FRASE FINAL (Frase Completa? Sim/Nã
59 %      * O grande salto da Atualização Definitiva. Verifica se a fras
60 %      * Nunca envia frases incompletas.
61 %      */
```

```

62 %     public checkAndRepairSentence(text: string, intention: Intention) {
63 %         let current = text.trim();
64 %         if (!current) return { text: current, isSentenceComplete: false };
65 %
66 %         const lower = current.toLowerCase();
67 %
68 %         // 1. Detecção de fragmentos desgarrados conhecidos (O problema é que as coisas não estão bem com você)
69 %         if (/^(estão as coisas|estao as coisas|bem com você|bem com v
estão)$/.test(lower)) {
70 %             if (lower.includes('coisas') || lower.includes('como estão as coisas')) {
71 %                 return {
72 %                     text: 'Como estão as coisas no desenvolvimento do seu projeto?',
73 %                     isSentenceComplete: true,
74 %                     repaired: true,
75 %                     actionExplanation: 'ETAPA 9 [Finalizador]: Fechamento de frases com coisas?'.',
76 %                 };
77 %             }
78 %             if (lower.includes('bem')) {
79 %                 return {
80 %                     text: 'Tudo bem com você? Estou pronto para analisar seu projeto.',
81 %                     isSentenceComplete: true,
82 %                     repaired: true,
83 %                     actionExplanation: 'ETAPA 9 [Finalizador]: Restauro est
coisas?'.',
84 %                 };
85 %             }
86 %             if (lower.includes('novo dia')) {
87 %                 return {
88 %                     text: 'Amanhã será um novo dia repleto de novas conquistas.',
89 %                     isSentenceComplete: true,
90 %                     repaired: true,
91 %                     actionExplanation: 'ETAPA 9 [Finalizador]: Estruturação de frases com novo dia'.',
92 %                 };
93 %             }
94 %         }
95 %
96 %         // 2. Verificação Genérica de Frase Incompleta (terminou em preposição)
97 %         const words = current.split(/ /);
98 %         const lastWord = words[words.length - 1].toLowerCase().replace(/s$/, '');
99 %         const trailingPrepositions = new Set(['de', 'da', 'do', 'em', 'no', 'na', 'para', 'com', 'por', 'que', 'se', 'os', 'as', 'um', 'uma', 'e', 'o', 'a', 'ou']);
100 %
101 %         if (trailingPrepositions.has(lastWord)) {
102 %             let completion = '...';
103 %             if (['PROGRAMAÇÃO', 'ROBLOX', 'LUAU', 'JAVASCRIPT', 'ERROS DE CODIFICAÇÃO'].includes(lastWord)) {
104 %                 completion = 'a estrutura principal do código e suas dependências.';
105 %             } else if (category === 'MATEMÁTICA') {
106 %                 completion = 'o cálculo algébrico correspondente na fórmula.';
107 %             } else {
108 %                 completion = 'o andamento natural da nossa conversa.';
109 %             }
110 %
111 %             const fixedText = current.replace(/([.!?,:;]+)$/, ' ' + completion);
112 %             return {
113 %                 text: fixedText,
114 %                 isSentenceComplete: true,
115 %                 repaired: true,
116 %                 actionExplanation: `ETAPA 9 [Finalizador]: Detecção de frase incompleta e
adicionado.`',
117 %             };
118 %         }
119 %
120 %         // 3. Garantia de Pontuação Final e Verificação de Frase Completa
121 %         const hasPunctuation = /[.!?]$/.test(current);
122 %         if (!hasPunctuation) {
123 %             const punc = intention === 'pergunta' ? '?' : intention === 'afirmação' ? '.' : '!';

```

```
1 2 4 %           r e t u r n  {
1 2 5 %           t e x t :   c u r r e n t   +   p u n c ,
1 2 6 %           i s S e n t e n c e C o m p l e t e :   t r u e ,
1 2 7 %           r e p a i r e d :   t r u e ,
1 2 8 %           a c t i o n E x p l a n a t i o n :   ` E T A P A   9   [ F i n a l i z a d o r ] :   C o n c l u s ã o   c o m
COMPLETA.` ,
1 2 9 %           } ;
1 3 0 %           }
1 3 1 %
1 3 2 %           r e t u r n  {   t e x t :   c u r r e n t ,   i s S e n t e n c e C o m p l e t e :   t r u e ,   r e p a i r e d :   f
classificada como COMPLETA (SIM).' };
1 3 3 %           }
1 3 4 %           }
1 3 5 %
```

```

1 % // =====
2 % // MODELO ADAPTIVE N-GRAM + SKIP-GRAM + CACHE LM
3 % // Integra N-Gram Backoff, Skip-Gram (distância 2), Cache LM (rec
4 % // Semantic Attention, Lookahead Beam Search e Classes de Fala (P
5 % // NENHUMA resposta pronta é usada. Aprende APENAS com feedback a
6 % // =====
7 %
8 % import type { Category } from '../classifier/intent_classifier';
9 % import { EmbeddingsEngine } from '../embeddings_engine';
10 % import { POSEngine, PartOfSpeech } from '../pos_engine';
11 %
12 % interface PredictionResult {
13 %     token: string;
14 %     prob: number;
15 %     lookaheadScore?: number;
16 %     attentionWeight?: number;
17 %     postTag?: PartOfSpeech;
18 % }
19 %
20 % export class StatisticalLanguageModel {
21 %     // %% N-Gram padrão %%%%%%%%%%%%%%%
22 %     private unigrams = new Map<string, number>();
23 %     private bigrams = new Map<string, Map<string, number>>();
24 %     private trigrams = new Map<string, Map<string, number>>();
25 %     private quadrigrams = new Map<string, Map<string, number>>();
26 %
27 %     // %% N-Gram por Categoria (MoE) %%%%%%%%%%%%%%%
28 %     private categoryBigrams = new Map<Category, Map<string, Map<str
29 %     private categoryTrigrams = new Map<Category, Map<string, Map<st
30 %
31 %     // %% Skip-Gram (distância 1 e 2) - captura relações não-adjace
32 %     // Ex: "local [result] = ..." !' liga "local" a "result" mesmo c
33 %     private skipBigrams = new Map<string, Map<string, number>>();
34 %
35 %     // %% Cache LM - dá bônus a tokens usados recentemente na conve
36 %     // Inspirado em: "Cache Language Models for Speech Recognition"
37 %     private cacheTokens: string[] = []; // últimos 60 tokens do c
38 %     private CACHE_MAX = 60;
39 %     private CACHE_BONUS = 0.35; // bônus de probabilidade
40 %
41 %     // %% Penalidades de Dislike %%%%%%%%%%%%%%%
42 %     private penalizedBigrams = new Map<string, Map<string, number>>
43 %     private penalizedTrigrams = new Map<string, Map<string, number>
44 %
45 %     // %% Metadados globais %%%%%%%%%%%%%%%
46 %     private totalWords = 0;
47 %     private trainedCategories = new Set<Category>();
48 %     private embeddings = new EmbeddingsEngine();
49 %     private posEngine = new POSEngine();
50 %
51 %     constructor() {}
52 %
53 %     /**
54 %      * Tokenização fluente - preserva pontuação e conectores gramat
55 %      */
56 %     private tokenize(text: string): string[] {
57 %         return text.toLowerCase()
58 %             .replace(/\n/g, ' \n ')
59 %             .match(/[wÄ-ú]+|[.,!?:]+/g) || [];
60 %     }
61 %
62 %     /**
63 %      * Treina o modelo com texto bruto e multiplicadores de feedback

```

```

64 %      * /
65 %      public train(text: string, category?: Category, multiplier: num
66 %          const tokens = this.tokenize(text);
67 %          if (tokens.length === 0) return;
68 %
69 %          if (category) {
70 %              this.trainedCategories.add(category);
71 %              if (!this.categoryBigrams.has(category)) {
72 %                  this.categoryBigrams.set(category, new Map());
73 %                  this.categoryTrigrams.set(category, new Map());
74 %              }
75 %          }
76 %
77 %          for (let i = 0; i < tokens.length; i++) {
78 %              const w1 = tokens[i];
79 %              const w2 = i + 1 < tokens.length ? tokens[i + 1] : null;
80 %              const w3 = i + 2 < tokens.length ? tokens[i + 2] : null;
81 %              const w4 = i + 3 < tokens.length ? tokens[i + 3] : null;
82 %              // %% Skip-Gram: palavra com distância 2 (pula um token no
83 %              const wSkip2 = i + 2 < tokens.length ? tokens[i + 2] : null;
84 %
85 %              this.unigrams.set(w1, (this.unigrams.get(w1) || 0) + multipl
86 %              this.totalWords += multiplier;
87 %
88 %              // %% Skip-Gram registro (peso menor que o bigram direto) %
89 %              if (wSkip2) {
90 %                  const skipMult = multiplier * 0.5;
91 %                  if (!this.skipBigrams.has(w1)) this.skipBigrams.set(w1, n
92 %                  const sMap = this.skipBigrams.get(w1)!;
93 %                  sMap.set(wSkip2, (sMap.get(wSkip2) || 0) + skipMult);
94 %              }
95 %
96 %              if (w2) {
97 %                  if (!this.bigrams.has(w1)) this.bigrams.set(w1, new Map()
98 %                  const bMap = this.bigrams.get(w1)!;
99 %                  bMap.set(w2, (bMap.get(w2) || 0) + multiplier);
100 %
101 %                  if (category) {
102 %                      const catBigrams = this.categoryBigrams.get(category)!;
103 %                      if (!catBigrams.has(w1)) catBigrams.set(w1, new Map());
104 %                      const catBMap = catBigrams.get(w1)!;
105 %                      catBMap.set(w2, (catBMap.get(w2) || 0) + multiplier);
106 %                  }
107 %
108 %                  if (w3) {
109 %                      const ctx2 = `${w1} ${w2}`;
110 %                      if (!this.trigrams.has(ctx2)) this.trigrams.set(ctx2, n
111 %                      const tMap = this.trigrams.get(ctx2)!;
112 %                      tMap.set(w3, (tMap.get(w3) || 0) + multiplier);
113 %
114 %                      if (category) {
115 %                          const catTrigrams = this.categoryTrigrams.get(categor
116 %                          if (!catTrigrams.has(ctx2)) catTrigrams.set(ctx2, new
117 %                          const catTMap = catTrigrams.get(ctx2)!;
118 %                          catTMap.set(w3, (catTMap.get(w3) || 0) + multiplier);
119 %                      }
120 %
121 %                      if (w4) {
122 %                          const ctx3 = `${w1} ${w2} ${w3}`;
123 %                          if (!this.quadrigrams.has(ctx3)) this.quadrigrams.set
124 %                          const qMap = this.quadrigrams.get(ctx3)!;
125 %                          qMap.set(w4, (qMap.get(w4) || 0) + multiplier);
126 %                      }
127 %                  }
128 %              }
129 %          }

```

```

130 %     }
131 %
132 %     / **
133 %     * Adiciona penalidades de dislikes (Diminui o peso de transição)
134 %     */
135 %     public penalizeSequence(text: string): void {
136 %         const tokens = this.tokenize(text);
137 %         if (tokens.length === 0) return;
138 %
139 %         for (let i = 0; i < tokens.length; i++) {
140 %             const w1 = tokens[i];
141 %             const w2 = i + 1 < tokens.length ? tokens[i + 1] : null;
142 %             const w3 = i + 2 < tokens.length ? tokens[i + 2] : null;
143 %
144 %             if (w2) {
145 %                 if (!this.penalizedBigrams.has(w1)) {
146 %                     this.penalizedBigrams.set(w1, new Map());
147 %                 }
148 %                 const currentPenalty = this.penalizedBigrams.get(w1)!.get(
149 %                     this.penalizedBigrams.get(w1)!.set(w2, currentPenalty * 0
150 %
151 %                 if (w3) {
152 %                     const ctx2 = `${w1} ${w2}`;
153 %                     if (!this.penalizedTrigrams.has(ctx2)) {
154 %                         this.penalizedTrigrams.set(ctx2, new Map());
155 %                     }
156 %                     const currentTriPenalty = this.penalizedTrigrams.get(
157 %                         this.penalizedTrigrams.get(ctx2)!.set(w3, currentTriPen
158 %                 }
159 %             }
160 %         }
161 %     }
162 %
163 %     / **
164 %     * Zera e reaplica todos os pesos de feedback carregados do banco
165 %     */
166 %     public applyFeedbackWeights(approved: any[], rejected: any[]): void {
167 %         for (const app of approved) {
168 %             const weight = app.superLike ? 50 : Math.max(10, app.likes);
169 %             this.train(app.resposta || app.answer || app.correctedAnswer,
170 %         }
171 %
172 %         for (const rej of rejected) {
173 %             this.penalizeSequence(rej.resposta || rej.answer);
174 %         }
175 %     }
176 %
177 %     / **
178 %     * Prevê próxima palavra combinando Backoff N-Gram, Atenção Sem
179 %     */
180 %     public predictNextWord(
181 %         contextTokens: string[],
182 %         category?: Category,
183 %         targetContextText: string = '',
184 %         generatedCount: number = 0
185 %     ): PredictionResult[] {
186 %         const len = contextTokens.length;
187 %         let candidates = new Map<string, number>();
188 %         let weight = 1.0;
189 %
190 %         const isTechnicalCategory = ['PROGRAMAÇÃO', 'ROBLOX', 'LUAU',
'IA'].includes(category || 'CONVERSA');
191 %
192 %         // PRIORIDADE 1: Categoria específica
193 %         if (category && this.trainedCategories.has(category)) {
194 %             if (len >= 2) {

```

```

195 %         const ctx = `${contextTokens[len-2]} ${contextTokens[len-1]}`;
196 %         const catTri = this.categoryTrigrams.get(category)?.get(ctx);
197 %         if (catTri && catTri.size > 0) {
198 %             candidates = catTri;
199 %             weight = 1.3;
200 %         }
201 %     }
202 %     if (candidates.size === 0 && len >= 1) {
203 %         const catBi = this.categoryBigrams.get(category)?.get(ctx);
204 %         if (catBi && catBi.size > 0) {
205 %             candidates = catBi;
206 %             weight = 1.0;
207 %         }
208 %     }
209 % }
210 %
211 % // PRIORIDADE 2: N-Grams Globais com Backoff
212 % if (candidates.size === 0 && len >= 3) {
213 %     const ctx = `${contextTokens[len-3]} ${contextTokens[len-2]} ${contextTokens[len-1]}`;
214 %     if (this.quadrigrams.has(ctx)) {
215 %         candidates = this.quadrigrams.get(ctx);
216 %         weight = 0.85;
217 %     }
218 % }
219 % if (candidates.size === 0 && len >= 2) {
220 %     const ctx = `${contextTokens[len-2]} ${contextTokens[len-1]}`;
221 %     if (this.trigrams.has(ctx)) {
222 %         candidates = this.trigrams.get(ctx);
223 %         weight = 0.7;
224 %     }
225 % }
226 % if (candidates.size === 0 && len >= 1) {
227 %     const ctx = contextTokens[len-1];
228 %     if (this.bigrams.has(ctx)) {
229 %         candidates = this.bigrams.get(ctx);
230 %         weight = 0.5;
231 %     }
232 % }
233 %
234 % // PRIORIDADE 3: Skip-Gram (fallback para relações não-adjacentes)
235 % if (candidates.size === 0 && len >= 1) {
236 %     const skipCands = this.skipBigrams.get(contextTokens[len-1]);
237 %     if (skipCands && skipCands.size > 0) {
238 %         candidates = skipCands;
239 %         weight = 0.35;
240 %     }
241 % }
242 %
243 % // %% Cache LM: conjunto de tokens usados recentemente no contexto
244 % // Se um token apareceu nos últimos CACHE_MAX tokens, recebe peso 1.0
245 % const cacheSet = new Set(this.cacheTokens);
246 %
247 % const results: PredictionResult[] = [];
248 % if (candidates.size > 0) {
249 %     const total = Array.from(candidates.values()).reduce((a, b) => a + b, 0);
250 %     for (const [word, count] of candidates.entries()) {
251 %         let prob = (count / total) * weight;
252 %         // Cache LM bônus: palavra usada recentemente é mais provável
253 %         if (cacheSet.has(word)) prob += this.CACHE_BONUS * (prob - 0.5);
254 %
255 %         // Penalidade de Dislike
256 %         if (len >= 2) {
257 %             const ctx2 = `${contextTokens[len-2]} ${contextTokens[len-1]} ${word}`;
258 %             const penalty = this.penalizedTrigrams.get(ctx2)?.get(word);
259 %             if (penalty !== undefined) prob *= penalty;
260 %         }

```

```

261 %         if (len >= 1) {
262 %             const ctx1 = contextTokens[len - 1];
263 %             const penalty = this.penalizedBigrams.get(ctx1)?.get(word);
264 %             if (penalty !== undefined) prob * penalty;
265 %         }
266 %
267 %         // SEMANTIC ATTENTION BIAS (Vetor de Foco)
268 %         let attentionWeight = 1.0;
269 %         if (targetContextText) {
270 %             const sim = this.embeddings.cosineSimilarity(word, targetContextText);
271 %             attentionWeight = 0.7 + (sim * 1.5);
272 %         }
273 %
274 %         // POS MODIFIER (Classes de Fala) - Poda saudações excessivas
275 %         const posTag = this.posEngine.getPOS(word);
276 %         const posMod = this.posEngine.calculatePOSModifier(word, posTag);
277 %
278 %         // LOOKAHEAD BEAM SCORING
279 %         let lookaheadScore = 1.0;
280 %         if (this.bigrams.has(word)) {
281 %             const nextCandidates = Array.from(this.bigrams.get(word));
282 %             if (targetContextText && nextCandidates.length > 0) {
283 %                 const bestCosine = nextCandidates.reduce((best, nextWord) => {
284 %                     const sim = this.embeddings.cosineSimilarity(`${word} ${nextWord}`);
285 %                     return sim > best ? sim : best;
286 %                 }, 0);
287 %                 lookaheadScore = 0.6 + (bestCosine * 1.2);
288 %             }
289 %         }
290 %
291 %         results.push({ token: word, prob: prob * attentionWeight * lookaheadScore,
posTag });
292 %     }
293 % }
294 %
295 % // Fallback triplo robusto
296 % if (results.length === 0) {
297 %     const top = Array.from(this.unigrams.entries())
298 %         .sort((a, b) => b[1] - a[1])
299 %         .slice(0, 15);
300 %     for (const [word, count] of top) {
301 %         const posTag = this.posEngine.getPOS(word);
302 %         const posMod = this.posEngine.calculatePOSModifier(word, posTag);
303 %         results.push({ token: word, prob: (count / Math.max(1, this.unigrams.get(word) || 1)) *
attentionWeight: 1.0, posTag });
304 %     }
305 % }
306 %
307 % if (results.length === 0) {
308 %     results.push({ token: '.', prob: 0.5, posTag: 'Geral' }, {
0.2, posTag: 'Geral' });
309 % }
310 %
311 % return results.filter(r => r.prob > 0).sort((a, b) => b.prob - a.prob);
312 % }
313 %
314 % /**
315 %  * Geração em streaming inteligente com POS e Beam Search
316 %  */
317 % /**
318 %  * Limpa o Cache LM (chamar no início de cada nova conversa)
319 %  */
320 % public clearCache(): void {
321 %     this.cacheTokens = [];
322 % }
323 %

```



```

390 %         postTag: p.postTag,
391 %         weight: Math.exp(Math.log(Math.max(p.prob, 1e-10)) / te
392 %     )));
393 %     const sumWeight = scaled.reduce((a, b) => a + b.weight, 0
394 %
395 %     let random = Math.random() * sumWeight;
396 %     chosen = {
397 %         token: adjusted[0].token,
398 %         prob: adjusted[0].prob,
399 %         lookaheadScore: adjusted[0].lookaheadScore,
400 %         attentionWeight: adjusted[0].attentionWeight,
401 %         postTag: adjusted[0].postTag
402 %     };
403 %     for (const p of scaled) {
404 %         random -= p.weight;
405 %         if (random <= 0) {
406 %             chosen = { token: p.token, prob: p.prob, lookaheadScore
posTag: p.postTag };
407 %             break;
408 %         }
409 %     }
410 % }
411 %
412 %     if (chosen.token === '\n') {
413 %         consecutiveBreaks++;
414 %         if (consecutiveBreaks > 1) {
415 %             chosen = { token: '.', prob: chosen.prob, lookaheadScore
chosen.attentionWeight, postTag: 'Geral' };
416 %             consecutiveBreaks = 0;
417 %         }
418 %     } else {
419 %         consecutiveBreaks = 0;
420 %     }
421 %
422 %     tokens.push(chosen.token);
423 %     recentTokens.push(chosen.token);
424 %     if (recentTokens.length > 8) recentTokens.shift();
425 %
426 %     await new Promise(r => setTimeout(r, 20));
427 %
428 %     yield chosen;
429 %
430 %     if ((chosen.token === '.' || chosen.token === '?' || chosen
431 %         if (Math.random() > 0.65) break;
432 %     }
433 % }
434 % }
435 %
436 % public getStats() {
437 %     return {
438 %         vocabSize: this.unigrams.size,
439 %         totalWords: this.totalWords,
440 %         bigramContexts: this.bigrams.size,
441 %         trigramContexts: this.trigrams.size,
442 %         quadrigramContexts: this.quadrigrams.size,
443 %         trainedCategories: Array.from(this.trainedCategories),
444 %     };
445 % }
446 %
447 % public searchKnowledge(query: string, texts: string[]): string
448 %     if (texts.length === 0) return '';
449 %
450 %     let bestText = texts[0];
451 %     let bestSim = -1;
452 %
453 %     for (const text of texts) {

```

```
4 5 4 %           c o n s t   s i m   =   t h i s . e m b e d d i n g s . c o s i n e S i m i l a r i t y ( q u e r y ,   t e x t ) ;
4 5 5 %           i f   ( s i m   >   b e s t S i m )   {
4 5 6 %               b e s t S i m   =   s i m ;
4 5 7 %               b e s t T e x t   =   t e x t ;
4 5 8 %           }
4 5 9 %       }
4 6 0 %
4 6 1 %           r e t u r n   b e s t T e x t ;
4 6 2 %       }
4 6 3 %   }
4 6 4 %
```

```
1 % // =====
2 % // PHI-3 ENGINE - LLM local no navegador (sem API, sem servidor)
3 % // Usa @huggingface/transformers com WebGPU ou WASM como fallback
4 % // O modelo é baixado UMA VEZ e cached no IndexedDB/Cache API do
5 % // =====
6 %
7 % type Phi3Status =
8 %     | 'idle'
9 %     | 'checking'
10 %     | 'downloading'
11 %     | 'loading'
12 %     | 'ready'
13 %     | 'generating'
14 %     | 'error'
15 %     | 'unsupported';
16 %
17 % export interface Phi3State {
18 %     status: Phi3Status;
19 %     progress: number; // 0-100
20 %     progressLabel: string;
21 %     backend: 'webgpu' | 'wasm' | null;
22 %     modelId: string;
23 %     error?: string;
24 % }
25 %
26 % export interface Phi3CorrectionResult {
27 %     improved: string;
28 %     confidence: number;
29 %     model: string;
30 %     usedBackend: 'webgpu' | 'wasm';
31 %     tokensGenerated: number;
32 % }
33 %
34 % // Modelo principal: Phi-3.5 Mini quantizado para browser.
35 % // Fallback: SmolLM2-360M, bem menor, caso Phi-3 não rode no apar
36 % const PHI3_MODEL_ID = 'onnx-community/Phi-3.5-mini-instruct-onnx-
37 % const FALLBACK_MODEL_ID = 'HuggingFaceTB/SmolLM2-360M-Instruct';
38 %
39 % // Prompt de sistema para correção de respostas do N-Gram
40 % const SYSTEM_PROMPT = `Você é um assistente de correção de lingua
41 % Sua tarefa é pegar uma resposta gerada por um modelo N-Gram e mel
42 % REGRAS OBRIGATÓRIAS:
43 % - Nunca responda com respostas prontas ou decoradas.
44 % - Mantenha o assunto original da resposta.
45 % - Se a resposta estiver incompleta, complete-a.
46 % - Se tiver erros gramaticais, corrija-os.
47 % - Responda SEMPRE em português brasileiro.
48 % - Máximo 3 frases.`;
49 %
50 % let generatorInstance: any = null;
51 % let currentModelId: string | null = null;
52 % let isWebGPUAvailable = false;
53 %
54 % async function detectWebGPU(): Promise<boolean> {
55 %     try {
56 %         if (typeof navigator === 'undefined' || !(navigator as any).g
57 %             const adapter = await (navigator as any).gpu.requestAdapter()
58 %             return adapter !== null;
59 %         } catch {
60 %             return false;
61 %         }
62 %     }
63 % }
```

```

64 % export class Phi3Engine {
65 %     public state: Phi3State = {
66 %         status: 'idle',
67 %         progress: 0,
68 %         progressLabel: 'Phi-3 Mini: aguardando ativação',
69 %         backend: null,
70 %         modelId: PHI3_MODEL_ID,
71 %     };
72 %
73 %     private onStateChange: (state: Phi3State) => void;
74 %
75 %     constructor(onStateChange: (state: Phi3State) => void) {
76 %         this.onStateChange = onStateChange;
77 %     }
78 %
79 %     private setState(partial: Partial<Phi3State>) {
80 %         this.state = { ...this.state, ...partial };
81 %         this.onStateChange(this.state);
82 %     }
83 %
84 %     /**
85 %      * Verifica se o browser suporta WebGPU ou WASM e carrega o mod
86 %      */
87 %     public async load(): Promise<void> {
88 %         if (this.state.status === 'ready' || this.state.status === 'l
89 %             return;
90 %         }
91 %
92 %         this.setState({ status: 'checking', progressLabel: 'Detectand
93 %
94 %         isWebGPUAvailable = await detectWebGPU();
95 %         const backend: 'webgpu' | 'wasm' = isWebGPUAvailable ? 'webgp
96 %
97 %         this.setState({
98 %             status: 'downloading',
99 %             backend,
100 %             progressLabel: `Baixando Phi-3.5 Mini (${backend.toUpperCase
101 %         });
102 %
103 %         try {
104 %             // Importação via CDN dinâmica para manter bundle pequeno.
105 %             // O caminho /dist/transformers.min.mjs não existe em algum
106 %             // Por isso tentamos URLs oficiais/fallbacks em ordem.
107 %             const cdnUrls = [
108 %                 'https://cdn.jsdelivr.net/npm/@huggingface/transformers@3
109 %                 'https://cdn.jsdelivr.net/npm/@huggingface/transformers',
110 %                 'https://unpkg.com/@huggingface/transformers@3.5.0',
111 %             ];
112 %             let transformers: any = null;
113 %             let lastImportError: unknown = null;
114 %
115 %             for (const url of cdnUrls) {
116 %                 try {
117 %                     transformers = await (Function('url', 'return import(ur
118 %                     break;
119 %                 } catch (err) {
120 %                     lastImportError = err;
121 %                 }
122 %             }
123 %
124 %             if (!transformers) {
125 %                 throw new Error(`Falha ao carregar Transformers.js via CD
126 %             }
127 %             const { pipeline, env } = transformers;
128 %
129 %             // Configuração para rodar no browser

```

```

130 %     env.allowLocalModels = false;
131 %     env.useBrowserCache = true;
132 %     if (env.backends?.onnx?.wasm) {
133 %         (env.backends.onnx.wasm as any).numThreads = 1;
134 %     }
135 %
136 %     const makeProgress = (modelLabel: string) => (info: any) =>
137 %         if (info.status === 'progress') {
138 %             const pct = info.total > 0 ? Math.round((info.loaded /
139 %                 const mbLoaded = Math.round(info.loaded / 1024 / 1024);
140 %                 const mbTotal = Math.round(info.total / 1024 / 1024);
141 %                 this.setState({
142 %                     status: 'downloading',
143 %                     progress: pct,
144 %                     progressLabel: `Baixando ${modelLabel}: ${mbLoaded} MB
145 %                 });
146 %             } else if (info.status === 'initiate' || info.status ===
147 %                 this.setState({
148 %                     status: 'loading',
149 %                     progress: 95,
150 %                     progressLabel: `Inicializando ${modelLabel} no browse
151 %                 });
152 %             }
153 %         };
154 %
155 %     try {
156 %         generatorInstance = await pipeline(
157 %             'text-generation',
158 %             PHI3_MODEL_ID,
159 %             {
160 %                 dtype: 'q4' as any,
161 %                 device: backend === 'webgpu' ? 'webgpu' : 'wasm',
162 %                 progress_callback: makeProgress('Phi-3 Mini'),
163 %             }
164 %         );
165 %         currentModelId = PHI3_MODEL_ID;
166 %     } catch (phiErr) {
167 %         console.warn('Phi-3 falhou, tentando modelo local leve:',
168 %             this.setState({
169 %                 status: 'downloading',
170 %                 progress: 0,
171 %                 progressLabel: 'Phi-3 não carregou neste aparelho. Tent
172 %             });
173 %         generatorInstance = await pipeline(
174 %             'text-generation',
175 %             FALLBACK_MODEL_ID,
176 %             {
177 %                 dtype: 'q8' as any,
178 %                 device: backend === 'webgpu' ? 'webgpu' : 'wasm',
179 %                 progress_callback: makeProgress('SmolLM2 local'),
180 %             }
181 %         );
182 %         currentModelId = FALLBACK_MODEL_ID;
183 %     }
184 %
185 %     this.setState({
186 %         status: 'ready',
187 %         progress: 100,
188 %         progressLabel: `  ${currentModelId?.includes('Phi')} ? 'Ph
- modelo em cache no browser`,
189 %         backend,
190 %     });
191 % } catch (e: any) {
192 %     const msg = e?.message || String(e);
193 %     this.setState({
194 %         status: 'error',

```

```

195 %         error: msg,
196 %         progressLabel: `Erro ao carregar Phi-3 Mini: ${msg.substr
197 %     });
198 % }
199 % }
200 %
201 % / * *
202 % * Usa o Phi-3 Mini para melhorar a resposta do N-Gram
203 % * O N-Gram gerou algo, o Phi-3 revisa e completa.
204 % * /
205 % public async correctResponse(
206 %     userMessage: string,
207 %     rawNgramResponse: string,
208 %     category: string,
209 %     subject: string
210 % ): Promise<Phi3CorrectionResult> {
211 %     if (!generatorInstance || this.state.status !== 'ready') {
212 %         throw new Error('Phi-3 Mini ainda não está carregado. Aguarde
213 %     }
214 %
215 %     this.setState({ status: 'generating', progressLabel: 'Phi-3 M
216 %
217 %     const prompt = `${SYSTEM_PROMPT}
218 %
219 % Contexto: O usuário falou sobre "${subject}" (categoria: ${catego
220 % Mensagem do usuário: "${userMessage}"
221 % Resposta bruta do N-Gram (pode estar incompleta ou com problemas)
222 %
223 % Melhore a resposta acima. Resposta corrigida:`;
224 %
225 %     try {
226 %         const output: any = await generatorInstance(prompt, {
227 %             max_new_tokens: 120,
228 %             temperature: 0.6,
229 %             do_sample: true,
230 %             top_p: 0.9,
231 %             return_full_text: false,
232 %         });
233 %
234 %         const improved: string = (output?.[0]?.generated_text || ra
235 %         const tokenCount = improved.split(/ /).length;
236 %
237 %         this.setState({
238 %             status: 'ready',
239 %             progressLabel: `Phi-3 Mini corrigiu a resposta (${token
240 %         });
241 %
242 %         return {
243 %             improved: improved || rawNgramResponse,
244 %             confidence: 0.92,
245 %             model: currentModelId || PHI3_MODEL_ID,
246 %             usedBackend: this.state.backend || 'wasm',
247 %             tokensGenerated: tokenCount,
248 %         };
249 %     } catch (e: any) {
250 %         this.setState({
251 %             status: 'ready',
252 %             progressLabel: `&#x2610 Phi-3 Mini falhou - mantendo resposta
253 %         });
254 %         throw e;
255 %     }
256 % }
257 %
258 % / * *
259 % * Faz o Phi-3 pesquisar/explicar um assunto baseado no context
260 % * /

```

```

261 %         public async research(
262 %             userMessage: string,
263 %             category: string,
264 %             subject: string
265 %         ): Promise<Phi3CorrectionResult> {
266 %             if (!generatorInstance || this.state.status !== 'ready') {
267 %                 throw new Error('Phi-3 Mini ainda não está carregado.');
```

268 % }

269 %

270 % this.setState({ status: 'generating', progressLabel: `Phi-3 M

271 %

272 % const researchPrompt = `Você é um assistente técnico especial

273 % Responda em português brasileiro, de forma clara e objetiva (máxi

274 % Não use respostas prontas. Baseie-se no que sabe sobre \${subject}

275 %

276 % Pergunta do usuário: \${userMessage}

277 %

278 % Resposta técnica: `;

279 %

280 % try {

281 % const output: any = await generatorInstance(researchPrompt,

282 % max_new_tokens: 180,

283 % temperature: 0.5,

284 % do_sample: true,

285 % top_p: 0.92,

286 % return_full_text: false,

287 % });

288 %

289 % const text: string = (output?.[0]?.generated_text || '').tr

290 % const tokenCount = text.split(/ /).length;

291 %

292 % this.setState({

293 % status: 'ready',

294 % progressLabel: `Phi-3 Mini pesquisou sobre "\${subject}"

295 % });

296 %

297 % return {

298 % improved: text,

299 % confidence: 0.88,

300 % model: currentModelId || PHI3_MODEL_ID,

301 % usedBackend: this.state.backend || 'wasm',

302 % tokensGenerated: tokenCount,

303 % };

304 % } catch (e: any) {

305 % this.setState({

306 % status: 'ready',

307 % progressLabel: `⚠️ Phi-3 Mini pesquisa falhou`,

308 % });

309 % throw e;

310 % }

311 % }

312 %

313 % public isReady(): boolean {

314 % return this.state.status === 'ready';

315 % }

316 %

317 % public reset(): void {

318 % generatorInstance = null;

319 % currentModelId = null;

320 % this.setState({

321 % status: 'idle',

322 % progress: 0,

323 % progressLabel: 'Phi-3 Mini: aguardando ativação',

324 % backend: null,

325 % error: undefined,

326 % });


```
3 2 7 %    }  
3 2 8 %    }  
3 2 9 %
```

Ø=ÜÄ src/generation/pos_engine.ts

```
1 % // =====
2 % // POS ENGINE - Taxonomia com 100+ classes de palavra
3 % // Classifica tokens para guiar o Adaptive N-Gram sem respostas p
4 % // =====
5 %
6 % export type PartOfSpeech = string;
7 %
8 % export interface TokenClassification {
9 %     word: string;
10 %     pos: PartOfSpeech;
11 % }
12 %
13 % export const POS_CLASS_NAMES = [
14 %     'Saudacao', 'Despedida', 'Agradecimento', 'Pedido', 'Pergunta',
'Discordancia',
15 %     'EmocaoPositiva', 'EmocaoNegativa', 'Empatia', 'Motivacao', 'Hu
'Pessoa',
16 %     'PronomePessoal', 'PronomePossessivo', 'Artigo', 'Preposicao',
'Pontuacao', 'Numero',
17 %     'VerboAcao', 'VerboEstado', 'VerboCognitivo', 'VerboComunicacao
'SubstantivoAbstrato', 'SubstantivoTech', 'SubstantivoJogo',
18 %     'AdjetivoQualidade', 'AdjetivoTecnico', 'ConectorExplicativo',
'ConectorSequencia', 'ConectorCondicao', 'MarcadorExemplo', 'MarcadorResumo',
19 %     'CodigoGeral', 'LinguagemProgramacao', 'TermoJavaScript', 'Term
'EstruturaLua', 'TermoRoblox', 'ServicoRoblox',
20 %     'ObjetoRoblox', 'EventoRoblox', 'DataStoreRoblox', 'UIRoblox',
'SolucaoBug', 'DebugTermo', 'StackTrace',
21 %     'SintaxeCodigo', 'OperadorCodigo', 'TipoDado', 'EstruturaContro
'BancoDados', 'APIWeb', 'RedeHTTP',
22 %     'FrameworkFrontend', 'FrameworkBackend', 'BuildTool', 'TestesOf
'SegurancaWeb', 'IAConceito', 'MLConceito',
23 %     'NLPConceito', 'RAGConceito', 'EmbeddingConceito', 'ModeloLingu
'MemoriaConceito', 'ContextoConceito', 'RaciocinioConceito',
24 %     'MatematicaGeral', 'OperacaoMatematica', 'Algebra', 'Geometria'
'Porcentagem', 'LogicaMatematica', 'UnidadeMedida',
25 %     'JogoGeral', 'JogoRoblox', 'JogoMinecraft', 'JogoFreeFire', 'Ga
'GiriaBR', 'Interjeicao',
26 %     'FragmentoIncompleto', 'RiscoAlucinacao', 'RespostaCurta', 'Res
'HistoricoConversa', 'ConhecimentoGlobal', 'Neutro', 'Geral'
27 % ] as const;
28 %
29 % export class POSEngine {
30 %     private dictionary: Map<string, PartOfSpeech> = new Map();
31 %
32 %     constructor() {
33 %         this.initializePOSDictionary();
34 %     }
35 %
36 %     private initializePOSDictionary() {
37 %         this.registerWords(['oi', 'olá', 'ola', 'salve', 'hey', 'eai', 'eae
38 %         this.registerWords(['tchau', 'falou', 'flw', 'adeus', 'bye', 'até
39 %         this.registerWords(['obrigado', 'obrigada', 'valeu', 'grato', 'ag
40 %         this.registerWords(['crie', 'faça', 'gera', 'gere', 'mostre', 'exp
41 %         this.registerWords(['qual', 'quem', 'quando', 'onde', 'porque', 'p
42 %         this.registerWords(['dúvida', 'duvida', 'ajuda', 'help', 'socorro
43 %         this.registerWords(['não', 'nao', 'nunca', 'jamaiz', 'nem', 'sem']
44 %         this.registerWords(['sim', 'claro', 'certo', 'ok', 'okay', 'beleza
45 %         this.registerWords(['concordo', 'exato', 'perfeito', 'correto', '
46 %         this.registerWords(['discordo', 'errado', 'incorreto', 'falhou',
47 %         this.registerWords(['feliz', 'animado', 'empolgado', 'orgulhoso'
48 %         this.registerWords(['triste', 'cansado', 'chateado', 'bravo', 'ru
49 %         this.registerWords(['entendo', 'calma', 'sinto', 'desculpa', 'apo
50 %         this.registerWords(['continue', 'tente', 'consegue', 'persistênc
```

```
5 1 %      this.registerWords(['kkkk', 'kkk', 'haha', 'rsrs'], 'Humor');
5 2 %      this.registerWords(['muito', 'super', 'extremamente', 'totalment
5 3 %      this.registerWords(['talvez', 'meio', 'pouco', 'quase', 'aproxima
5 4 %      this.registerWords(['hoje', 'amanhã', 'ontem', 'agora', 'depois',
5 5 %      this.registerWords(['aqui', 'ali', 'lá', 'onde', 'local', 'lugar']
5 6 %      this.registerWords(['eu', 'você', 'voce', 'ele', 'ela', 'nós', 'nos
5 7 %      this.registerWords(['meu', 'minha', 'seu', 'sua', 'nosso', 'nossa'
5 8 %      this.registerWords(['o', 'a', 'os', 'as', 'um', 'uma'], 'Artigo');
5 9 %      this.registerWords(['de', 'da', 'do', 'em', 'no', 'na', 'para', 'com
6 0 %      this.registerWords(['e', 'ou', 'mas', 'porém', 'porem', 'entretant
6 1 %      this.registerWords(['assim', 'rapidamente', 'lentamente', 'bem',
6 2 %      this.registerWords(['quando', 'agora', 'depois', 'hoje', 'sempre'
6 3 %      this.registerWords(['onde', 'aqui', 'ali', 'lá'], 'AdverbioLugar
6 4 %      this.registerWords(['.', '?', '!', ',', ':', ';'], 'Pontuacao');
6 5 %
6 6 %
this.registerWords(['código', 'codigo', 'script', 'função', 'function', 'sistema', 'arquitetura', 'algoritmo', 'programação', 'programacao'],
'SubstantivoTech');
6 7 %
this.registerWords(['programar', 'compilar', 'depurar', 'executar', 'iterar', 'clonar', 'implementar', 'desenvolver', 'otimizar', 'refatorar'],
'VerboTecnico');
6 8 %      this.registerWords(['portanto', 'todavia', 'entretanto', 'visto
'ConectorExplicativo');
6 9 %      this.registerWords(['porque', 'pois', 'causa', 'devido', 'logo'],
7 0 %      this.registerWords(['mas', 'porém', 'porem', 'entretanto', 'todav
7 1 %      this.registerWords(['então', 'portanto', 'logo', 'concluindo', 'r
7 2 %      this.registerWords(['primeiro', 'segundo', 'terceiro', 'depois',
7 3 %      this.registerWords(['se', 'caso', 'quando', 'enquanto'], 'Conect
7 4 %      this.registerWords(['exemplo', 'exemplos', 'tipo'], 'MarcadorEx
7 5 %      this.registerWords(['resumo', 'resumindo', 'conclusão', 'conclus
7 6 %
7 7 %      this.registerWords(['javascript', 'js'], 'TermoJavaScript');
7 8 %      this.registerWords(['typescript', 'ts', 'tsx', 'jsx'], 'TermoTyp
7 9 %      this.registerWords(['html', 'tag', 'div', 'span', 'input'], 'Term
8 0 %      this.registerWords(['css', 'flex', 'grid', 'tailwind', 'style'],
8 1 %      this.registerWords(['python', 'py', 'def', 'self', 'lambda'], 'Te
8 2 %      this.registerWords(['lua', 'luaui', 'local', 'pairs', 'ipairs', 'me
'EstruturaLua');
8 3 %      this.registerWords(['roblox', 'workspace', 'players', 'humanoid'
event', 'remotefunction', 'datastore', 'part', 'instance'], 'TermoRoblox');
8 4 %
this.registerWords(['workspace', 'players', 'replicatedstorage', 'serverscriptservice', 'startergui', 'datastoreservice', 'tweenservice'],
'ServicoRoblox');
8 5 %      this.registerWords(['part', 'model', 'humanoid', 'character', 'to
8 6 %      this.registerWords(['remoteevent', 'remotefunction', 'fireclien
'EventoRoblox');
8 7 %      this.registerWords(['datastore', 'getasync', 'setasync', 'update
8 8 %      this.registerWords(['screenui', 'frame', 'textlabel', 'button',
8 9 %      this.registerWords(['velocity', 'bodyvelocity', 'anchored', 'cfr
9 0 %      this.registerWords(['server', 'cliente', 'client', 'validar', 'ex
9 1 %
9 2 %      this.registerWords(['erro', 'error', 'bug', 'falha', 'problema',
9 3 %      this.registerWords(['nil', 'undefined', 'null', 'missing', 'synta
9 4 %      this.registerWords(['solução', 'solucao', 'resolve', 'corrige',
9 5 %      this.registerWords(['debug', 'log', 'print', 'console', 'output',
9 6 %      this.registerWords(['if', 'else', 'for', 'while', 'switch', 'retur
9 7 %      this.registerWords(['+', '-', '*', '/', '=', '==', '===', '>', '<'],
9 8 %      this.registerWords(['string', 'number', 'boolean', 'nil', 'null',
9 9 %      this.registerWords(['variavel', 'variável', 'constante', 'let',
10 0 %      this.registerWords(['require', 'import', 'export', 'module', 'pac
10 1 %      this.registerWords(['sql', 'sqlite', 'indexeddb', 'dexie', 'datab
10 2 %      this.registerWords(['api', 'endpoint', 'request', 'response', 'fe
10 3 %      this.registerWords(['react', 'vite', 'next', 'vue', 'svelte'], 'F
10 4 %      this.registerWords(['express', 'node', 'server', 'backend'], 'Fr
10 5 %      this.registerWords(['build', 'bundle', 'vite', 'webpack', 'rollup
10 6 %      this.registerWords(['teste', 'test', 'jest', 'vitest', 'playwrih
```

```

107 %         this.registerWords(['git', 'commit', 'branch', 'merge', 'pull', 'p
108 %         this.registerWords(['deploy', 'docker', 'container', 'ci', 'pipel
109 %         this.registerWords(['cloud', 'nuvem', 'storage', 'backup', 'persi
110 %         this.registerWords(['xss', 'csrf', 'injection', 'token', 'senha',
111 %
112 %         this.registerWords(['ia', 'ai', 'inteligencia', 'inteligência', '
113 %         this.registerWords(['machine', 'learning', 'treino', 'dataset', '
114 %         this.registerWords(['texto', 'linguagem', 'token', 'tokenização'
115 %         this.registerWords(['rag', 'retrieval', 'documentos', 'busca', 'c
116 %         this.registerWords(['embedding', 'embeddings', 'vetor', 'vetoria
117 %         this.registerWords(['markov', 'cadeia', 'ngrams', 'n-gram', 'bigr
118 %         this.registerWords(['like', 'dislike', 'superlike', 'feedback', '
119 %         this.registerWords(['memoria', 'memória', 'historico', 'histórico
120 %         this.registerWords(['contexto', 'assunto', 'tema', 'intenção', 'i
121 %         this.registerWords(['raciocinio', 'raciocínio', 'lógica', 'logic
122 %
123 %         this.registerWords(['matematica', 'matemática', 'calculo', 'cálco
124 %         this.registerWords(['soma', 'subtracao', 'subtração', 'multiplic
'OperacaoMatematica');
125 %         this.registerWords(['algebra', 'álgebra', 'equação', 'equacao', '
126 %         this.registerWords(['geometria', 'triangulo', 'triângulo', 'area
127 %         this.registerWords(['seno', 'cosseno', 'tangente', 'hipotenusa', '
128 %         this.registerWords(['probabilidade', 'chance', 'aleatorio', 'ale
129 %         this.registerWords(['estatistica', 'estatística', 'media', 'médi
130 %         this.registerWords(['porcentagem', 'percentual', 'desconto', 'ju
131 %         this.registerWords(['verdadeiro', 'falso', 'condicao', 'condição
132 %         this.registerWords(['metro', 'segundo', 'pixel', 'bytes', 'mb', 'g
133 %
134 %         this.registerWords(['jogo', 'game', 'player', 'partida', 'mapa'],
135 %         this.registerWords(['minecraft', 'craft', 'creeper', 'bloco'], '
136 %         this.registerWords(['free', 'fire', 'fff', 'garena'], 'JogoFreeFi
137 %         this.registerWords(['nobru', 'neymar', 'youtuber', 'streamer'],
138 %         this.registerWords(['mano', 'cara', 'veí', 'mó', 'top', 'bora'],
139 %         this.registerWords(['nossa', 'uau', 'caramba', 'mds', 'eita'], 'I
140 %         this.registerWords(['estão', 'estao', 'coisas', 'bem', 'você', 'vo
141 %     }
142 %
143 %     private registerWords(words: string[], pos: PartOfSpeech) {
144 %         for (const w of words) {
145 %             this.dictionary.set(w.toLowerCase().normalize('NFD').replac
146 %         }
147 %     }
148 %
149 %     public getPOS(word: string): PartOfSpeech {
150 %         const clean = word.toLowerCase().normalize('NFD').replace(/[\
151 %         if (this.dictionary.has(clean)) return this.dictionary.get(cl
152 %         if (/^\d+(\.\d+)?$/ .test(clean)) return 'Numero';
153 %         if (['.', ',', '?', '!', ':', ';'].includes(clean)) return 'P
154 %         if (clean.endsWith('ar') || clean.endsWith('er') || clean.end
155 %         if (clean.endsWith('cao') || clean.endsWith('dade') || clean.
156 %         if (clean.endsWith('mente')) return 'AdverbioModo';
157 %         return 'Geral';
158 %     }
159 %
160 %     public getAllClasses(): readonly string[] {
161 %         return POS_CLASS_NAMES;
162 %     }
163 %
164 %     public analyzeSentenceClasses(sentence: string): { tokens: Token
165 %         const words = sentence.toLowerCase().match(/[\\wÃ-ú]+/g) || [];
166 %         const tokens: TokenClassification[] = [];
167 %         const classesFoundSet = new Set<PartOfSpeech>();
168 %         for (const w of words) {
169 %             if (w.length > 2) {
170 %                 const pos = this.getPOS(w);
171 %                 tokens.push({ word: w, pos });

```

```

172 %             classesFoundSet.add(pos);
173 %         }
174 %     }
175 %     return { tokens, classesFound: Array.from(classesFoundSet) };
176 % }
177 %
178 % public calculatePOSModifier(candidateToken: string, isTechnical
179 %     const pos = this.getPOS(candidateToken);
180 %     if (isTechnicalIntent) {
181 %         if (pos === 'Saudacao' || pos === 'Despedida' || pos === 'A
182 %         if (['VerboTecnico', 'SubstantivoTech', 'SolucaoBug', 'Debu
'ConectorSequencia'].includes(pos)) return 1.5;
183 %         if (['TermoRoblox', 'ServicoRoblox', 'ObjetoRoblox', 'Event
'TermoHTML', 'TermoCSS'].includes(pos)) return 1.7;
184 %         if (['ErroProgramacao', 'StackTrace', 'SegurancaWeb', 'Banc
185 %         if (pos === 'GiriaBR' || pos === 'Interjeicao') return 0.45
186 %     } else {
187 %         if (pos === 'Saudacao' && generatedWordsCount > 3) return 0
188 %         if (pos === 'FragmentoIncompleto') return 0.12;
189 %         if (['ConectorExplicativo', 'ConectorSequencia', 'Empatia',
190 %     }
191 %     return 1.0;
192 % }
193 % }

```

```

1 % // =====
2 % // RESPONSE RANKER - Avaliador de Qualidade da Resposta
3 % // Mede coerência, relevância, completude e alinhamento ao vetor
4 % // =====
5 %
6 % import { ContextVector, ContextEngine } from './context_engine';
7 %
8 % export interface RankedCandidate {
9 %     text: string;
10 %     tokens: { token: string; prob: number }[];
11 %     coherenceScore: number;
12 %     relevanceScore: number;
13 %     completenessScore: number;
14 %     contextAlignmentScore: number;
15 %     totalScore: number;
16 % }
17 %
18 % const contextEngine = new ContextEngine();
19 %
20 % /**
21 %  * Calcula um score combinado de altíssima precisão para a resposta
22 %  * Garante que a IA soe super inteligente e alinhada ao contexto.
23 %  */
24 % export function rankResponse(
25 %     generatedText: string,
26 %     tokens: { token: string; prob: number }[],
27 %     userMessage: string,
28 %     contextVector: ContextVector
29 % ): RankedCandidate {
30 %     const normalized = generatedText.toLowerCase().trim();
31 %     const words = (normalized.match(/[\\w\\u00c0-\\u00ff]+/g) || []).length;
32 %
33 %     // 1. Coerência: avalia tamanho da frase em relação à estrutura
34 %     let coherenceScore = 0.8;
35 %     if (words < 2) {
36 %         coherenceScore = 0.2; // Risco de ter gerado algo muito incompleto
37 %     } else if (contextVector.expectedStructure === 'curta' && words < 10) {
38 %         coherenceScore = 0.4; // Excesso de palavras para uma resposta curta
39 %     } else if (contextVector.expectedStructure === 'explicativa' && words > 50) {
40 %         coherenceScore = 0.5; // Falta de detalhes na explicação
41 %     }
42 %
43 %     // Penalidade severa para loops ou repetições de palavras exatas
44 %     const uniqueWords = new Set(normalized.match(/[\\w\\u00c0-\\u00ff]+/g) || []);
45 %     const repetitionRatio = uniqueWords.size / Math.max(1, words);
46 %     if (repetitionRatio < 0.45) {
47 %         coherenceScore *= 0.4;
48 %     }
49 %
50 %     // 2. Relevância: sobreposição com os tokens do usuário
51 %     const userTokens = new Set(
52 %         (userMessage.toLowerCase().match(/[\\w\\u00c0-\\u00ff]+/g) || []).filter(w => w.length > 3)
53 %     );
54 %     const responseTokens = normalized.match(/[\\w\\u00c0-\\u00ff]+/g) || [];
55 %     const overlapCount = responseTokens.filter(w => userTokens.has(w)).length;
56 %     const relevanceScore = Math.min(1, (overlapCount / Math.max(1, responseTokens.length)));
57 %
58 %     // 3. Completude: terminou com pontuação correta ou formato de frase
59 %     const endsWithPunctuation = /[.!?]$/.test(generatedText.trim());
60 %     const hasSentenceStructure = /[.!?]/.test(normalized);
61 %     let completenessScore = 0.4;
62 %     if (endsWithPunctuation) completenessScore += 0.35;
63 %     if (hasSentenceStructure) completenessScore += 0.25;

```

```
64 %
65 % // 4. Alinhamento ao Vetor de Contexto (Context Engine)
66 % const validation = contextEngine.validateCoherence(generatedText)
67 % const contextAlignmentScore = validation.score;
68 %
69 % // Pontuação Total Ponderada (Fórmula do Motor de Inteligência)
70 % const totalScore =
71 %     coherenceScore * 0.25 +
72 %     relevanceScore * 0.25 +
73 %     completenessScore * 0.20 +
74 %     contextAlignmentScore * 0.30;
75 %
76 % return {
77 %     text: generatedText,
78 %     tokens,
79 %     coherenceScore,
80 %     relevanceScore,
81 %     completenessScore,
82 %     contextAlignmentScore,
83 %     totalScore: Math.min(1, Math.max(0, totalScore)),
84 % };
85 % }
86 %
```

```

1 % // =====
2 % // SEMANTIC SEARCH - Busca contextual com TF-IDF Real e Relevância
3 % // Analisa a frase inteira, intenções e entidades para pesquisa d
4 % // =====
5 %
6 % export interface SearchResult {
7 %     text: string;
8 %     title: string;
9 %     category: string;
10 %     score: number;
11 %     matchedTerms: string[];
12 % }
13 %
14 % /**
15 %  * Tokeniza e normaliza removendo stopwords e ruído
16 %  */
17 % function meaningfulTokens(text: string): string[] {
18 %     const stopwords = new Set([
19 %         'de', 'da', 'do', 'dos', 'das', 'e', 'a', 'o', 'as', 'os',
20 %         'um', 'uma', 'uns', 'umas', 'para', 'com', 'por', 'que', 'no',
21 %         'se', 'é', 'foi', 'ser', 'em', 'ao', 'aos', 'the', 'is', 'of',
22 %         'to', 'in', 'it', 'you', 'that', 'was', 'are', 'on', 'at', 'b
23 %     ]);
24 %
25 %     return text
26 %         .toLowerCase()
27 %         .normalize('NFD')
28 %         .replace(/[\u0300-\u036f]/g, '')
29 %         .match(/[\\w]+/g)
30 %         ?.filter(w => w.length > 2 && !stopwords.has(w)) || [];
31 % }
32 %
33 % /**
34 %  * Calcula similaridade TF-IDF entre consulta e documento com bôn
35 %  */
36 % function tfIdfSimilarity(query: string, document: string): { scor
37 %     const queryTokens = meaningfulTokens(query);
38 %     const docTokens = meaningfulTokens(document);
39 %
40 %     if (queryTokens.length === 0 || docTokens.length === 0) {
41 %         return { score: 0, matched: [] };
42 %     }
43 %
44 %     // TF do documento (contagem de termos)
45 %     const docTf = new Map<string, number>();
46 %     for (const t of docTokens) {
47 %         docTf.set(t, (docTf.get(t) || 0) + 1);
48 %     }
49 %
50 %     const docLength = docTokens.length;
51 %     const matched: string[] = [];
52 %
53 %     let score = 0;
54 %     for (const qToken of queryTokens) {
55 %         if (docTf.has(qToken)) {
56 %             const tf = docTf.get(qToken)! / docLength;
57 %             const idf = 1.2; // IDF base ponderado
58 %             score += tf * idf;
59 %             matched.push(qToken);
60 %         }
61 %     }
62 %
63 %     // Normaliza pelo tamanho da consulta

```



```

64 %     score = score / Math.sqrt(queryTokens.length);
65 %
66 %     // Bônus por correspondência exata de bigramas (frases de 2 pal
67 %     const queryBigrams = new Set<string>();
68 %     for (let i = 0; i < queryTokens.length - 1; i++) {
69 %         queryBigrams.add(`${queryTokens[i]} ${queryTokens[i + 1]}`);
70 %     }
71 %
72 %     let bigramBonus = 0;
73 %     for (let i = 0; i < docTokens.length - 1; i++) {
74 %         const bigram = `${docTokens[i]} ${docTokens[i + 1]}`;
75 %         if (queryBigrams.has(bigram)) {
76 %             bigramBonus += 0.45; // Reforço considerável para expressões
77 %         }
78 %     }
79 %
80 %     return { score: score + bigramBonus, matched };
81 % }
82 %
83 % /**
84 %  * Busca os documentos mais relevantes para uma consulta (Pesquis
85 %  */
86 % export function semanticSearch(
87 %     query: string,
88 %     documents: { text: string; title: string; category: string }[],
89 %     topK: number = 5
90 % ): SearchResult[] {
91 %     const scored: SearchResult[] = [];
92 %     const queryTokensNorm = meaningfulTokens(query).join(' ');
93 %
94 %     for (const doc of documents) {
95 %         const { score, matched } = tfIdfSimilarity(query, doc.text);
96 %
97 %         // Calcula um bônus adicional se o título do documento coinci
98 %         let titleBonus = 0;
99 %         const titleTokens = meaningfulTokens(doc.title);
100 %         if (titleTokens.some(t => queryTokensNorm.includes(t))) {
101 %             titleBonus = 0.35;
102 %         }
103 %
104 %         const finalScore = score + titleBonus;
105 %
106 %         if (finalScore > 0.05) {
107 %             scored.push({
108 %                 text: doc.text,
109 %                 title: doc.title,
110 %                 category: doc.category,
111 %                 score: finalScore,
112 %                 matchedTerms: matched,
113 %             });
114 %         }
115 %     }
116 %
117 %     // Ordena por score decrescente
118 %     scored.sort((a, b) => b.score - a.score);
119 %
120 %     return scored.slice(0, topK);
121 % }
122 %
123 % /**
124 %  * Analisa a frase inteira para extrair entidades e intenção
125 %  */
126 % export function analyzePhrase(text: string): {
127 %     entities: string[];
128 %     questionType: 'who' | 'what' | 'how' | 'where' | 'when' | 'why'
129 %     subject: string | null;

```

```

130 % } {
131 %   const normalized = text.toLowerCase().normalize('NFD').replace(
132 %   const tokens = meaningfulTokens(text);
133 %
134 %   let questionType: ReturnType<typeof analyzePhrase>['questionType'];
135 %   if (normalized.match(/^(quem|qual|quais)\b/)) questionType = 'who';
136 %   else if (normalized.match(/^(o que|oque)\b/)) questionType = 'what';
137 %   else if (normalized.match(/^(como|onde|quando|por que|porque|por
138 %       if (normalized.startsWith('como')) questionType = 'how';
139 %       else if (normalized.startsWith('onde')) questionType = 'where';
140 %       else if (normalized.startsWith('quando')) questionType = 'when';
141 %       else questionType = 'why';
142 %   } else if (normalized.match(/^(você|voce|tu|vcs|vocês)\b/)) questionType = 'you';
143 %   else if (normalized.endsWith('?')) questionType = 'yesno';
144 %
145 %   const entities = (text.match(/\b[A-ZÀ-Ú][a-zà-ú]+\b/g) || []).filter(
146 %       e => e.length > 2 && !['Que', 'Qual', 'Quem', 'Como', 'Onde',
147 %   );
148 %
149 %   const subject = tokens.length > 0 ? tokens[tokens.length - 1] :
150 %
151 %   return { entities, questionType, subject };
152 % }
153 %

```

```

1 % // =====
2 % // CONTROLE DE PARADA DA GERAÇÃO
3 % // Evita respostas longas demais, mudança de assunto e queda de c
4 % // Não gera respostas prontas: apenas decide quando parar o N-Gr
5 % // =====
6 %
7 % import type { Category } from '../classifier/intent_classifier';
8 %
9 % export interface GeneratedToken {
10 %     token: string;
11 %     prob: number;
12 % }
13 %
14 % export interface StopDecision {
15 %     accept: boolean;
16 %     stop: boolean;
17 %     reason?: string;
18 % }
19 %
20 % const TECH_TOKENS = new Set([
21 %     'workspace', 'function', 'nil', 'remoteevent', 'remotefunction'
22 %     'replicatedstorage', 'serverscriptservice', 'startergui', 'loca
23 %     'metatable', 'javascript', 'typescript', 'python', 'lua', 'robl
24 %     'print', 'waitforchild', 'findfirstchild', 'instance', 'workspa
25 % ]);
26 %
27 % const CONVERSATION_TOKENS = new Set([
28 %     'oi', 'olá', 'ola', 'bom', 'boa', 'dia', 'tarde', 'noite', 'tud
29 %     'você', 'voce', 'obrigado', 'obrigada', 'ajudar', 'ajuda', 'com
30 %     'prazer', 'conversar', 'legal', 'interessante', 'mensagem', 'de
31 % ]);
32 %
33 % function normalize(token: string): string {
34 %     return token.toLowerCase().normalize('NFD').replace(/[\u0300-\u
35 % }
36 %
37 % function countWords(text: string): number {
38 %     return (text.match(/[\wÀ-ú]+/g) || []).length;
39 % }
40 %
41 % function isGreeting(text: string): boolean {
42 %     const normalized = normalize(text.trim());
43 %     return /^(oi|ola|olá|bom dia|boa tarde|boa noite|eai|eae|fala|h
44 % }
45 %
46 % function sentenceWords(sentence: string): Set<string> {
47 %     return new Set((sentence.match(/[\wÀ-ú]+/g) || []).map(normaliz
48 % }
49 %
50 % export class GenerationStopController {
51 %     private firstProbs: number[] = [];
52 %     private recentProbs: number[] = [];
53 %     private acceptedTokens: string[] = [];
54 %     private completedSentences = 0;
55 %     private readonly simpleMessage: boolean;
56 %     private readonly greeting: boolean;
57 %     private readonly maxTokens: number;
58 %     private userMessageTokens: Set<string>;
59 %
60 %     constructor(private category: Category, userMessage: string) {
61 %         const words = countWords(userMessage);
62 %         this.greeting = isGreeting(userMessage);
63 %         this.simpleMessage = words <= 4 && ['CONVERSA', 'PORTUGUÊS'].

```

```

64 %         this.maxTokens = this.greeting ? 15 : this.simpleMessage ? 28
65 %         this.userMessageTokens = sentenceWords(userMessage);
66 %     }
67 %
68 %     evaluate(next: GeneratedToken, currentText: string): StopDecision {
69 %         const token = normalize(next.token);
70 %         const wordsSoFar = countWords(currentText);
71 %
72 %         if (this.acceptedTokens.length >= this.maxTokens) {
73 %             return { accept: false, stop: true, reason: 'limite_de_tamanho' };
74 %         }
75 %
76 %         // Mudança de assunto: conversa simples não deve entrar em contexto
77 %         if (['CONVERSA', 'PORTUGUÊS'].includes(this.category) && TECH_WORDS.length > 0) {
78 %             return { accept: false, stop: true, reason: 'mudanca_de_assunto' };
79 %         }
80 %
81 %         // Categoria técnica não deve virar conversa solta demais logo
82 %         if (['PROGRAMAÇÃO', 'ROBLOX', 'LUAU', 'JAVASCRIPT', 'MATEMÁTICA'].includes(this.category) &&
83 %             if (wordsSoFar > 10 && CONVERSATION_TOKENS.has(token) && this.category !== 'CONVERSA') {
84 %                 return { accept: false, stop: true, reason: 'mudanca_de_assunto' };
85 %             }
86 %         }
87 %
88 %         // Sistema de confiança: se a confiança cai muito, para antes de gerar
89 %         if (this.acceptedTokens.length < 4) this.firstProbs.push(next.token);
90 %         this.recentProbs.push(next.token);
91 %         if (this.recentProbs.length > 4) this.recentProbs.shift();
92 %
93 %         const baseConfidence = this.firstProbs.reduce((a, b) => a + b);
94 %         const recentConfidence = this.recentProbs.reduce((a, b) => a + b);
95 %         if (this.acceptedTokens.length > 8 && baseConfidence > 0 && recentConfidence < 0.5) {
96 %             return { accept: false, stop: true, reason: 'queda_de_confianca' };
97 %         }
98 %         if (this.acceptedTokens.length > 6 && next.token < 0.002) {
99 %             return { accept: false, stop: true, reason: 'token_muito_baixo' };
100 %         }
101 %
102 %         // SCORE DE CONTEXTO E COERÊNCIA (Missão 2)
103 %         // Avalia a coesão de cada nova frase gerada em relação ao prompt e ao contexto
104 %         const sentences = currentText.split(/([.!?]+)/).map(s => s.trim());
105 %         if (sentences.length >= 2) {
106 %             const currentSentence = sentences[sentences.length - 1];
107 %             const previousSentence = sentences[sentences.length - 2];
108 %
109 %             const currWords = sentenceWords(currentSentence);
110 %             const prevWords = sentenceWords(previousSentence);
111 %
112 %             // Similaridade com a frase anterior (Coesão)
113 %             const prevOverlap = Array.from(currWords).filter(w => prevWords.includes(w)).length;
114 %
115 %             // Similaridade com a mensagem original do usuário (Alinhamento)
116 %             const userOverlap = Array.from(currWords).filter(w => this.userMessageTokens.includes(w)).length;
117 %
118 %             // Classificação/Assunto esperado: se estamos conversando,
119 %             const hasTechWord = Array.from(currWords).some(w => TECH_WORDS.includes(w));
120 %             const hasConvWord = Array.from(currWords).some(w => CONVERSATION_TOKENS.has(w));
121 %
122 %             let score = 0.5; // score inicial neutro
123 %
124 %             if (currWords.size > 1) {
125 %                 // Se a nova frase compartilha palavras com o prompt ou contexto
126 %                 const alignmentFactor = (userOverlap / currWords.size) * 0.2;
127 %                 const cohesionFactor = (prevOverlap / currWords.size) * 0.2;
128 %                 score = 0.2 + alignmentFactor + cohesionFactor;
129 %             }

```

```

130 % // Penalidade severa por desvio de assunto
131 % if ([ 'CONVERSA', 'PORTUGUES', 'INGLES' ].includes( this.category )) {
132 %     score -= 0.4;
133 % }
134 % if ([ 'LUA', 'ROBLOX', 'JAVASCRIPT', 'TYPESCRIPT', 'PYTHON' ].includes( this.category )) {
wordsSoFar > 12) {
135 %     score -= 0.3;
136 % }
137 % }
138 %
139 % // Se o score cair abaixo do limite de coerência (20%), encerra a resposta
140 % if (score < 0.20) {
141 %     return { accept: false, stop: true, reason: `desvio_de_coerencia` };
142 % }
143 % }
144 %
145 % return { accept: true, stop: false };
146 % }
147 %
148 % recordAccepted(token: string): StopDecision {
149 %     const normalized = normalize(token);
150 %     this.acceptedTokens.push(normalized);
151 %
152 %     // Sistema de finalização: frase completa encerra respostas simples
153 %     if (token === '.' || token === '?' || token === '!') {
154 %         this.completedSentences++;
155 %         if (this.greeting && this.acceptedTokens.length >= 3) {
156 %             return { accept: true, stop: true, reason: 'saudacao_completa' };
157 %         }
158 %         if (this.simpleMessage && this.acceptedTokens.length >= 5) {
159 %             return { accept: true, stop: true, reason: 'resposta_simples' };
160 %         }
161 %         if (this.completedSentences >= 2) {
162 %             return { accept: true, stop: true, reason: 'duas frases completas' };
163 %         }
164 %     }
165 %
166 %     return { accept: true, stop: false };
167 % }
168 % }
169 %

```

```
1 % // =====
2 % // CORPUS TÉCNICO DEFINITIVO E BANCADA DE CONHECIMENTO
3 % // Bancos completamente separados: PROGRAMAÇÃO, ROBLOX, LUAU, JAV
4 % // NENHUMA resposta pronta ou pares decorados.
5 % // =====
6 %
7 % export const PROGRAMMING_CORPUS = `
8 % A engenharia de software consiste em transformar problemas do mun
9 % Para garantir a longevidade e facilidade de manutenção de um sist
única.
10 % O desenvolvimento web divide-se em frontend, focado na interativi
regras de negócios.
11 % Arquiteturas modulares reduzem dependências cíclicas e garantem q
adjacentes.
12 % A injeção de dependência e a criação de interfaces bem definidas
infraestrutura.
13 % Bancos de dados relacionais garantem consistência transacional vi
dinâmicos e escala horizontal.
14 % `;
15 %
16 % export const ROBLOX_CORPUS = `
17 % Na hierarquia estrutural de um jogo no Roblox, o Workspace atua c
tridimensionais que interagem no mundo físico.
18 % O serviço Players gerencia todas as entidades conectadas ao servi
do place.
19 % Para realizar transferências de dados seguras e assíncronas entre
rigoroso de RemoteEvent e RemoteFunction.
20 % Enquanto o RemoteEvent lida com disparos de mão única via FireSer
que exigem um valor de retorno através de InvokeServer.
21 % O serviço ReplicatedStorage armazena módulos, interfaces e assets
locais e pelos scripts de servidor.
22 % O ServerScriptService atua como um repositório restrito e altamen
cliente final.
23 % O componente Humanoid reside no interior de um Model de personage
Health, velocidade de movimento WalkSpeed e aplicação de pacotes de animação.
24 % O DataStoreService possibilita a persistência de tabelas de progr
entre sessões através de chaves únicas.
25 % `;
26 %
27 % export const LUAU_CORPUS = `
28 % A linguagem Luau, derivada do Lua convencional, incorpora um sist
para ambientes de alto rendimento.
29 % A declaração de variáveis com a palavra-chave local confina a sua
na tabela de símbolos do interpretador.
30 % A iteração eficiente em tabelas numéricas ordenadas é realizada a
percorre índices sequenciais desde o número um.
31 % Para atravessar tabelas associativas ou dicionários que empregam
a declaração do laço de repetição.
32 % As metatabelas constituem um mecanismo avançado de sobrecarga e e
tabela mediante a definição de metamétodos.
33 % O metamétodo __index é disparado automaticamente sempre que um sc
o compartilhamento de métodos entre instâncias.
34 % O metamétodo __newindex intercepta atribuições a chaves virgens n
estado de um objeto em tempo real.
35 % O construtor Instance.new instancia novas classes no ambiente, ac
instantaneamente o objeto à hierarquia do pai.
36 % `;
37 %
38 % export const JAVASCRIPT_CORPUS = `
39 % No ecossistema JavaScript moderno, a declaração de referências im
ponteiro de memória permaneça constante.
40 % Variáveis destinadas a reatribuições dentro de um escopo de bloco
instrução let, evitando os efeitos colaterais de hoisting associados ao var.
```

4 1 % A programação assíncrona domina o fluxo de requisições de rede, u
podem ser resolvidos ou rejeitados.

4 2 % A sintaxe `async` e `await` fornece uma fachada sequencial limpa sobr
excessivo de manipuladores de callback.

4 3 % A manipulação imutável de coleções é realizada através de métodos
elementos e `array.filter` filtra subconjuntos com base em predicados lógicos.

4 4 % A invocação da API global `fetch` permite realizar requisições HTTP
retornando um fluxo de resposta consumível em formato JSON.

4 5 % ` ;

4 6 %

4 7 % `export const MATH_CORPUS = ``

4 8 % A matemática pura e aplicada estrutura os alicerces lógicos dos a

4 9 % A operação de adição combina parcelas para obter um total agregad
entre o minuendo e o subtraendo.

5 0 % A multiplicação representa somas repetidas de fatores para gerar
calcular o quociente e o resto.

5 1 % Em equações algébricas lineares do primeiro grau, o objetivo cent
operações matemáticas idênticas em ambos os lados da igualdade.

5 2 % A hipotenusa de um triângulo retângulo oposta ao ângulo de novent
da hipotenusa equivale à soma dos quadrados dos catetos.

5 3 % Operações trigonométricas como seno, cosseno e tangente estabelece
triângulo e as amplitudes de seus ângulos internos.

5 4 % As funções logarítmicas invertem a exponenciação, respondendo qua
produzir um determinado número absoluto.

5 5 % Cálculos de porcentagem estabelecem proporções com base no valor
compostos e taxas de desconto em sistemas financeiros.

5 6 % ` ;

5 7 %

5 8 % `export const ERRORS_CORPUS = ``

5 9 % A depuração de código exige uma leitura atenta da pilha de rastre

6 0 % O clássico erro `attempt to index nil value` manifesta-se sempre qu
referência que aponta para um endereço nulo ou não inicializado.

6 1 % A abordagem correta para prevenir o erro de índice nulo consiste
prévia, certificando-se de que a instância possui um valor válido antes de realizar a invocação.

6 2 % O aviso `infinite yield possible on WaitForChild` ocorre quando um
jamais foi criada ou nomeada de maneira incorreta dentro do escopo alvo.

6 3 % Para contornar travamentos em `WaitForChild`, recomenda-se fornecer
expiração tolerado para a busca.

6 4 % Erros apontados como `syntax error unexpected symbol` alertam que a
motivada por falta de parênteses de fechamento, vírgulas suprimidas em declarações de tabela ou a ausência da instrução `end` para selar
um bloco de controle.

6 5 % Em ambientes baseados em JavaScript, a falha `cannot read property`
de encadeamento opcional e coalescência nula.

6 6 % ` ;

6 7 %

6 8 % `export const DOCUMENTATION_CORPUS = ``

6 9 % A elaboração de uma documentação técnica impecável serve como o m
microserviços.

7 0 % Um arquivo README bem estruturado deve expor de maneira cristalina
as rotinas de inicialização da plataforma.

7 1 % Manuais de API especificam os contratos de dados, detalhando os p
respostas esperadas para cada código de status HTTP.

7 2 % A inclusão de exemplos práticos de utilização de bibliotecas redu
terceiros que desejam integrar módulos externos.

7 3 % O comentário de código deve priorizar a explicação do porquê uma
limpo já demonstra claramente o como ela funciona.

7 4 % ` ;

7 5 %

7 6 % `export const HELP_CORPUS = ``

7 7 % Como assistente inteligente, minha função central é esclarecer pa
implementação e colaborar na detecção e correção de falhas lógicas.

7 8 % Você pode fornecer fragmentos de código com defeito para que eu r
incrementais e elabore explicações pautadas nos manuais oficiais.

7 9 % ` ;

8 0 %

Fim de src/knowledge/programming_corpus.ts

```
1 % // =====
2 % //  L O G G E R   D E   A U D I T O R I A
3 % //  L o g s   s i m p l e s   e   r a s t r e á v e i s   p a r a   m e m ó r i a / p e r s i s t ê n c i a
4 % //  =====
5 %
6 % export const auditLog = (tag: string, payload: Record<string, unknown>): void => {
7 %     const stamp = new Date().toISOString();
8 %     console.log(`[${tag}]`, { at: stamp, ...payload });
9 % };
10 %
```



```

1 % // =====
2 % // MEMORY ENGINE - Memória Hierárquica, Perfil de Usuário e Conte
3 % // Implementa ETAPA 3 (Memória Hierárquica), ETAPA 4 (Perfil do U
Local).
4 % // =====
5 %
6 % import { db, User, Message, UserMemoryFact } from '../database/sc
7 % import { auditLog } from '../logs/logger';
8 %
9 % export interface UserMemoryProfile {
10 %     userId: string;
11 %     userName: string;
12 %     // ETAPA 3 - MEMÓRIA HIERÁRQUICA
13 %     memory: {
14 %         curtaDuracao: Message[]; // últimas mensagens
15 %         mediaDuracao: string[]; // assuntos recentes
16 %         longaDuracao: UserMemoryFact[]; // preferências do usuário
17 %     };
18 %     // ETAPA 4 - PERFIL DO USUÁRIO
19 %     profile: {
20 %         gostade: string[]; // Ex: Roblox, Lua, Slimes, IA
21 %         confidenceScore: number; // score de confiança do perfil
22 %     };
23 %     // ETAPA 17 - CONTEXTO GLOBAL
24 %     globalContext: {
25 %         assuntoAtual: string;
26 %         assuntoAnterior: string;
27 %         objetivoConversa: string;
28 %     };
29 %     interestsBiases: string[]; // Tokens extraídos para o N-Gram
30 % }
31 %
32 % export class MemoryEngine {
33 %     private SHORT_TERM_LIMIT = 12;
34 %
35 %     constructor() {}
36 %
37 %     /**
38 %      * Carrega o perfil completo de memória de um usuário com separ
39 %      * Nunca mistura usuários.
40 %      */
41 %     async loadUserProfile(user: User): Promise<UserMemoryProfile> {
42 %         const messages = await db.messages
43 %             .where('userId').equals(user.uuid)
44 %             .reverse()
45 %             .limit(this.SHORT_TERM_LIMIT)
46 %             .toArray();
47 %
48 %         const facts = await db.facts
49 %             .where('userId').equals(user.uuid)
50 %             .toArray();
51 %
52 %         const recentMessages = messages.reverse();
53 %
54 %         // ETAPA 3: Média duração (assuntos recentes baseados nas últ
55 %         const mediaDuracao: string[] = [];
56 %         for (const msg of recentMessages) {
57 %             if (msg.category && !mediaDuracao.includes(msg.category)) {
58 %                 mediaDuracao.push(msg.category);
59 %             }
60 %         }
61 %
62 %         // ETAPA 4: Perfil do Usuário (Constrói perfil automaticament

```

```

63 %         const gostaDeSet = new Set<string>();
64 %     for (const fact of facts) {
65 %         if (fact.key === 'preferencia' || fact.key === 'projeto_atu
66 %             const words = fact.value.toLowerCase().match(/[\\wÀ-ú]+/g)
67 %             for (const w of words) {
68 %                 if (w.length > 2) gostaDeSet.add(w);
69 %             }
70 %         }
71 %     }
72 %
73 %     // Injeção de interesses padrão detectados na conversa se o u
74 %     const allTextContent = recentMessages.map(m => m.content.toLo
75 %     if (allTextContent.includes('roblox')) gostaDeSet.add('Roblox
76 %     if (allTextContent.includes('lua')) gostaDeSet.add('Lua');
77 %     if (allTextContent.includes('slimes')) gostaDeSet.add('Slimes
78 %     if (allTextContent.includes('ia') || allTextContent.includes(
79 %
80 %     const gostaDe = Array.from(gostaDeSet);
81 %     const confidenceScore = Math.min(1.0, 0.5 + (gostaDe.length *
82 %
83 %     // ETAPA 17: Contexto Global (assunto atual, anterior e objet
84 %     let assuntoAtual = mediaDuracao.length > 0 ? mediaDuracao[med
85 %     let assuntoAnterior = mediaDuracao.length > 1 ? mediaDuracao[
86 %
87 %     let objetivoConversa = 'Explorar conceitos e responder dúvida
88 %     const lastUserMsg = recentMessages.filter(m => m.role === 'us
89 %     if (lastUserMsg) {
90 %         if (lastUserMsg.intention === 'corrigindo') objetivoConvers
91 %         else if (lastUserMsg.intention === 'reportando_erro') objet
92 %         else if (lastUserMsg.intention === 'perguntando') objetivoC
93 %         else if (lastUserMsg.intention === 'pedindo') objetivoConve
94 %     }
95 %
96 %     const interestsBiases = gostaDe.map(g => g.toLowerCase());
97 %
98 %     auditLog('LOAD_HIERARCHICAL_PROFILE', {
99 %         userId: user.uuid,
100 %         userName: user.name,
101 %         curtaCount: recentMessages.length,
102 %         mediaAssuntos: mediaDuracao,
103 %         gostaDe,
104 %         confidenceScore,
105 %     });
106 %
107 %     return {
108 %         userId: user.uuid,
109 %         userName: user.name,
110 %         memory: {
111 %             curtaDuracao: recentMessages,
112 %             mediaDuracao,
113 %             longaDuracao: facts,
114 %         },
115 %         profile: {
116 %             gostaDe,
117 %             confidenceScore,
118 %         },
119 %         globalContext: {
120 %             assuntoAtual,
121 %             assuntoAnterior,
122 %             objetivoConversa,
123 %         },
124 %         interestsBiases,
125 %     };
126 % }
127 %
128 % async saveUserMessage(msg: Message): Promise<number> {

```

```

129 %         const id = await db.messages.add(msg);
130 %         auditLog('SAVE_MSG', {
131 %             userId: msg.userId,
132 %             role: msg.role,
133 %             category: msg.category || 'CONVERSA',
134 %             intention: msg.intention || 'indefinida',
135 %             messageId: id,
136 %         });
137 %         return id;
138 %     }
139 %
140 %     async saveUserFact(userId: string, key: string, value: string):
141 %         const existing = await db.facts
142 %             .where('userId').equals(userId)
143 %             .and(f => f.key === key)
144 %             .first();
145 %
146 %         if (existing) {
147 %             await db.facts.update(existing.id!, { value, createdAt: new
148 %                 auditLog('UPDATE_FACT', { userId, key, value });
149 %         } else {
150 %             await db.facts.add({ userId, key, value, createdAt: new Date
151 %                 auditLog('SAVE_FACT', { userId, key, value });
152 %         }
153 %     }
154 %
155 %     async extractAndSaveFacts(userId: string, message: string): Promise<
156 %         const extracted: UserMemoryFact[] = [];
157 %
158 %         const nameMatch = message.match(/(?:meu nome é|me chamo|eu me
159 %         if (nameMatch) {
160 %             await this.saveUserFact(userId, 'nome_apresentado', nameMat
161 %             extracted.push({ userId, key: 'nome_apresentado', value: na
162 %         }
163 %
164 %         const projectMatch = message.match(/(?:estou fazendo|estou tr
em) ([^.!?]+)/i);
165 %         if (projectMatch) {
166 %             await this.saveUserFact(userId, 'projeto_atual', projectMat
167 %             extracted.push({ userId, key: 'projeto_atual', value: proje
168 %         }
169 %
170 %         const prefMatch = message.match(/(?:eu gosto de|gosto de|pref
171 %         if (prefMatch) {
172 %             await this.saveUserFact(userId, 'preferencia', prefMatch[1]
173 %             extracted.push({ userId, key: 'preferencia', value: prefMat
174 %         }
175 %
176 %         return extracted;
177 %     }
178 %
179 %     /**
180 %      * Constrói a string de contexto com base na Memória Hierárquica
181 %      */
182 %     buildMemoryContextString(profile: UserMemoryProfile): string {
183 %         const parts: string[] = [];
184 %
185 %         parts.push(`conversando com o usuario ${profile.userName}`);
186 %         parts.push(`[Objetivo da Conversa]: ${profile.globalContext.o
187 %         parts.push(`[Assunto Atual]: ${profile.globalContext.assuntoA
188 %
189 %         if (profile.profile.gostade.length > 0) {
190 %             parts.push(`o usuario gosta de: ${profile.profile.gostade.j
100).toFixed(0)}}%`));
191 %         }
192 %

```

```

193 %         for (const fact of profile.memory.longaDuracao) {
194 %             if (fact.key === 'projeto_atual') {
195 %                 parts.push(`o usuario esta trabalhando no projeto ${fact.value}`);
196 %             } else if (fact.key === 'preferencia') {
197 %                 parts.push(`o usuario prefere e gosta de ${fact.value}`);
198 %             } else if (fact.key === 'nome_apresentado') {
199 %                 parts.push(`o usuario confirmou que se chama ${fact.value}`);
200 %             }
201 %         }
202 %
203 %         // ETAPA 18 - CONTEXTO LOCAL (Mantém ligação com as últimas m
204 %         if (profile.memory.curtaDuracao.length > 0) {
205 %             const dialogues = profile.memory.curtaDuracao.slice(-6).map(m => {
206 %                 const speaker = m.role === 'user' ? profile.userName : 'a
207 %                 return `${speaker}: ${m.content}`;
208 %             });
209 %             parts.push(dialogues.join(' \n '));
210 %         }
211 %
212 %         return parts.join(' \n ');
213 %     }
214 % }
215 %

```

```

1 % // =====
2 % // MEMÓRIA POR USUÁRIO
3 % // Memória de curto prazo (últimas mensagens) + Longo prazo (fatos)
4 % // IMPORTANTE: cada usuário tem seu próprio espaço, nunca se misturam
5 % // =====
6 %
7 % import { db, User, Message, UserMemoryFact } from '../database/schema';
8 % import { auditLog } from '../logs/logger';
9 %
10 % export interface MemoryContext {
11 %     shortTerm: Message[]; // últimas N mensagens da conversa
12 %     longTerm: UserMemoryFact[]; // fatos salvos (nome, projeto, etc)
13 %     userName: string;
14 %     userId: string;
15 % }
16 %
17 % export class UserMemoryManager {
18 %     private SHORT_TERM_LIMIT = 12;
19 %
20 %     /**
21 %      * Carrega contexto completo de memória de um usuário específico
22 %      */
23 %     async load(user: User): Promise<MemoryContext> {
24 %         const messages = await db.messages
25 %             .where('userId').equals(user.uuid)
26 %             .reverse()
27 %             .limit(this.SHORT_TERM_LIMIT)
28 %             .toArray();
29 %
30 %         const facts = await db.facts
31 %             .where('userId').equals(user.uuid)
32 %             .toArray();
33 %
34 %         auditLog('LOAD', {
35 %             userId: user.uuid,
36 %             userName: user.name,
37 %             messages_loaded: messages.length,
38 %             facts_loaded: facts.length,
39 %             memory_loaded: true,
40 %         });
41 %
42 %         return {
43 %             shortTerm: messages.reverse(),
44 %             longTerm: facts,
45 %             userName: user.name,
46 %             userId: user.uuid,
47 %         };
48 %     }
49 %
50 %     /**
51 %      * Salva uma mensagem na memória de curto prazo
52 %      */
53 %     async saveMessage(msg: Message): Promise<number> {
54 %         const id = await db.messages.add(msg);
55 %         auditLog('SAVE', {
56 %             userId: msg.userId,
57 %             role: msg.role,
58 %             category: msg.category || null,
59 %             message_id: id,
60 %             memory_saved: true,
61 %         });
62 %         return id;
63 %     }

```

```

64 %
65 %      / * *
66 %      *   Salva ou atualiza um fato na memória de longo prazo
67 %      * /
68 %      async saveFact(userId: string, key: string, value: string): Promise<void> {
69 %          const existing = await db.facts
70 %              .where('userId').equals(userId)
71 %              .and(f => f.key === key)
72 %              .first();
73 %
74 %          if (existing) {
75 %              await db.facts.update(existing.id!, { value, createdAt: new Date() });
76 %              auditLog('UPDATE', {
77 %                  userId,
78 %                  fact_key: key,
79 %                  fact_value: value,
80 %                  updated: true,
81 %              });
82 %          } else {
83 %              await db.facts.add({
84 %                  userId,
85 %                  key,
86 %                  value,
87 %                  createdAt: new Date(),
88 %              });
89 %              auditLog('SAVE', {
90 %                  userId,
91 %                  fact_key: key,
92 %                  fact_value: value,
93 %                  fact_saved: true,
94 %              });
95 %          }
96 %      }
97 %
98 %      / * *
99 %      *   Extrai fatos importantes de uma mensagem do usuário automaticamente
100 %      * /
101 %      async extractAndSaveFacts(userId: string, message: string): Promise<void> {
102 %          const extracted: UserMemoryFact[] = [];
103 %
104 %          // Padrão: "meu nome é X" / "me chamo X"
105 %          const nameMatch = message.match(/(?:meu nome é|me chamo|eu me chamo) (.+)/);
106 %          if (nameMatch) {
107 %              await this.saveFact(userId, 'nome_apresentado', nameMatch[1].trim());
108 %              extracted.push({
109 %                  userId, key: 'nome_apresentado', value: nameMatch[1].trim()
110 %              });
111 %          }
112 %
113 %          // Padrão: "estou fazendo X" / "estou trabalhando em X"
114 %          const projectMatch = message.match(/(?:estou fazendo|estou trabalhando em) (.+)/);
115 %          if (projectMatch) {
116 %              await this.saveFact(userId, 'projeto_atual', projectMatch[1].trim());
117 %              extracted.push({
118 %                  userId, key: 'projeto_atual', value: projectMatch[1].trim()
119 %              });
120 %          }
121 %
122 %          // Padrão: "eu gosto de X" / "prefiro X"
123 %          const prefMatch = message.match(/(?:eu gosto de|prefiro) (.+)/);
124 %          if (prefMatch) {
125 %              await this.saveFact(userId, 'preferencia', prefMatch[1].trim());
126 %              extracted.push({
127 %                  userId, key: 'preferencia', value: prefMatch[1].trim(), createdAt: new Date()
128 %              });
129 %          }

```

```

130 %
131 %         return extracted;
132 %     }
133 %
134 %     /**
135 %      * Monta string de contexto conversacional para alimentar o ger
136 %      * Não contém respostas prontas - apenas fornece o contexto par
137 %      */
138 %     buildContextString(memory: MemoryContext): string {
139 %         const parts: string[] = [];
140 %
141 %         // Quem é o usuário
142 %         parts.push(`conversando com ${memory.userName}`);
143 %
144 %         // Fatos de longo prazo em linguagem natural
145 %         for (const fact of memory.longTerm) {
146 %             if (fact.key === 'projeto_atual') {
147 %                 parts.push(`o usuario esta trabalhando em ${fact.value}`);
148 %             } else if (fact.key === 'preferencia') {
149 %                 parts.push(`o usuario gosta de ${fact.value}`);
150 %             } else if (fact.key === 'nome_apresentado') {
151 %                 parts.push(`o nome do usuario e ${fact.value}`);
152 %             }
153 %         }
154 %
155 %         // Últimas mensagens no formato conversacional
156 %         if (memory.shortTerm.length > 0) {
157 %             const dialogues = memory.shortTerm.slice(-8).map(m => {
158 %                 const speaker = m.role === 'user' ? memory.userName : 'as
159 %                 return `${speaker}: ${m.content}`;
160 %             });
161 %             parts.push(dialogues.join(' \n '));
162 %         }
163 %
164 %         return parts.join(' \n ');
165 %     }
166 % }
167 %

```

```
1 % // =====
2 % // SAFETY & GUARDRAILS (SEGURANÇA DA IA)
3 % // Protege a IA contra injeções, vazamento de chaves/dados pessoais
4 % // e evita o treinamento com informações confidenciais.
5 % // =====
6 %
7 % export interface SafetyResult {
8 %     isSafe: boolean;
9 %     reason?: string;
10 %     redactedText: string;
11 % }
12 %
13 % export class SafetyGuardrails {
14 %     // Padrões de dados sensíveis para evitar vazamento de dados pessoais
15 %     private SENSITIVE_PATTERNS = {
16 %         apiKey: /(?:api[_]?key|secret|token|password|senha)\s*[:=]\s/,
17 %         openaiKey: /sk-[a-zA-Z0-9]{32,}/,
18 %         creditCard: /\b(?:\d[ -]*?){13,16}\b/,
19 %         cpf: /\b\d{3}\.\?\d{3}\.\?\d{3}\-\?\d{2}\b/,
20 %         email: /\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b/,
21 %         dbUri: /mongodb(?:\+srv)?(?:\/\/\w+:\w+@|postgres:\/\/\w+:\w+@|mysql:\/\/\w+:\w+@)/,
22 %     };
23 %
24 %     // Termos de toxicidade, ódio, automutilação ou requisição de morte
25 %     private DANGEROUS_TERMS = [
26 %         // Malware e Invasão (Ataques cibernéticos perigosos)
27 %         'criar malware', 'criar ransomware', 'como hackear', 'invadir',
28 %         'criar virus', 'criar vírus', 'defacer', 'ddos script', 'explorar',
29 %         'roubar credenciais', 'script de phishing', 'phishing clonar',
30 %         // Linguagem de ódio e termos extremamente abusivos
31 %         'discurso de ódio', 'hate speech', 'suicídio', 'como me matar',
32 %     ];
33 %
34 %     /**
35 %      * Sanitiza o texto de entrada contra XSS (Cross-Site Scripting)
36 %      */
37 %     public sanitize(text: string): string {
38 %         return text
39 %             .replace(/&/g, '&')
40 %             .replace(/</g, '<')
41 %             .replace(/>/g, '>')
42 %             .replace(/"/g, '"')
43 %             .replace(/'/g, ''')
44 %             .replace(/\\/g, '/');
45 %     }
46 %
47 %     /**
48 %      * Verifica se o prompt é seguro e não viola políticas de segurança
49 %      * Censura dados sensíveis para evitar que entrem no treinamento
50 %      */
51 %     public verifyPrompt(text: string): SafetyResult {
52 %         const sanitized = this.sanitize(text);
53 %         const lowerText = sanitized.toLowerCase();
54 %
55 %         // 1. Verificar termos perigosos (cybersecurity / abusos)
56 %         for (const term of this.DANGEROUS_TERMS) {
57 %             if (lowerText.includes(term)) {
58 %                 return {
59 %                     isSafe: false,
60 %                     reason: `Detecção de termo inseguro ou atividade maliciosa: ${term}`,
61 %                     redactedText: '[BLOQUEADO POR POLÍTICA DE SEGURANÇA]',
62 %                 };
63 %             }
64 %         }
65 %     }
66 % }
```



```

64 %     }
65 %
66 %     // 2. Detectar e censurar dados sensíveis (redaction) para não
67 %     let redacted = sanitized;
68 %
69 %     for (const [key, pattern] of Object.entries(this.SENSITIVE_PATTERNS)) {
70 %         if (pattern.test(redacted)) {
71 %             redacted = redacted.replace(pattern, `[DADO_SENSÍVEL_${key}]`);
72 %         }
73 %     }
74 %
75 %     return {
76 %         isSafe: true,
77 %         redactedText: redacted,
78 %     };
79 % }
80 %
81 % /**
82 %  * Modera a resposta gerada de forma probabilística para garantir
83 %  * a segurança da IA.
84 %  */
85 % public moderateOutput(text: string): string {
86 %     // Garante que a IA nunca gere termos perigosos por acidente
87 %     let moderated = text;
88 %     for (const term of this.DANGEROUS_TERMS) {
89 %         const regex = new RegExp(term, 'gi');
90 %         if (regex.test(moderated)) {
91 %             moderated = moderated.replace(regex, `[CENSURADO]`);
92 %         }
93 %     }
94 %     return moderated;
95 % }
96 %

```

```

1 % // =====
2 % // VALIDADOR DE TREINAMENTO
3 % // Decide se um texto deve ENTRAR no corpus do modelo
4 % // Filtra spam, repetições e conteúdo sem significado
5 % // =====
6 %
7 % export interface ValidationResult {
8 %     valid: boolean;
9 %     reason?: string;
10 %     score: number; // 0 a 1 (qualidade do texto)
11 % }
12 %
13 % export class TrainingValidator {
14 %     /**
15 %      * Valida se um texto vale a pena ser usado para treinar
16 %      */
17 %     validate(text: string): ValidationResult {
18 %         const trimmed = text.trim();
19 %
20 %         // Vazio
21 %         if (trimmed.length === 0) {
22 %             return { valid: false, reason: 'texto vazio', score: 0 };
23 %         }
24 %
25 %         // Muito curto (menos de 3 palavras)
26 %         const words = trimmed.split(/ /);
27 %         if (words.length < 3) {
28 %             return { valid: false, reason: 'muito curto', score: 0.1 };
29 %         }
30 %
31 %         // Spam: caracteres repetidos (kkkkkkk, aaaaa, hahahaha)
32 %         if (this.isSpam(trimmed)) {
33 %             return { valid: false, reason: 'spam ou repetição', score:
34 %         }
35 %
36 %         // Proporção de letras vs símbolos
37 %         const letterRatio = this.getLetterRatio(trimmed);
38 %         if (letterRatio < 0.5) {
39 %             return { valid: false, reason: 'muitos símbolos não-textuai
40 %         }
41 %
42 %         // Diversidade de palavras (vocabulário muito repetitivo)
43 %         const diversity = this.getWordDiversity(words);
44 %         if (diversity < 0.3 && words.length > 10) {
45 %             return { valid: false, reason: 'baixa diversidade de palavr
46 %         }
47 %
48 %         // Calcula score de qualidade
49 %         let score = 0.5;
50 %         score += Math.min(0.2, words.length / 100); // mais palavras
51 %         score += diversity * 0.2;
52 %         score += letterRatio * 0.1;
53 %         score = Math.min(1, score);
54 %
55 %         return { valid: true, score };
56 %     }
57 %
58 %     /**
59 %      * Detecta spam: kkkk, aaaa, hahaha
60 %      */
61 %     private isSpam(text: string): boolean {
62 %         // Mesmo caractere repetido 4+ vezes
63 %         if (/(\.)\1{4,}/.test(text)) return true;

```

```

64 %
65 %         // Mesma palavra repetida 3+ vezes seguidas
66 %         if (/\\b(\\w+)(\\1){2,}\\b/i.test(text)) return true;
67 %
68 %         // Apenas risadas/repetições
69 %         const onlyLaughs = /^[khaeiou\\s]+$/i.test(text) && text.length
70 %         if (onlyLaughs) return true;
71 %
72 %         return false;
73 %     }
74 %
75 %     /**
76 %     * Proporção de caracteres alfanuméricos
77 %     */
78 %     private getLetterRatio(text: string): number {
79 %         const total = text.replace(/\\s/g, '').length;
80 %         if (total === 0) return 0;
81 %         const letters = text.match(/[\\wÀ-ú]/g)?.length || 0;
82 %         return letters / total;
83 %     }
84 %
85 %     /**
86 %     * Diversidade lexical (palavras únicas / total)
87 %     */
88 %     private getWordDiversity(words: string[]): number {
89 %         if (words.length === 0) return 0;
90 %         const unique = new Set(words.map(w => w.toLowerCase()));
91 %         return unique.size / words.length;
92 %     }
93 % }
94 %

```

```
1 % import { clsx, type ClassValue } from "clsx";
2 % import { twMerge } from "tailwind-merge";
3 %
4 % export function cn(...inputs: ClassValue[]) {
5 %     return twMerge(clsx(inputs));
6 % }
7 %
```